

ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN CƠ SỞ

Nhóm 5

Xây dựng và Kiểm Thử Hệ Nhúng Đa Tiến Trình

Thời Gian Thực

Giảng viên hướng dẫn: ThS. Vũ Quang Dũng

Thành viên nhóm:

1. Nguyễn Thị Thùy Linh – 23010633.
2. Nguyễn Anh Quân – 23010375.
3. Nguyễn Ngọc Minh – 23010623.
4. Trần Thảo Vy – 23010588.

Hà Nội, 2026.

MỤC LỤC

Phân công thực hiện đồ án.....	
CHƯƠNG 1. GIỚI THIỆU.....	1
1.1. Bối cảnh đề tài	1
1.2. Lý do chọn đề tài	1
1.3. Mục tiêu của đề tài	2
1.4. Phạm vi thực hiện.....	3
1.5. Phương pháp thực hiện	3
1.6. Đối tượng kiểm thử	5
1.7. Các nhóm kiểm thử chính	7
1.8. Ý nghĩa khoa học và thực tiễn.....	7
1.9. Đóng góp của đề tài.....	8
CHƯƠNG 2 – CƠ SỞ LÝ THUYẾT	9
2.1 Embedded Systems.....	9
2.2 RTOS và FreeRTOS	10
2.3 Scheduler, Priority Scheduling và Priority Preemption	11
2.4 Concurrency và Synchronization	12
2.5 Hard Deadline, Soft Deadline và Timing Analysis	13
2.6 Test Case, Test Suite và Log Analysis	13
2.7 Tổng kết Chương 2.....	14
CHƯƠNG 3 – THIẾT KẾ HỆ THỐNG.....	16
3.1 Yêu cầu thiết kế hệ thống	16
3.2 Kiến trúc tổng thể FreeRTOS Simulator.....	17
3.3 Thiết kế các Task.....	18
3.4 Thiết kế Scheduler và Priority	20
3.5 Thiết kế Synchronization và Shared Resource	22
3.6 Thiết kế Deadline Monitoring.....	23
3.7 Thiết kế Logging và Metrics Collection	26
3.8 Thiết kế Fault Injection	27
3.9 Workflow Kiểm thử	28
3.10 Tổng kết chương.....	28
CHƯƠNG 4 – CÀI ĐẶT HỆ THỐNG	30
4.1 Môi trường cài đặt	30
4.2 Cài đặt cấu trúc chương trình	30

4.3 Cài đặt tác vụ và ưu tiên.....	31
4.4 Cài đặt đồng bộ.....	33
4.5 Cài đặt Deadline Monitor, Logger và Fault Injector.....	33
4.6 Tổng kết chương.....	34
CHƯƠNG 5 – THIẾT KẾ KIỂM THỬ	35
5.1 Chiến lược kiểm thử.....	35
5.2 Quy trình kiểm thử	35
5.3 Môi trường kiểm thử	36
5.4 Runtime Metrics và Tiêu chí Pass/Fail	38
5.5 Scheduler Testing.....	40
5.5.1 SCH-01 — Sensor Kiểm Soát Người Ra Vào	41
5.5.2 SCH-02 — Trạm Quan Trắc Thời Tiết	42
5.5.3 SCH-03 — Hệ Thống Ghi Log Runtime.....	43
5.6 Priority Scheduling và Priority Preemption Testing.....	44
5.6.1 PRI-01 —Camera An Ninh Phát Hiện Xâm Nhập.....	44
5.6.2 PRI-02 — Drone Tránh Chướng Ngại Vật	45
5.6.3 PRE-01 — Nhà Máy Có Emergency Stop	46
5.7 Hard Deadline và Soft Deadline Testing.....	47
5.7.1 HD-01 — Hệ Thống Báo Cháy Thông Minh.....	48
5.7.2 HD-02 — Nhà Máy Có Emergency Stop	49
5.7.3 SD-01 — Trạm Quan Trắc Thời Tiết	49
5.7.4 SD-02 — Hệ Thống Giám Sát Chất Lượng Không Khí.....	50
5.8 Đồng bộ và Concurrency Testing	52
5.8.1 SYN-01 — Hệ Thống Ghi Log Nhà Thông Minh	52
5.8.2 SYN-02 — Camera An Ninh Ghi Log Đồng Thời	53
5.8.3 CON-01 — Drone Xử Lý Nhiều Cảm Biến	54
5.8.4 CON-02 — Nhà Máy Nhiều Tín Hiệu Điều Khiển	55
5.9 Fault Injection Testing.....	56
5.9.1 FLT-01 — CPU Overload Trong Nhà Máy	57
5.9.2 FLT-02 — Queue Blocking Khi Truyền Dữ Liệu.....	58
5.9.3 FLT-03 — Artificial Delay Trong Drone Control	59
5.9.4 FLT-04 — Sensor Nhiễu Dữ Liệu	60
5.10 Timing Analysis Testing	61
5.10.1 TIM-01 — Phân Tích Response Time của IntrusionAlert	62

5.10.2 TIM-02 — Phân Tích WCET của Navigation / CriticalWork	63
5.10.3 TIM-03 — Phân Tích Context Switch khi Preemption	64
5.10.4 TIM-04 — Phân Tích CPU Usage dưới Tải Cao	65
5.11 Tổng hợp bộ Test.....	66
5.12 Tổng kết chương.....	68
CHƯƠNG 6: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	69
6.1 Môi trường và kịch bản thực nghiệm.....	69
6.1.1. Môi trường thực nghiệm	69
6.1.2. Ảnh xạ Task Logic và Task Triển khai	70
6.1.3 Kịch bản thực nghiệm	71
6.1.4. Quy trình thực nghiệm	72
6.2. Test Execution Matrix	73
6.3 Kịch bản Camera An Ninh Phát Hiện Xâm Nhập.....	74
6.4 Kịch bản Nhà Máy Có Emergency Stop	77
6.5 Kịch bản Hệ Thống Báo Cháy Thông Minh.....	78
6.6 Kịch bản Trạm Quan Trắc Thời Tiết	79
6.7 Kịch bản Fault Injection.....	80
6.8 Đánh giá Timing Analysis	82
6.9. Đánh giá tổng hợp Pass/Fail.....	85
6.10. Tổng kết chương.....	86
CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	88
7.1. Kết quả đạt được	88
7.2. Đánh giá mức độ hoàn thành mục tiêu đề tài.....	88
7.3. Hạn chế của đề tài	89
7.4. Hướng phát triển.....	89
7.5. Tổng kết chương.....	90
TÀI LIỆU THAM KHẢO.....	

Phân công thực hiện đồ án

Mã sinh viên	Họ và tên	Lớp	Nhiệm vụ
23010633	Nguyễn Thị Thuỳ Linh	CSE702131-1-2-25(N01)	Phân tích yêu cầu đề tài, thiết kế kiến trúc hệ thống, cài đặt FreeRTOS Windows Simulator, xây dựng Scheduler, Priority Scheduling, Priority Preemption, thiết kế Task Management, xây dựng Test Suite, tích hợp Runtime Logging, thực hiện Scheduler Testing, Priority Scheduling Testing, Priority Preemption Testing, tổng hợp kết quả và biên soạn các Chương 1, 2, 3, 4, chỉnh sửa báo cáo cuối cùng.
23010588	Trần Thảo Vy	CSE702131-1-2-25(N01)	Nghiên cứu và triển khai Synchronization (Queue, Mutex, Semaphore), Shared Resource Management, xây dựng các kịch bản Synchronization Testing và Concurrency Testing, thực hiện Test 06 và Test 07, hỗ trợ hoàn thiện Chương 5.
23010375	Nguyễn Anh Quân	CSE702131-1-2-25(N01)	Xây dựng cơ chế Deadline Monitoring, Timing Analysis, WCET, Response Time Measurement, Fault Injection, thực hiện Hard Deadline Testing, Soft Deadline Testing, Fault Injection Testing và Timing Analysis Testing (Test 04, 05, 08, 09), hỗ trợ thu thập dữ liệu runtime.
23010623	Nguyễn Ngọc Minh	CSE702131-1-2-25(N01)	Xây dựng công cụ phân tích Runtime Log bằng Python, xử lý dữ liệu Metrics, tạo biểu đồ trực quan, tổng hợp số liệu thực nghiệm, hỗ trợ viết Chương 6, Chương 7.

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh đề tài

Trong các hệ thống nhúng hiện đại, nhiều tác vụ thường phải hoạt động đồng thời để xử lý cảm biến, điều khiển thiết bị, truyền thông dữ liệu, ghi log và phản ứng với các sự kiện khẩn cấp. Khác với phần mềm ứng dụng thông thường, hệ thống nhúng thường bị giới hạn về tài nguyên xử lý, bộ nhớ và đặc biệt phải đáp ứng các yêu cầu thời gian thực. Một tác vụ không chỉ cần cho ra kết quả đúng, mà còn phải hoàn thành trong khoảng thời gian cho phép.

Ví dụ, trong hệ thống báo cháy thông minh, tác vụ phát hiện cháy và kích hoạt cảnh báo phải được xử lý trong thời gian rất ngắn. Nếu tác vụ này bị trì hoãn bởi các tác vụ ít quan trọng hơn như ghi log hoặc gửi dữ liệu định kỳ, hệ thống có thể gây ra hậu quả nghiêm trọng. Tương tự, trong nhà máy có Emergency Stop, khi có tín hiệu dừng khẩn cấp, tác vụ điều khiển an toàn phải được ưu tiên xử lý ngay lập tức.

Vì vậy, việc kiểm thử hệ thống nhúng đa tiến trình thời gian thực không chỉ dừng lại ở kiểm thử chức năng, mà còn phải kiểm thử các đặc tính phi chức năng như thời gian phản hồi, độ trễ, khả năng đáp ứng deadline, khả năng lập lịch, cơ chế ưu tiên, preemption và đồng bộ hóa tài nguyên dùng chung.

Đề tài “Xây dựng và kiểm thử hệ nhúng đa tiến trình thời gian thực” được thực hiện nhằm xây dựng một môi trường mô phỏng hệ thống nhúng sử dụng FreeRTOS Simulator, từ đó thiết kế và thực hiện các test case để đánh giá hành vi của hệ thống trong các tình huống thực tế.

1.2. Lý do chọn đề tài

FreeRTOS là một hệ điều hành thời gian thực nhẹ, được sử dụng phổ biến trong các hệ thống nhúng. FreeRTOS cung cấp các cơ chế quan trọng như task scheduling, priority scheduling, preemption, semaphore, mutex, queue và delay theo tick hệ thống. Đây là nền tảng phù hợp để mô phỏng và kiểm thử hệ thống nhúng đa tiến trình.

Đề tài được lựa chọn vì các lý do sau:

Thứ nhất, đề tài giúp sinh viên hiểu rõ bản chất của hệ thống thời gian thực. Trong hệ thống thời gian thực, kết quả đúng nhưng đến muộn vẫn có thể bị xem là sai. Do đó, việc đo thời gian phản hồi, deadline miss, latency và jitter là yêu cầu quan trọng.

Thứ hai, đề tài giúp kiểm chứng cơ chế lập lịch của RTOS. Thông qua các test case như “Camera An Ninh Phát Hiện Xâm Nhập” hoặc “Nhà Máy Có Emergency Stop”, nhóm có thể đánh giá task ưu tiên cao có được scheduler cấp CPU đúng lúc hay không.

Thứ ba, đề tài giúp làm rõ vấn đề đồng thời và đồng bộ hóa trong hệ thống đa task. Khi nhiều task cùng truy cập tài nguyên dùng chung, hệ thống có thể xuất hiện race

condition, deadlock hoặc priority inversion. Các lỗi này khó phát hiện nếu chỉ kiểm thử chức năng thông thường.

Thứ tư, đề tài tạo cơ hội áp dụng kỹ thuật kiểm thử phần mềm vào hệ thống nhúng. Thay vì chỉ viết chương trình chạy được, nhóm cần thiết kế test case, thu thập log, phân tích kết quả và đưa ra kết luận pass/fail dựa trên dữ liệu đo đạc.

1.3. Mục tiêu của đề tài

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống mô phỏng đa tiến trình thời gian thực sử dụng FreeRTOS Simulator và thực hiện kiểm thử các cơ chế lập lịch, ưu tiên, preemption, deadline, đồng bộ hóa và xử lý lỗi.

Bảng 1.1. Mục tiêu của đề tài

Mục tiêu	Mô tả chi tiết	Ví dụ kịch bản
Xây dựng hệ thống FreeRTOS đa task	Nhiều task chạy đồng thời, được cấu hình mức ưu tiên (priority), chu kỳ thực thi (period) và deadline riêng biệt.	DoorSensor, Navigation, LogConsumer, IntrusionAlert
Kiểm thử Scheduler và Priority Scheduling	Đánh giá khả năng lập lịch của FreeRTOS, đảm bảo task có mức ưu tiên cao được thực thi trước task có mức ưu tiên thấp.	Camera An Ninh Phát Hiện Xâm Nhập, Nhà Máy Emergency Stop
Phân tích Deadline và Timing Metrics	Kiểm thử Hard Deadline và Soft Deadline; đo Execution Time, Response Time, Deadline Miss Rate và Worst-Case Execution Time (WCET).	Hệ Thống Báo Cháy Thông Minh, Trạm Quan Trắc Thời Tiết
Kiểm thử Concurrency và Synchronization	Đánh giá việc truy cập tài nguyên dùng chung thông qua Queue, Mutex và Semaphore nhằm tránh race condition và deadlock.	Hệ Thống Ghi Log Nhà Thông Minh, Drone Tránh Chướng Ngại Vật
Fault Injection Testing	Chủ động tạo lỗi như CPU Overload, Deadline Miss hoặc Task Delay để đánh giá khả năng phát hiện và xử lý lỗi của hệ thống.	CPU Overload, Critical Workload

Thu thập Log và Metrics	Ghi nhận các thông tin như Timestamp, Task Name, Priority, Event, Execution Time, Response Time và Status của từng task.	Tất cả Test Case
Pass/Fail Evaluation	So sánh kết quả thực tế với tiêu chí mong đợi để xác định trạng thái PASS hoặc FAIL cho từng test case và toàn bộ test suite.	Toàn bộ Test Suite

Trong phạm vi thực nghiệm, hệ thống hiện thực 9 nhóm kiểm thử chính gồm Scheduler Testing, Priority Scheduling, Priority Preemption, Hard Deadline, Soft Deadline, Synchronization, Concurrency, Fault Injection và Timing Analysis. Các nhóm kiểm thử này được triển khai trong chương trình kiểm thử FreeRTOS nhằm đánh giá toàn diện hành vi của hệ thống nhúng đa tiến trình thời gian thực.

1.4. Phạm vi thực hiện

Phạm vi đề tài tập trung vào kiểm thử hệ thống nhúng đa tiến trình thời gian thực trong môi trường mô phỏng FreeRTOS trên máy tính cá nhân. Đề tài không tập trung xây dựng web application, AI application hoặc hệ thống quản lý dữ liệu phức tạp. Các công cụ phụ trợ như script phân tích log hoặc biểu đồ chỉ đóng vai trò hỗ trợ cho quá trình kiểm thử.

Phạm vi kỹ thuật bao gồm:

- FreeRTOS Simulator.
- Ngôn ngữ lập trình C.
- Môi trường Windows hoặc Linux.
- VSCode hoặc Visual Studio.
- Các task mô phỏng hệ thống nhúng.
- Mutex, semaphore, queue.
- Logger ghi nhận timestamp và trạng thái task.
- Deadline monitor để phát hiện deadline miss.
- Fault injector để tạo lỗi có kiểm soát.
- File log hoặc CSV để phân tích kết quả.

Các nội dung không thuộc trọng tâm đề tài:

- Không xây dựng hệ thống web quản lý quy mô lớn.
- Không phát triển ứng dụng AI.
- Không xây dựng dashboard phức tạp nếu chưa hoàn thiện phần kiểm thử.
- Không tập trung vào giao diện người dùng.

1.5. Phương pháp thực hiện

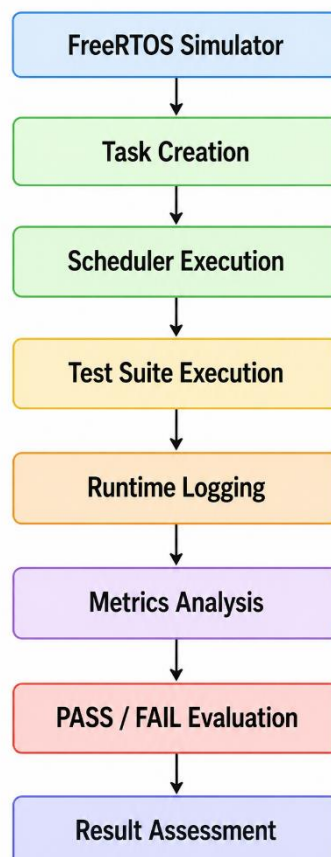
Đề tài được thực hiện theo phương pháp thực nghiệm kết hợp phân tích học thuật nhằm đánh giá hành vi của hệ thống nhúng đa tiến trình thời gian thực trong môi trường mô phỏng FreeRTOS.

Trước hết, nhóm nghiên cứu cơ sở lý thuyết liên quan đến hệ thống nhúng, RTOS, FreeRTOS, scheduler, priority scheduling, priority preemption, concurrency, synchronization và deadline analysis. Đây là nền tảng quan trọng để thiết kế kiến trúc hệ thống, xây dựng task và xác định các tiêu chí kiểm thử phù hợp.

Tiếp theo, nhóm xây dựng hệ thống FreeRTOS Simulator gồm nhiều task có mức priority, period, WCET và deadline khác nhau. Các task được thiết kế nhằm mô phỏng các thành phần thường gặp trong hệ thống nhúng thực tế như cảm biến, điều khiển, truyền thông, ghi log và xử lý sự kiện khẩn cấp.

Sau giai đoạn xây dựng hệ thống, nhóm tiến hành thiết kế bộ test case theo các nhóm kiểm thử chính gồm scheduler testing, priority scheduling, priority preemption, hard deadline, soft deadline, synchronization, concurrency, fault injection và timing analysis. Mỗi test case đều được xây dựng với mục tiêu rõ ràng, bao gồm lý do thực hiện, giá trị kiểm thử, phương pháp đo đạc, kết quả mong đợi và tiêu chí đánh giá pass/fail.

Quy trình thực hiện kiểm thử của đề tài được mô tả trong Hình 1.1.



Hình 1.1. Quy trình thực hiện kiểm thử hệ thống nhúng đa tiến trình thời gian thực.

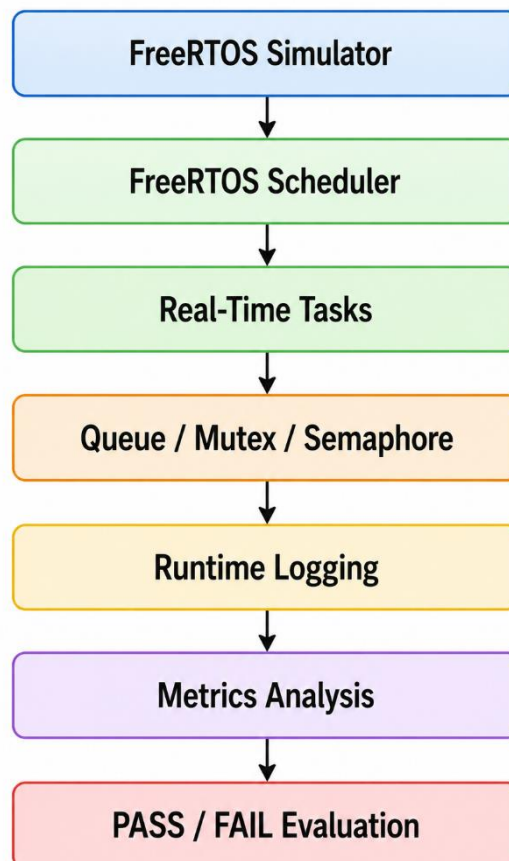
Trong quá trình chạy kiểm thử, hệ thống ghi nhận dữ liệu runtime thông qua cơ chế logging theo timestamp. Các metrics được thu thập bao gồm execution time, response time, deadline miss, context switch count và Worst-Case Execution Time (WCET). Dữ liệu thu được được sử dụng để xây dựng bảng kết quả, biểu đồ phân tích và đánh giá hiệu năng hệ thống.

Cuối cùng, nhóm tiến hành phân tích kết quả thực nghiệm dựa trên dữ liệu đo đạc thực tế nhằm đánh giá tính đúng đắn của scheduler, cơ chế ưu tiên, khả năng đáp ứng deadline, hiệu quả đồng bộ hóa và khả năng xử lý lỗi của hệ thống. Từ đó, nhóm đưa ra kết luận, chỉ ra hạn chế và đề xuất hướng phát triển trong tương lai.

1.6. Đối tượng kiểm thử

Đối tượng kiểm thử trong đề tài là hệ thống FreeRTOS mô phỏng nhiều task chạy đồng thời trên môi trường FreeRTOS Simulator. Hệ thống được xây dựng nhằm kiểm thử các cơ chế scheduler, priority scheduling, priority preemption, synchronization, deadline handling, fault injection và timing analysis trong hệ thống nhúng đa tiến trình thời gian thực.

Kiến trúc tổng quan của hệ thống kiểm thử được trình bày trong Hình 1.2.



Hình 1.2. Kiến trúc tổng quan hệ thống kiểm thử FreeRTOS Simulator.

Hệ thống bao gồm lớp task thời gian thực, lớp đồng bộ hóa tài nguyên, lớp ghi nhận dữ liệu runtime và lớp đánh giá kết quả kiểm thử. Trong quá trình thực hiện, các

task được tạo và quản lý bởi FreeRTOS Scheduler với các mức ưu tiên khác nhau nhằm mô phỏng hành vi của hệ thống nhúng thực tế.

Bảng 1.2. Các thành phần chính của hệ thống kiểm thử

Thành phần	Vai trò
FreeRTOS Scheduler	Quản lý trạng thái của các task (Ready, Running, Blocked, Suspended) và phân phối CPU theo cơ chế lập lịch thời gian thực.
Real-Time Tasks	Mô phỏng các thành phần chức năng của hệ thống nhúng, thực hiện các tác vụ theo chu kỳ hoặc theo sự kiện.
Queue	Hỗ trợ trao đổi dữ liệu giữa các task theo cơ chế FIFO, đảm bảo tính toàn vẹn dữ liệu trong môi trường đa nhiệm.
Mutex	Bảo vệ tài nguyên dùng chung, ngăn ngừa race condition khi nhiều task cùng truy cập một tài nguyên.
Semaphore	Đồng bộ hóa hoạt động giữa các task hoặc giữa task và sự kiện hệ thống.
Runtime Logging	Ghi nhận thông tin thực thi của hệ thống như timestamp, task name, priority, event và trạng thái hoạt động.
Metrics Analysis	Phân tích Execution Time, Response Time, WCET, Deadline Miss và Context Switch Count.
PASS / FAIL Evaluation	Đánh giá kết quả kiểm thử dựa trên các tiêu chí mong đợi để xác định trạng thái PASS hoặc FAIL của từng test case.

Ngoài các task hệ thống nêu trên, đề tài còn xây dựng các kịch bản kiểm thử thực tế như Sensor Kiểm Soát Người Ra Vào, Camera An Ninh Phát Hiện Xâm Nhập, Hệ Thống Báo Cháy Thông Minh, Trạm Quan Trắc Thời Tiết, Hệ Thống Ghi Log Nhà Thông Minh, Drone Tránh Chướng Ngại Vật và CPU Overload. Các kịch bản này được triển khai thông qua các task trong hệ thống nhằm đánh giá hành vi của scheduler, cơ chế ưu tiên, deadline và đồng bộ hóa tài nguyên.

Việc thiết kế các task trên giúp hệ thống có đủ điều kiện để kiểm thử các đặc trưng quan trọng của hệ nhúng thời gian thực như context switching, priority scheduling, priority preemption, deadline miss detection, queue communication, mutex protection và fault injection.

1.7. Các nhóm kiểm thử chính

Bộ kiểm thử của đề tài được chia thành các nhóm chính sau:

Nhóm kiểm thử	Mục tiêu	Test case điển hình
Scheduler Testing	Kiểm tra task được lập lịch đúng	Sensor Kiểm Soát Người Ra Vào, Trạm Quan Trắc Thời Tiết
Priority Scheduling	Task priority cao chạy trước	Camera An Ninh Phát Hiện Xâm Nhập
Priority Preemption	Task khẩn cấp chiếm CPU ngay	Nhà Máy Có Emergency Stop
Hard Deadline	Task quan trọng hoàn thành đúng thời hạn	Hệ Thống Báo Cháy Thông Minh
Soft Deadline	Task ít quan trọng hoàn thành chậm vẫn chấp nhận	Trạm Quan Trắc Thời Tiết
Synchronization	Bảo vệ tài nguyên dùng chung	Hệ Thống Ghi Log Nhà Thông Minh
Concurrency	Phát hiện race condition, deadlock	Robot Công Nghiệp Tránh Va Chạm
Fault Injection	Kiểm tra khả năng phát hiện Deadline Miss do tải hệ thống tăng cao	CPU Overload, Deadline Miss Detection
Timing Analysis	Đo latency, jitter, response time	Drone Tránh Chướng Ngại Vật
Pass/Fail Evaluation	Đánh giá kết quả	Tất cả test case

1.8. Ý nghĩa khoa học và thực tiễn

Về mặt khoa học, đề tài giúp làm rõ mối quan hệ giữa lý thuyết hệ điều hành thời gian thực và kiểm thử phần mềm. Các khái niệm như priority scheduling, preemption, context switching, mutex, semaphore, hard deadline và soft deadline không chỉ được trình bày ở mức lý thuyết, mà còn được kiểm chứng thông qua dữ liệu thực nghiệm.

Về mặt thực tiễn, đề tài mô phỏng các tình huống có thể xuất hiện trong hệ thống nhúng thực tế. Ví dụ, trong “Hệ Thống Báo Cháy Thông Minh”, deadline miss có thể dẫn đến cảnh báo chậm. Trong “Drone Tránh Chướng Ngại Vật”, response time cao có thể khiến hệ thống phản ứng không kịp. Trong “Hệ Thống Ghi Log Nhà Thông Minh”, việc nhiều task cùng ghi log có thể gây race condition nếu không có mutex.

Do đó, kết quả của đề tài không chỉ chứng minh hệ thống chạy được, mà còn chứng minh nhóm có khả năng thiết kế kiểm thử, đo đạc, phân tích và đánh giá hành vi của hệ thống nhúng thời gian thực.

1.9. Đóng góp của đề tài

Đề tài có các đóng góp chính như sau:

- Xây dựng môi trường mô phỏng hệ thống nhúng đa tiến trình thời gian thực trên nền tảng FreeRTOS Simulator.
- Thiết kế và triển khai bộ kiểm thử gồm 9 nhóm kiểm thử chính.
- Xây dựng cơ chế Runtime Logging phục vụ thu thập dữ liệu thực thi.
- Phân tích các chỉ số thời gian thực như Execution Time, Response Time, WCET và Deadline Miss.
- Đánh giá kết quả PASS/FAIL dựa trên dữ liệu thực nghiệm.
- Góp phần minh họa việc áp dụng kỹ thuật kiểm thử phần mềm trong môi trường hệ thống nhúng thời gian thực.

Từ các yêu cầu, mục tiêu và phạm vi đã trình bày trong Chương 1, việc nghiên cứu các cơ sở lý thuyết về hệ thống nhúng, hệ điều hành thời gian thực FreeRTOS, cơ chế lập lịch, đồng bộ hóa và phân tích thời gian là cần thiết. Do đó, Chương 2 sẽ trình bày các kiến thức nền tảng phục vụ cho việc thiết kế, triển khai và kiểm thử hệ thống trong các chương tiếp theo.

CHƯƠNG 2 – CƠ SỞ LÝ THUYẾT

Chương này trình bày các kiến thức nền tảng phục vụ trực tiếp cho việc xây dựng và kiểm thử hệ thống nhúng đa tiến trình thời gian thực sử dụng FreeRTOS Simulator. Nội dung chương tập trung vào các khái niệm về Embedded Systems, RTOS, FreeRTOS, scheduler, priority scheduling, synchronization, deadline handling và timing analysis. Các nội dung này được liên kết trực tiếp với hệ thống, task và test case đã giới thiệu trong Chương 1 nhằm tạo cơ sở cho quá trình thiết kế, cài đặt, kiểm thử và đánh giá thực nghiệm trong các chương tiếp theo.

2.1 Embedded Systems

Trong các hệ thống kỹ thuật hiện đại, nhiều thiết bị không còn chỉ thực hiện một chức năng đơn lẻ mà phải xử lý đồng thời nhiều nhiệm vụ với yêu cầu phản hồi nghiêm ngặt về thời gian. Những hệ thống này thường được triển khai dưới dạng hệ thống nhúng, nơi khả năng tính toán, bộ nhớ và năng lượng đều bị giới hạn nhưng vẫn phải đảm bảo tính đúng hạn trong quá trình vận hành.

Hệ thống nhúng (Embedded System) là hệ thống máy tính chuyên dụng được tích hợp vào một thiết bị hoặc sản phẩm cụ thể nhằm thực hiện các chức năng xác định trong môi trường vận hành thực tế. Khác với máy tính đa năng, hệ thống nhúng thường được thiết kế để thực hiện một nhóm nhiệm vụ chuyên biệt với yêu cầu cao về độ tin cậy, tính ổn định và khả năng đáp ứng thời gian thực.

Trong các hệ thống nhúng hiện đại, nhiều tác vụ phải hoạt động đồng thời để thực hiện các chức năng như thu thập dữ liệu cảm biến, xử lý tín hiệu điều khiển, truyền thông dữ liệu, ghi nhận trạng thái hệ thống và phản hồi các sự kiện bất thường. Khi số lượng tác vụ gia tăng cùng với sự khác biệt về mức độ ưu tiên và yêu cầu thời gian xử lý, mô hình thực thi tuần tự truyền thống không còn đảm bảo hiệu năng vận hành. Điều này dẫn đến nhu cầu sử dụng các cơ chế quản lý đa nhiệm, lập lịch thực thi và điều phối tài nguyên trong môi trường thời gian thực.

Để giải quyết bài toán đa nhiệm và kiểm soát thời gian đáp ứng trong hệ thống nhúng, các hệ điều hành thời gian thực (RTOS) thường được sử dụng như một lớp trung gian chịu trách nhiệm quản lý scheduler, task execution, synchronization và timing control. Đây cũng là định hướng kỹ thuật được sử dụng trong đề tài nhằm xây dựng môi trường kiểm thử hệ thống nhúng đa tiến trình thời gian thực.

Như đã trình bày trong Chương 1, các kịch bản ứng dụng được lựa chọn cho đồ án như Drone Tránh Chướng Ngại Vật, Camera An Ninh Phát Hiện Xâm Nhập, Sensor Kiểm Soát Người Ra Vào, Hệ Thống Báo Cháy Thông Minh và Trạm Quan Trắc Thời Tiết đều yêu cầu xử lý đồng thời nhiều tác vụ với các ràng buộc thời gian khác nhau. Trong các hệ thống này, dữ liệu cảm biến phải được cập nhật liên tục, tác vụ điều khiển phải phản hồi trong giới hạn thời gian xác định, trong khi các nhiệm vụ phụ trợ như truyền thông hoặc ghi log vẫn cần duy trì hoạt động song song. Sự đồng thời của nhiều tác vụ tạo ra bài toán về scheduler, priority scheduling, concurrency, synchronization và deadline handling.

Trong phạm vi đồ án, nhóm sử dụng FreeRTOS Simulator để tái tạo môi trường vận hành của hệ thống nhúng đa tiến trình thời gian thực. Hệ thống được xây dựng gồm nhiều nhóm task như task cảm biến, task điều khiển, task truyền thông, task ghi log và task xử lý sự kiện khẩn cấp. Các task này được cấu hình với mức ưu tiên khác nhau nhằm mô phỏng các thành phần thường gặp trong hệ thống nhúng thực tế.

Việc phân chia các task theo mức độ ưu tiên khác nhau cho phép đánh giá trực tiếp cơ chế scheduler, priority scheduling, priority preemption, synchronization và timing analysis trong môi trường FreeRTOS Simulator. Đồng thời, các task này cũng là nền tảng để triển khai các nhóm kiểm thử như Scheduler Testing, Priority Scheduling, Priority Preemption, Hard Deadline, Soft Deadline, Synchronization, Concurrency, Fault Injection và Timing Analysis.

Thông qua môi trường mô phỏng này, đề tài có thể quan sát trực tiếp hành vi của scheduler, khả năng preemption của các task ưu tiên cao, hiệu quả của cơ chế đồng bộ tài nguyên và mức độ đáp ứng deadline của từng nhiệm vụ. Dữ liệu runtime thu thập được tiếp tục được sử dụng cho timing analysis nhằm đánh giá execution time, response time, WCET, deadline miss và context switch count của hệ thống.

Bên cạnh bài toán lập lịch, việc nhiều task cùng truy cập tài nguyên dùng chung cũng đặt ra yêu cầu về synchronization và concurrency control. Nếu không có chiến lược quản lý phù hợp, hệ thống có thể xuất hiện race condition, deadlock hoặc priority inversion, làm suy giảm tính ổn định và khả năng đáp ứng thời gian thực. Những vấn đề này tạo nền tảng lý thuyết cho các mục tiếp theo về RTOS, FreeRTOS, Scheduler và Synchronization.

2.2 RTOS và FreeRTOS

2.2. RTOS và FreeRTOS

Như đã trình bày trong Chương 1, các kịch bản kiểm thử của đề tài đều đặt ra yêu cầu xử lý đồng thời nhiều tác vụ với các ràng buộc thời gian khác nhau. Trong hệ thống nhúng đa nhiệm, việc nhiều task cùng tồn tại với mức độ ưu tiên và yêu cầu thời gian thực riêng biệt đòi hỏi một cơ chế quản lý thực thi có khả năng điều phối tài nguyên xử lý theo cách có thể dự đoán được. Đây chính là vai trò của hệ điều hành thời gian thực (Real-Time Operating System – RTOS).

RTOS là hệ điều hành được thiết kế nhằm đảm bảo các tác vụ quan trọng được thực hiện trong khoảng thời gian xác định. Khác với các hệ điều hành đa nhiệm thông thường, RTOS không chỉ quan tâm đến tính đúng đắn của kết quả xử lý mà còn chú trọng đến thời điểm kết quả được tạo ra. Trong nhiều ứng dụng thời gian thực, một kết quả đúng nhưng được đưa ra quá muộn vẫn bị xem là không đạt yêu cầu.

RTOS đóng vai trò như một lớp quản lý trung gian, chịu trách nhiệm phân bổ CPU, quản lý trạng thái task và điều phối quá trình chuyển đổi thực thi giữa các nhiệm vụ đang hoạt động. Các trạng thái như Ready, Running, Blocked và Suspended cho phép hệ thống kiểm soát vòng đời của task, trong khi cơ chế lập lịch đảm bảo tài nguyên xử lý được cấp phát phù hợp với mức priority và deadline của từng nhiệm vụ.

Trong số các RTOS hiện nay, FreeRTOS được sử dụng rộng rãi trong các hệ thống nhúng nhờ kiến trúc gọn nhẹ, khả năng cấu hình linh hoạt và hỗ trợ đầy đủ các cơ chế quản lý task, synchronization và inter-task communication. FreeRTOS cung cấp nhiều API cho phép xây dựng môi trường thực thi đa nhiệm có kiểm soát, phù hợp với yêu cầu kiểm thử scheduler, deadline handling, synchronization và concurrency của đề tài.

Sử dụng FreeRTOS Windows PC Simulator, nhóm triển khai hệ thống kiểm thử mà không phụ thuộc trực tiếp vào phần cứng thực. Môi trường mô phỏng này cho phép quan sát hành vi của scheduler, thu thập log runtime và đánh giá các chỉ số như Execution Time, Response Time, WCET, Deadline Miss và Context Switch Count.

Bên cạnh khả năng quản lý task, FreeRTOS còn cung cấp các cơ chế đồng bộ hóa như Mutex, Semaphore và Queue. Đây là các thành phần được sử dụng trực tiếp trong đồ án để kiểm thử synchronization và inter-task communication. Thông qua các cơ chế này, hệ thống có thể bảo vệ tài nguyên dùng chung, trao đổi dữ liệu giữa các task và hạn chế các lỗi phát sinh do truy cập đồng thời.

Thông qua việc triển khai FreeRTOS Simulator, nhóm xác định rõ mối quan hệ giữa khái niệm RTOS, cơ chế Scheduler, Priority Scheduling, Priority Preemption và các kịch bản kiểm thử thực tế. Từ góc độ đề tài, FreeRTOS không chỉ đóng vai trò môi trường thực thi mà còn là nền tảng để khảo sát hành vi của hệ thống nhúng đa tiến trình thời gian thực.

2.3 Scheduler, Priority Scheduling và Priority Preemption

Trong hệ điều hành thời gian thực, Scheduler là thành phần chịu trách nhiệm quản lý việc thực thi các task và phân phối CPU cho các nhiệm vụ đang ở trạng thái Ready. Scheduler quyết định task nào được thực thi dựa trên mức độ ưu tiên, trạng thái hoạt động và chính sách lập lịch của hệ thống. Hiệu quả của Scheduler ảnh hưởng trực tiếp đến khả năng đáp ứng deadline, thời gian phản hồi và tính ổn định của hệ thống nhúng.

FreeRTOS sử dụng cơ chế Priority Scheduling, trong đó task có mức priority cao hơn sẽ được ưu tiên thực thi trước task có priority thấp hơn khi cùng ở trạng thái Ready. Cơ chế này giúp đảm bảo các nhiệm vụ quan trọng được xử lý đúng thời điểm và phù hợp với yêu cầu của hệ thống thời gian thực.

Trong môi trường hệ thống nhúng, các tác vụ thường được phân chia thành nhiều mức ưu tiên khác nhau như task cảm biến, task điều khiển, task truyền thông, task xử lý khẩn cấp hoặc task ghi log. Việc phân chia mức ưu tiên cho phép hệ thống tập trung tài nguyên xử lý vào các nhiệm vụ quan trọng hơn khi xuất hiện cạnh tranh CPU.

Gắn liền với Priority Scheduling là cơ chế Priority Preemption. Khi một task có priority cao chuyển sang trạng thái Ready, Scheduler sẽ tạm dừng task đang chạy để cấp CPU cho task ưu tiên cao hơn. Đây là cơ chế quan trọng giúp hệ thống phản ứng nhanh với các sự kiện khẩn cấp và đảm bảo các nhiệm vụ quan trọng được xử lý đúng thời điểm.

Trong phạm vi đồ án, cơ chế Priority Scheduling và Priority Preemption được sử dụng làm nền tảng để xây dựng các nhóm kiểm thử liên quan đến điều phối CPU và xử lý sự kiện khẩn cấp. Các kịch bản như Camera An Ninh Phát Hiện Xâm Nhập hoặc Nhà Máy Emergency Stop được sử dụng để đánh giá khả năng ưu tiên và chiếm quyền thực thi của các task quan trọng.

Thông qua FreeRTOS Simulator, nhóm có thể quan sát trực tiếp quá trình lập lịch, chuyển đổi ngữ cảnh và cơ chế ưu tiên của các task. Các kết quả này được sử dụng để đánh giá tính đúng đắn của Scheduler cũng như hiệu năng của hệ thống thông qua các chỉ số như Response Time, Context Switch Count, Deadline Miss và WCET.

Những cơ chế này đóng vai trò nền tảng cho các nội dung tiếp theo liên quan đến Synchronization, Deadline Handling, Timing Analysis và đánh giá Pass/Fail trong quá trình kiểm thử hệ thống nhúng đa tiến trình thời gian thực.

2.4 Concurrency và Synchronization

Trong hệ thống nhúng đa tiến trình, nhiều task có thể hoạt động đồng thời và cùng truy cập các tài nguyên dùng chung như biến hệ thống, vùng nhớ chung hoặc các thiết bị ngoại vi. Quá trình nhiều task cùng tồn tại và thực thi trong cùng một khoảng thời gian được gọi là Concurrency (tính đồng thời).

Concurrency giúp nâng cao khả năng đáp ứng của hệ thống bằng cách cho phép nhiều nhiệm vụ được xử lý song song về mặt logic. Tuy nhiên, việc nhiều task cùng truy cập tài nguyên dùng chung cũng có thể gây ra các vấn đề như Race Condition, Deadlock hoặc Priority Inversion nếu không có cơ chế quản lý phù hợp.

Race Condition xảy ra khi kết quả của chương trình phụ thuộc vào thứ tự thực thi của các task truy cập cùng một tài nguyên. Deadlock xảy ra khi hai hoặc nhiều task chờ lẫn nhau và không thể tiếp tục thực thi. Priority Inversion xuất hiện khi một task ưu tiên cao phải chờ một task ưu tiên thấp đang giữ tài nguyên dùng chung.

Để giải quyết các vấn đề trên, FreeRTOS cung cấp các cơ chế Synchronization như Mutex, Semaphore và Queue. Các cơ chế này cho phép điều phối truy cập tài nguyên dùng chung, đồng bộ hóa sự kiện giữa các task và hỗ trợ trao đổi dữ liệu trong môi trường đa nhiệm.

Mutex được sử dụng để bảo vệ tài nguyên dùng chung, đảm bảo tại một thời điểm chỉ có một task được phép truy cập tài nguyên. Semaphore được sử dụng để đồng bộ hóa hoạt động giữa các task hoặc giữa task với sự kiện bên ngoài. Queue cho phép truyền dữ liệu giữa các task theo cơ chế FIFO, giúp giảm sự phụ thuộc trực tiếp giữa các thành phần trong hệ thống.

Trong phạm vi đồ án, các cơ chế Synchronization được sử dụng để xây dựng các nhóm kiểm thử liên quan đến Queue Communication, Mutex Protection, Shared Resource Access và Concurrency Testing. Thông qua các kịch bản kiểm thử này, nhóm có thể đánh giá tính đúng đắn của việc trao đổi dữ liệu giữa các task cũng như khả năng bảo vệ tài nguyên dùng chung trong môi trường đa nhiệm.

Từ góc độ kiểm thử, Concurrency và Synchronization là những yếu tố quan trọng ảnh hưởng trực tiếp đến độ tin cậy và tính ổn định của hệ thống nhúng thời gian thực. Việc đánh giá các cơ chế này giúp phát hiện sớm các lỗi đồng bộ hóa trước khi triển khai hệ thống trong môi trường thực tế.

2.5 Hard Deadline, Soft Deadline và Timing Analysis

Trong hệ thống thời gian thực, Deadline là khoảng thời gian tối đa cho phép một task hoàn thành công việc của mình kể từ thời điểm được kích hoạt. Khả năng đáp ứng deadline là một trong những tiêu chí quan trọng nhất để đánh giá chất lượng của hệ thống thời gian thực.

Dựa trên mức độ quan trọng của nhiệm vụ, Deadline thường được chia thành Hard Deadline và Soft Deadline.

Hard Deadline là loại deadline bắt buộc phải đáp ứng. Nếu task hoàn thành sau thời điểm quy định, hệ thống có thể xảy ra lỗi nghiêm trọng hoặc mất chức năng quan trọng. Các ứng dụng như hệ thống báo cháy, hệ thống dừng khẩn cấp hoặc hệ thống an toàn công nghiệp thường yêu cầu Hard Deadline.

Ngược lại, Soft Deadline cho phép task hoàn thành muộn trong một giới hạn nhất định mà không gây ảnh hưởng nghiêm trọng đến toàn bộ hệ thống. Các tác vụ giám sát, truyền thông hoặc ghi log thường thuộc nhóm Soft Deadline. Trong trường hợp này, việc hoàn thành chậm có thể làm giảm hiệu năng nhưng không làm hệ thống mất khả năng hoạt động.

Để đánh giá khả năng đáp ứng thời gian thực, cần thực hiện Timing Analysis nhằm đo lường và phân tích các chỉ số liên quan đến thời gian thực thi của hệ thống. Một số chỉ số quan trọng bao gồm:

- Execution Time: Thời gian thực thi của một task.
- Response Time: Thời gian từ khi task được kích hoạt đến khi hoàn thành.
- WCET (Worst-Case Execution Time): Thời gian thực thi lớn nhất của task trong các điều kiện hoạt động khác nhau.
- Deadline Miss: Số lần task không hoàn thành trước deadline.
- Context Switch Count: Số lần chuyển đổi thực thi giữa các task.

Trong phạm vi đồ án, các chỉ số trên được sử dụng để đánh giá hiệu quả của Scheduler, khả năng đáp ứng deadline và mức độ ổn định của hệ thống trong các tình huống vận hành khác nhau. Kết quả Timing Analysis được sử dụng làm cơ sở để đánh giá PASS hoặc FAIL cho từng nhóm kiểm thử.

2.6 Test Case, Test Suite và Log Analysis

Trong kiểm thử phần mềm, Test Case là một kịch bản kiểm thử cụ thể được xây dựng nhằm đánh giá một chức năng hoặc cơ chế xác định của hệ thống. Mỗi Test Case

thường bao gồm mục tiêu kiểm thử, điều kiện thực hiện, dữ liệu đầu vào, kết quả mong đợi và tiêu chí đánh giá.

Nhiều Test Case có cùng mục tiêu sẽ được nhóm lại thành Test Suite nhằm hỗ trợ việc tổ chức, thực thi và quản lý kiểm thử một cách hiệu quả. Việc sử dụng Test Suite giúp giảm thời gian kiểm thử, tăng khả năng tái sử dụng và đảm bảo tính nhất quán trong quá trình đánh giá hệ thống.

Trong phạm vi đồ án, các Test Suite được xây dựng theo các nhóm kiểm thử chính gồm Scheduler Testing, Priority Scheduling, Priority Preemption, Synchronization, Concurrency, Hard Deadline, Soft Deadline, Fault Injection và Timing Analysis. Mỗi nhóm kiểm thử được thiết kế nhằm đánh giá một đặc tính cụ thể của hệ thống nhúng thời gian thực.

Tiêu chí PASS/FAIL được xác định dựa trên khả năng đáp ứng hành vi mong đợi của Scheduler, mức độ tuân thủ Deadline, tính đúng đắn của cơ chế Synchronization và các chỉ số Timing Metrics thu được từ dữ liệu Runtime. Một Test Case được xem là PASS khi hệ thống hoạt động đúng theo thiết kế và các chỉ số đo được nằm trong giới hạn cho phép. Ngược lại, Test Case được xem là FAIL khi xuất hiện lỗi chức năng hoặc vi phạm các tiêu chí thời gian thực đã xác định.

Để hỗ trợ quá trình đánh giá, hệ thống sử dụng cơ chế Runtime Logging nhằm ghi nhận dữ liệu trong quá trình thực thi. Các sự kiện như trạng thái task, context switch, queue communication, mutex access, deadline miss và kết quả PASS/FAIL đều được lưu lại phục vụ cho quá trình phân tích.

Log Analysis đóng vai trò quan trọng vì đây là cơ sở để đánh giá khách quan hiệu năng của hệ thống. Thay vì quan sát thủ công, nhóm sử dụng dữ liệu runtime để kiểm chứng hoạt động của Scheduler, cơ chế đồng bộ hóa và khả năng xử lý lỗi của hệ thống. Những dữ liệu này sẽ được sử dụng trực tiếp trong các chương thực nghiệm để xây dựng bảng kết quả, biểu đồ phân tích và đánh giá hiệu năng hệ thống.

2.7 Tổng kết Chương 2

Chương 2 đã trình bày các cơ sở lý thuyết phục vụ trực tiếp cho việc xây dựng và kiểm thử hệ thống nhúng đa tiến trình thời gian thực sử dụng FreeRTOS Simulator.

Trước hết, các khái niệm về Embedded Systems, RTOS và FreeRTOS đã được giới thiệu nhằm làm rõ vai trò của hệ điều hành thời gian thực trong việc quản lý task, lập lịch và điều phối tài nguyên hệ thống. Đây là nền tảng cho việc xây dựng môi trường kiểm thử đa nhiệm trong đồ án.

Tiếp theo, các cơ chế Scheduler, Priority Scheduling và Priority Preemption được trình bày như nền tảng để thực hiện các nhóm kiểm thử liên quan đến điều phối CPU và xử lý các tác vụ có mức ưu tiên khác nhau. Các cơ chế này có ảnh hưởng trực tiếp đến khả năng đáp ứng thời gian thực của hệ thống.

Bên cạnh đó, các vấn đề Concurrency và Synchronization cũng được phân tích thông qua các cơ chế Mutex, Semaphore và Queue nhằm đảm bảo tính nhất quán của dữ liệu khi nhiều task hoạt động đồng thời trong môi trường đa nhiệm.

Chương cũng giới thiệu các khái niệm Hard Deadline, Soft Deadline và Timing Analysis cùng các chỉ số đánh giá như Execution Time, Response Time, WCET, Deadline Miss và Context Switch Count. Đây là cơ sở để thực hiện việc đo đạc, thu thập log và đánh giá PASS/FAIL trong các chương tiếp theo.

Ngoài ra, các khái niệm Test Case, Test Suite và Log Analysis được trình bày nhằm làm rõ quy trình xây dựng bộ kiểm thử và phương pháp đánh giá kết quả thực nghiệm. Việc sử dụng dữ liệu Runtime Logging giúp tăng tính khách quan và độ tin cậy của quá trình đánh giá hệ thống.

Nhìn chung, các nội dung trình bày trong chương này đóng vai trò nền tảng lý thuyết cho quá trình thiết kế hệ thống, xây dựng bộ kiểm thử, triển khai thực nghiệm và phân tích kết quả của đề tài trong các chương tiếp theo.

CHƯƠNG 3 – THIẾT KẾ HỆ THỐNG

Chương này trình bày quá trình thiết kế hệ thống kiểm thử được xây dựng trên nền tảng FreeRTOS Simulator nhằm phục vụ việc đánh giá các cơ chế của hệ thống nhúng đa tiến trình thời gian thực. Trên cơ sở lý thuyết đã trình bày trong Chương 2, nhóm tiến hành thiết kế kiến trúc tổng thể, xác định các task chức năng, cơ chế lập lịch, đồng bộ hóa tài nguyên, giám sát deadline và thu thập dữ liệu runtime.

Mục tiêu của chương là xây dựng một môi trường mô phỏng có khả năng tái hiện các đặc trưng thường gặp trong hệ thống nhúng thực tế, đồng thời hỗ trợ triển khai các test case liên quan đến scheduler, priority scheduling, priority preemption, synchronization, concurrency, fault injection và timing analysis. Các thành phần được thiết kế theo hướng phục vụ trực tiếp cho hoạt động kiểm thử và đánh giá pass/fail trong các chương tiếp theo.

3.1 Yêu cầu thiết kế hệ thống

Bảng 3.1. Yêu cầu thiết kế hệ thống

Yêu cầu	Mục đích
Multi-task Environment	Mô phỏng hệ thống nhúng đa nhiệm
Priority Scheduling	Kiểm thử cơ chế lập lịch
Priority Preemption	Kiểm thử xử lý sự kiện khẩn cấp
Synchronization	Kiểm thử Mutex, Semaphore và Queue
Deadline Handling & Analysis	Đánh giá Hard Deadline và Soft Deadline
Fault Injection	Mô phỏng lỗi hệ thống
Runtime Logging	Thu thập dữ liệu runtime
Metrics Collection	Đánh giá hiệu năng hệ thống

Xuất phát từ mục tiêu của đề tài, hệ thống được thiết kế theo hướng phục vụ kiểm thử các cơ chế của hệ điều hành thời gian thực FreeRTOS thay vì xây dựng một ứng dụng nhúng hoàn chỉnh. Do đó, hệ thống cần hỗ trợ nhiều task hoạt động đồng thời, cho phép quan sát cơ chế scheduler, priority scheduling, priority preemption và synchronization trong môi trường mô phỏng.

Bên cạnh các task chức năng như Sensor_Task, Control_Task và Communication_Task, hệ thống còn tích hợp Logger_Task để ghi nhận dữ liệu runtime và Fault_Injector_Task để tạo các tình huống lỗi phục vụ kiểm thử. Các dữ liệu thu thập

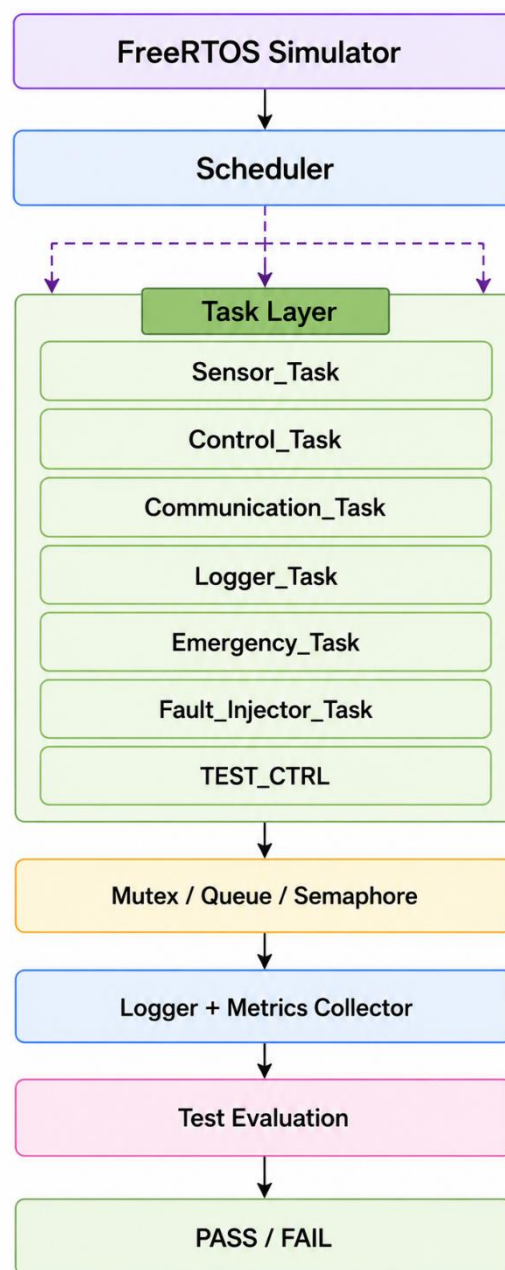
được sử dụng để tính toán các metrics như Execution Time, Response Time, WCET, Deadline Miss và Context Switch Count.

Những yêu cầu trên là cơ sở để xây dựng kiến trúc hệ thống, thiết kế các task và triển khai các test case trong các mục tiếp theo của chương này.

3.2 Kiến trúc tổng thể FreeRTOS Simulator

Hình 3.1 trình bày kiến trúc tổng thể của hệ thống kiểm thử được xây dựng trên nền tảng FreeRTOS Simulator. Kiến trúc được thiết kế theo mô hình nhiều lớp nhằm tách biệt giữa môi trường thực thi, các task chức năng, cơ chế đồng bộ hóa và thành phần thu thập dữ liệu runtime.

Hình 3.1. Kiến trúc tổng thể hệ thống FreeRTOS Simulator



Ở lớp thực thi, FreeRTOS Simulator đóng vai trò môi trường chạy chương trình và cung cấp Scheduler để quản lý các task. Hệ thống bao gồm nhiều task được xây dựng phục vụ cho các nhóm kiểm thử khác nhau như DoorSensor, VisitorCounter, FactoryHigh, FactoryMed, FactoryLow, CameraScan, IntrusionAlert, SmokeSensor, TempSensor, WindSensor, EventProducer, LogConsumer, Navigation, CriticalWork, CPUOverload, MeasuredTask và TestCtrl.

Các task được cấu hình với nhiều mức ưu tiên khác nhau nhằm phục vụ việc kiểm thử Scheduler, Priority Scheduling, Priority Preemption, Synchronization, Concurrency, Hard Deadline, Soft Deadline, Fault Injection và Timing Analysis.

Lớp đồng bộ hóa sử dụng các cơ chế Queue, Mutex và Semaphore để hỗ trợ trao đổi dữ liệu giữa các task và bảo vệ tài nguyên dùng chung. Điều này giúp hạn chế các lỗi đồng thời như Race Condition và hỗ trợ kiểm thử Synchronization trong môi trường đa nhiệm.

Trong quá trình thực thi, hệ thống sử dụng cơ chế Runtime Logging để ghi nhận các sự kiện phát sinh. Dữ liệu thu thập được được sử dụng để tính toán các chỉ số như Execution Time, Response Time, WCET, Deadline Miss và Context Switch Count.

Cuối cùng, các kết quả thu thập được sử dụng cho quá trình Test Evaluation nhằm đánh giá khả năng đáp ứng deadline, hiệu quả của Scheduler, cơ chế Synchronization và kết quả PASS/FAIL của từng test case.

3.3 Thiết kế các Task

Trong hệ thống FreeRTOS Simulator, task là đơn vị thực thi cơ bản được scheduler quản lý và phân bổ CPU. Dựa trên các yêu cầu thiết kế đã xác định ở mục 3.1 và kiến trúc tổng thể ở mục 3.2, nhóm xây dựng các task nhằm mô phỏng các thành phần thường gặp trong hệ thống nhúng thời gian thực như cảm biến, điều khiển, truyền thông, ghi log, xử lý khẩn cấp và tạo lỗi kiểm thử.

Mỗi task được cấu hình với mức priority khác nhau để phục vụ việc đánh giá scheduler, priority scheduling, priority preemption, synchronization, fault injection và timing analysis. Việc phân tách hệ thống thành nhiều task độc lập giúp mô phỏng môi trường đa nhiệm và tạo điều kiện triển khai các test case trong đồ án.

Bảng 3.2. Cấu hình các Task trong hệ thống

Task	Priority	Vai trò	Nội dung kiểm thử
TestCtrl	5	Điều phối và quản lý quá trình thực thi các test case	Test Execution Control

IntrusionAlert	4	Xử lý sự kiện phát hiện xâm nhập trong hệ thống camera an ninh	Priority Preemption
SmokeSensor	4	Mô phỏng cảm biến khói trong hệ thống báo cháy thông minh	Hard Deadline
FactoryHigh	3	Task có mức ưu tiên cao trong kịch bản kiểm thử Priority Scheduling	Priority Scheduling
Navigation	3	Thực hiện chức năng điều hướng trong hệ thống drone tránh chướng ngại vật	Concurrency Testing
CriticalWork	3	Task quan trọng chịu tác động của các kịch bản Fault Injection	Fault Injection
MeasuredTask	3	Thu thập và đánh giá các chỉ số thời gian thực của hệ thống	Timing Analysis, WCET
DoorSensor	2	Mô phỏng cảm biến cửa trong hệ thống kiểm soát người ra vào	Scheduler Testing
VisitorCounter	2	Mô phỏng bộ đếm số lượng người ra vào khu vực giám sát	Scheduler Testing
CameraScan	2	Thực hiện quét và xử lý hình ảnh trước khi phát hiện xâm nhập	Priority Preemption
TempSensor	2	Mô phỏng cảm biến nhiệt độ trong hệ thống quan trắc môi trường	Soft Deadline
WindSensor	2	Mô phỏng cảm biến tốc độ gió trong hệ thống quan trắc môi trường	Soft Deadline
EventProducer	2	Tạo dữ liệu và gửi vào Queue để mô phỏng giao tiếp giữa các task	Synchronization, Queue Testing
LogConsumer	2	Nhận dữ liệu từ Queue và ghi vào vùng log dùng chung	Synchronization, Mutex Testing

FactoryMed	2	Task có mức ưu tiên trung bình trong kịch bản Priority Scheduling	Priority Scheduling
CPUOverload	2	Tạo tải CPU nhân tạo để kiểm thử khả năng đáp ứng của hệ thống khi quá tải	Fault Injection
FactoryLow	1	Task có mức ưu tiên thấp trong kịch bản Priority Scheduling	Priority Scheduling

Các giá trị priority được lựa chọn nhằm tạo ra môi trường kiểm thử có nhiều mức ưu tiên khác nhau. TestCtrl có priority cao nhất để điều phối quá trình chạy test suite. IntrusionAlert và SmokeSensor được cấu hình ở mức ưu tiên cao nhằm mô phỏng các tác vụ quan trọng có yêu cầu phản hồi nhanh. Trong khi đó FactoryLow được thiết lập với mức ưu tiên thấp để phục vụ các kịch bản Priority Scheduling.

Nhóm các task DoorSensor và VisitorCounter được sử dụng để kiểm thử Scheduler và Context Switching. CameraScan và IntrusionAlert được sử dụng để đánh giá cơ chế Priority Preemption. TempSensor và WindSensor phục vụ các bài kiểm thử Soft Deadline. EventProducer và LogConsumer được triển khai nhằm đánh giá Queue Communication và Synchronization.

Bên cạnh đó, CPUOverload và CriticalWork được sử dụng trong các kịch bản Fault Injection nhằm tạo tải hệ thống và đánh giá khả năng phát hiện Deadline Miss. MeasuredTask được xây dựng để thu thập các chỉ số Timing Analysis như Execution Time, Response Time và WCET.

Nhìn chung, tập hợp các task trên được thiết kế nhằm tái hiện những thành phần quan trọng của hệ thống nhúng thời gian thực, đồng thời tạo môi trường phù hợp để triển khai toàn bộ các nhóm kiểm thử của đề tài.

3.4 Thiết kế Scheduler và Priority

Sau khi xác định các task chức năng của hệ thống, bước tiếp theo là xây dựng cơ chế lập lịch nhằm đảm bảo các task được thực thi theo đúng mức độ ưu tiên và yêu cầu thời gian thực. Trong hệ thống FreeRTOS Simulator, scheduler đóng vai trò quản lý trạng thái task, phân bổ CPU và thực hiện preemption khi xuất hiện các task có mức ưu tiên cao hơn.

Hệ thống sử dụng cơ chế Priority-Based Preemptive Scheduling của FreeRTOS. Theo cơ chế này, task có mức priority cao nhất trong trạng thái Ready sẽ được cấp CPU để thực thi. Khi một task có priority cao hơn chuyển sang trạng thái Ready, scheduler sẽ thực hiện context switch và chuyển quyền xử lý sang task đó.

Bảng 3.3. Phân bổ mức ưu tiên cho các task

Task	Priority
TestCtrl	5
IntrusionAlert	4
SmokeSensor	4
FactoryHigh	3
Navigation	3
CriticalWork	3
MeasuredTask	3
DoorSensor	2
VisitorCounter	2
CameraScan	2
TempSensor	2
WindSensor	2
EventProducer	2
LogConsumer	2
FactoryMed	2
CPUOverload	2
FactoryLow	1

Việc phân bổ priority được thực hiện dựa trên mức độ quan trọng của từng task đối với hoạt động của hệ thống. Các task như IntrusionAlert và SmokeSensor được gán mức ưu tiên cao nhằm mô phỏng các tình huống khẩn cấp hoặc có yêu cầu thời gian thực nghiêm ngặt. Các task thuộc nhóm FactoryHigh, Navigation, CriticalWork và MeasuredTask được gán mức ưu tiên cao để phục vụ các bài kiểm thử Priority Scheduling, Concurrency và Timing Analysis.

Các task DoorSensor, VisitorCounter, CameraScan, TempSensor, WindSensor, EventProducer, LogConsumer, FactoryMed và CPUOverload được cấu hình ở mức ưu tiên trung bình nhằm mô phỏng các thành phần hoạt động thường xuyên trong hệ thống. FactoryLow được cấu hình ở mức ưu tiên thấp nhất nhằm phục vụ việc đánh giá cơ chế Priority Scheduling.

Thiết kế cơ chế Priority Preemption

Một trong những mục tiêu quan trọng của đề tài là đánh giá khả năng Preemption của Scheduler khi xuất hiện task ưu tiên cao. Trong hệ thống được thiết kế, cơ chế này được kiểm tra chủ yếu thông qua cặp task CameraScan và IntrusionAlert.

Khi CameraScan đang thực thi và xuất hiện sự kiện xâm nhập, IntrusionAlert sẽ chuyển sang trạng thái Ready với mức ưu tiên cao hơn. Scheduler phải thực hiện Context Switch và cấp CPU cho IntrusionAlert ngay lập tức. Sau khi xử lý xong sự kiện, Scheduler tiếp tục thực thi task bị gián đoạn.

Cơ chế này được sử dụng trong các test case như Camera An Ninh Phát Hiện Xâm Nhập và Hệ Thống Báo Cháy Thông Minh nhằm đánh giá khả năng phản ứng của Scheduler và xác minh tính đúng đắn của Priority Preemption.

Thiết kế trạng thái Task

Trong quá trình vận hành, các task có thể chuyển đổi giữa các trạng thái Ready, Running, Blocked và Suspended dưới sự quản lý của Scheduler. Ready biểu thị task đã sẵn sàng thực thi, Running biểu thị task đang sử dụng CPU, Blocked xuất hiện khi task chờ Queue, Semaphore hoặc Delay Timer, trong khi Suspended được sử dụng khi task tạm thời không tham gia quá trình lập lịch.

Việc theo dõi các trạng thái này cho phép nhóm phân tích hành vi của Scheduler thông qua Runtime Logging, đồng thời hỗ trợ các hoạt động Timing Analysis và PASS/FAIL Evaluation trong các chương tiếp theo.

3.5 Thiết kế Synchronization và Shared Resource

Trong quá trình vận hành, nhiều task trong hệ thống cần trao đổi dữ liệu hoặc truy cập các tài nguyên dùng chung. Để đảm bảo tính nhất quán của dữ liệu và tránh các lỗi phát sinh do thực thi đồng thời, hệ thống được thiết kế sử dụng các cơ chế synchronization của FreeRTOS như Queue, Mutex và Semaphore. Các cơ chế này được triển khai trực tiếp trong kiến trúc hệ thống nhằm phục vụ các hoạt động kiểm thử liên quan đến Synchronization, Concurrency và Timing Analysis.

Trong hệ thống được thiết kế, Mutex được sử dụng để bảo vệ các tài nguyên dùng chung như Shared Log Buffer và Runtime Metrics. Khi nhiều task cùng thực hiện thao tác ghi log hoặc cập nhật dữ liệu hệ thống, Mutex đảm bảo chỉ một task được phép truy cập tài nguyên tại một thời điểm. Điều này giúp duy trì tính nhất quán của dữ liệu và hỗ trợ kiểm thử các tình huống Race Condition hoặc Priority Inversion.

Queue được sử dụng làm cơ chế trao đổi dữ liệu chính giữa các task. Trong kịch bản Synchronization Testing, EventProducer gửi dữ liệu vào Queue và LogConsumer nhận dữ liệu để xử lý. Việc sử dụng Queue giúp tách biệt giữa quá trình tạo dữ liệu và xử lý dữ liệu, đồng thời giảm sự phụ thuộc trực tiếp giữa các task.

Semaphore được sử dụng để đồng bộ hóa các sự kiện trong hệ thống. Cơ chế này hỗ trợ việc kích hoạt hoặc đánh thức task khi xuất hiện sự kiện cần xử lý, đồng thời giảm thời gian phản hồi của hệ thống trong môi trường đa nhiệm.

Bảng 3.4. Tài nguyên dùng chung và cơ chế bảo vệ

Tài nguyên	Task liên quan	Cơ chế bảo vệ
Event Queue	EventProducer, LogConsumer	Queue
Runtime Log Buffer	Tất cả các task	Mutex
Runtime Metrics	MeasuredTask, TestCtrl	Mutex
Event Notification	Nhiều task trong hệ thống	Semaphore

Thông qua việc xác định rõ tài nguyên dùng chung và cơ chế bảo vệ tương ứng, hệ thống có thể duy trì tính nhất quán của dữ liệu trong môi trường đa nhiệm, đồng thời tạo điều kiện thuận lợi cho việc triển khai các kịch bản kiểm thử liên quan đến Synchronization và Concurrency.

Để đánh giá khả năng xử lý đồng thời của hệ thống, nhiều task được thiết kế hoạt động song song trong cùng khoảng thời gian. Các tình huống này cho phép kiểm tra hiệu quả của Scheduler cũng như cơ chế Synchronization khi xuất hiện tranh chấp tài nguyên. Trong các test case Synchronization Testing và Concurrency Testing, dữ liệu runtime được thu thập để đánh giá hoạt động của Queue, Mutex, Semaphore và tác động của các cơ chế đồng bộ hóa tới hiệu năng hệ thống.

Những dữ liệu thu thập được từ các kịch bản này sẽ được sử dụng trong Chương 5 và Chương 6 để đánh giá tính đúng đắn của cơ chế Synchronization, khả năng duy trì dữ liệu nhất quán và mức độ ổn định của hệ thống trong điều kiện hoạt động đồng thời.

3.6 Thiết kế Deadline Monitoring

Một trong những yêu cầu quan trọng nhất của hệ thống nhúng thời gian thực là khả năng hoàn thành nhiệm vụ trong khoảng thời gian cho phép. Trong nhiều ứng dụng thực tế, việc đưa ra kết quả chính xác nhưng không đúng thời điểm vẫn được xem là lỗi hệ thống. Vì vậy, bên cạnh cơ chế Scheduler và Synchronization, hệ thống cần có khả năng giám sát Deadline nhằm đánh giá mức độ tuân thủ các ràng buộc thời gian của từng task.

Xuất phát từ các mục tiêu kiểm thử đã trình bày trong đề tài, hệ thống được thiết kế để hỗ trợ đánh giá cả Hard Deadline và Soft Deadline. Cơ chế này đóng vai trò quan trọng trong các nhóm kiểm thử Deadline Testing, Timing Analysis và PASS/FAIL Evaluation.

Để thực hiện chức năng này, hệ thống sử dụng cơ chế Runtime Monitoring tích hợp trong quá trình thực thi. Các thông tin như thời gian bắt đầu, thời gian hoàn thành, Response Time và Execution Time của task được ghi nhận thông qua cơ chế Runtime Logging và sử dụng để xác định các trường hợp vi phạm Deadline.

Thiết kế Hard Deadline

Hard Deadline được sử dụng cho các nhiệm vụ mà việc hoàn thành muộn được xem là lỗi nghiêm trọng của hệ thống. Trong phạm vi đề tài, các kịch bản như Hệ Thống Báo Cháy Thông Minh và Camera An Ninh Phát Hiện Xâm Nhập được xem là các tình huống có Hard Deadline.

Các task như IntrusionAlert và SmokeSensor thuộc nhóm nhiệm vụ có yêu cầu phản hồi nhanh. Nếu task hoàn thành sau Deadline quy định, test case sẽ được đánh giá là FAIL bất kể kết quả xử lý có chính xác hay không.

Thiết kế Soft Deadline

Khác với Hard Deadline, Soft Deadline cho phép một mức độ trễ nhất định mà không gây ảnh hưởng nghiêm trọng tới chức năng cốt lõi của hệ thống. Trong hệ thống được thiết kế, TempSensor, WindSensor và LogConsumer được xem là các task thuộc nhóm Soft Deadline.

Việc phân tách Hard Deadline và Soft Deadline cho phép hệ thống đánh giá chính xác hơn mức độ ảnh hưởng của từng trường hợp Deadline Miss và phản ánh sát hơn các điều kiện vận hành thực tế.

Bảng 3.5. Các task và loại Deadline

Task	Loại Deadline
IntrusionAlert	Hard Deadline
SmokeSensor	Hard Deadline
TempSensor	Soft Deadline
WindSensor	Soft Deadline
LogConsumer	Soft Deadline

Thiết kế Deadline Miss Detection

Để hỗ trợ các hoạt động kiểm thử, hệ thống được trang bị cơ chế phát hiện Deadline Miss. Trong quá trình thực thi, thời gian hoàn thành của task được so sánh với ngưỡng Deadline tương ứng nhằm xác định các trường hợp vi phạm thời gian thực.

Khi một task vượt quá thời hạn quy định, hệ thống sẽ ghi nhận sự kiện Deadline Miss vào Runtime Log. Các thông tin như tên task, thời điểm xảy ra vi phạm, Execution Time và trạng thái PASS/FAIL sẽ được lưu trữ để phục vụ quá trình phân tích.

Thiết kế Timing Metrics

Bên cạnh việc xác định Deadline Miss, hệ thống còn thu thập nhiều chỉ số thời gian nhằm phục vụ hoạt động Timing Analysis.

Bảng 3.6. Các chỉ số thời gian được giám sát

Chỉ số	Ý nghĩa
Execution Time	Thời gian thực thi của task
Response Time	Thời gian từ khi task sẵn sàng đến khi bắt đầu chạy
WCET	Thời gian thực thi lớn nhất của task (Worst-Case Execution Time)
Deadline Miss	Số lần task vi phạm deadline
Context Switch Count	Số lần chuyển ngữ cảnh giữa các task
Priority Preemption Count	Số lần task ưu tiên cao giành quyền thực thi từ task ưu tiên thấp hơn

Các chỉ số này được thu thập trong suốt quá trình chạy test và trở thành cơ sở cho các hoạt động đánh giá pass/fail ở Chương 5 và phân tích thực nghiệm ở Chương 6.

Thiết kế Pass/Fail Evaluation

Trong phạm vi đề tài, kết quả kiểm thử liên quan đến deadline được đánh giá dựa trên mức độ tuân thủ các ngưỡng thời gian đã xác định trước. Một test case được xem là Pass khi task hoàn thành trong khoảng thời gian cho phép và không xuất hiện Deadline Miss ngoài ngưỡng chấp nhận.

Ngược lại, test case sẽ được đánh giá là Fail nếu task vi phạm Hard Deadline hoặc số lượng Deadline Miss vượt quá tiêu chí cho phép. Đối với Soft Deadline, việc đánh giá được thực hiện dựa trên tỷ lệ đáp ứng deadline và mức độ ảnh hưởng tới hoạt động chung của hệ thống.

Cách tiếp cận này cho phép quá trình đánh giá được thực hiện dựa trên dữ liệu định lượng thay vì quan sát thủ công, đồng thời phản ánh đúng bản chất của các hệ thống nhúng thời gian thực.

3.7 Thiết kế Logging và Metrics Collection

Để phục vụ hoạt động kiểm thử và đánh giá hiệu năng của hệ thống, việc thu thập dữ liệu Runtime được xem là một thành phần quan trọng trong kiến trúc FreeRTOS Simulator. Thay vì đánh giá hệ thống thông qua quan sát thủ công, đề tài sử dụng cơ chế Runtime Logging nhằm ghi nhận các sự kiện phát sinh trong quá trình thực thi và hỗ trợ phân tích hành vi của Scheduler, Synchronization, Deadline Handling và Fault Injection.

Trong hệ thống được thiết kế, các task ghi log thông qua cơ chế LogCSV. Mỗi khi xảy ra các sự kiện như task được Scheduler cấp CPU, task hoàn thành xử lý, queue communication, deadline miss hoặc context switch, hệ thống sẽ tạo bản ghi log tương ứng. Các bản ghi này được lưu theo Timestamp nhằm phục vụ việc phân tích trình tự thực thi và đánh giá kết quả kiểm thử.

Bên cạnh các chỉ số Timing đã được trình bày ở mục 3.6, hệ thống còn thu thập các Metrics bổ sung phục vụ đánh giá hiệu năng và hành vi của Scheduler, Priority Preemption và cơ chế Synchronization.

Bảng 3.7. Các Metrics bổ sung được thu thập

Metric	Ý nghĩa
Execution Time	Thời gian thực thi của task
Response Time	Thời gian phản hồi của task
WCET	Thời gian thực thi lớn nhất (Worst-Case Execution Time)
Deadline Miss	Số lần vi phạm deadline
Context Switch Count	Số lần chuyển ngữ cảnh
Priority Preemption Count	Số lần xảy ra preemption (task ưu tiên cao giành CPU từ task ưu tiên thấp hơn)

Các chỉ số này cho phép đánh giá hiệu quả hoạt động của scheduler, khả năng đáp ứng deadline và tác động của các cơ chế synchronization tới hiệu năng hệ thống.

Bảng 3.8. Định dạng Log Runtime

Timestamp	Task	Event	Status
-----------	------	-------	--------

100 ms	Sensor_Task	TASK_START	PASS
120 ms	Control_Task	TASK_EXECUTE	PASS
150 ms	Communication_Task	QUEUE_SEND	PASS
180 ms	Emergency_Task	PREEMPTION	PASS
250 ms	Fault_Injector_Task	DEADLINE_MISS	FAIL

Thông qua cấu trúc log thống nhất, dữ liệu runtime có thể được xử lý tự động và sử dụng trong các hoạt động phân tích scheduler, synchronization, fault injection và pass/fail evaluation. Đồng thời, các dữ liệu này cũng là nguồn đầu vào cho các biểu đồ và bảng kết quả thực nghiệm được trình bày trong Chương 6.

3.8 Thiết kế Fault Injection

Bên cạnh việc đánh giá hệ thống trong điều kiện vận hành bình thường, đề tài còn xem xét khả năng hoạt động của hệ thống khi xuất hiện các điều kiện bất lợi hoặc lỗi bất thường. Để thực hiện điều này, hệ thống được trang bị cơ chế Fault Injection nhằm tạo ra các tình huống lỗi có kiểm soát và quan sát phản ứng của Scheduler cũng như các task liên quan.

Fault Injection cho phép đánh giá khả năng chịu lỗi của hệ thống, xác định giới hạn hoạt động của Scheduler và phân tích ảnh hưởng của lỗi tới các Timing Metrics. Đây là một trong những nội dung quan trọng được đề xuất trong proposal nhằm tăng tính thực tiễn của hoạt động kiểm thử.

Trong hệ thống, các tình huống lỗi được tạo ra bởi các task thực thi chuyên biệt như CPUOverload và CriticalWork, kết hợp với các task khác tham gia xử lý dữ liệu như EventProducer và LogConsumer. Các lỗi được thiết kế nhằm tác động tới Scheduler, Synchronization hoặc Deadline Handling.

Bảng 3.9. Fault Injection Matrix

Fault Scenario	Mục tiêu kiểm thử
CPU Overload	Đánh giá khả năng duy trì Deadline và hoạt động của Scheduler khi hệ thống chịu tải CPU cao.
Artificial Delay	Kiểm tra khả năng phát hiện và xử lý các trường hợp vi phạm Deadline do độ trễ nhân tạo.
Queue Blocking	Đánh giá cơ chế Synchronization và khả năng xử lý khi Queue bị nghẽn hoặc chậm phản hồi.

Race Condition	Đánh giá khả năng kiểm soát truy cập tài nguyên dùng chung trong môi trường đa nhiệm.
Priority Inversion	Đánh giá khả năng xử lý hiện tượng đảo ngược ưu tiên và cơ chế bảo vệ tài nguyên.
Burst Event	Kiểm tra khả năng phản ứng của Scheduler khi xuất hiện nhiều sự kiện đồng thời trong thời gian ngắn.
Resource Contention	Đánh giá hiệu quả của cơ chế Mutex trong việc quản lý tài nguyên dùng chung.

Ví dụ, trong kịch bản CPU Overload, task CPUOverload tạo tải xử lý lớn nhằm quan sát khả năng duy trì Deadline của các task quan trọng. Trong kịch bản Priority Inversion, nhiều task với mức Priority khác nhau cùng tranh chấp một tài nguyên được bảo vệ bằng Mutex, nhằm kiểm tra hiệu quả của cơ chế Priority Inheritance của FreeRTOS.

Thông qua các tình huống Fault Injection này, hệ thống có thể được đánh giá trong nhiều điều kiện vận hành khác nhau, từ đó phản ánh sát hơn hành vi thực tế của các hệ thống nhúng đa tiến trình.

3.9 Workflow Kiểm thử

Sau khi hoàn thiện các thành phần chức năng, Scheduler, Synchronization, Deadline Monitoring, Runtime Logging và Fault Injection, hệ thống được tổ chức thành một quy trình kiểm thử thống nhất nhằm hỗ trợ việc triển khai các test case trong chương tiếp theo.

Workflow kiểm thử mô tả toàn bộ quá trình từ khởi tạo hệ thống, thực thi task, thu thập dữ liệu Runtime cho tới đánh giá PASS/FAIL. Việc xây dựng Workflow rõ ràng giúp đảm bảo mọi test case đều được thực hiện theo cùng một quy trình và tạo điều kiện thuận lợi cho việc tái hiện kết quả thực nghiệm.

Trong mỗi lần chạy kiểm thử, hệ thống khởi tạo các task theo cấu hình đã xác định, kích hoạt Scheduler và bắt đầu thu thập dữ liệu Runtime. Sau khi hoàn thành test case, các Metrics được tổng hợp và so sánh với tiêu chí đánh giá tương ứng nhằm xác định kết quả PASS hoặc FAIL.

Workflow này được sử dụng cho toàn bộ các nhóm kiểm thử như Scheduler Testing, Priority Scheduling, Priority Preemption, Synchronization Testing, Concurrency Testing, Fault Injection Testing và Timing Analysis.

3.10 Tổng kết chương

Chương 3 đã trình bày quá trình thiết kế hệ thống kiểm thử được xây dựng trên nền tảng FreeRTOS Simulator nhằm phục vụ việc đánh giá các cơ chế của hệ thống

nhúng đa tiến trình thời gian thực. Trên cơ sở các yêu cầu đã xác định ở Chương 1 và các nền tảng lý thuyết ở Chương 2, nhóm đã xây dựng kiến trúc tổng thể của hệ thống, xác định các task chức năng và thiết kế cơ chế lập lịch dựa trên mức độ ưu tiên.

Bên cạnh đó, chương cũng trình bày cách thức tổ chức các cơ chế Synchronization, quản lý tài nguyên dùng chung, xử lý Deadline và thu thập dữ liệu Runtime. Các thành phần này đóng vai trò quan trọng trong việc triển khai các test case liên quan đến Scheduler, Priority Scheduling, Priority Preemption, Concurrency, Synchronization, Hard Deadline và Soft Deadline.

Ngoài các chức năng vận hành thông thường, hệ thống còn được bổ sung cơ chế Fault Injection nhằm tạo ra các điều kiện kiểm thử bất lợi và đánh giá khả năng chịu lỗi của Scheduler. Dữ liệu Runtime được thu thập thông qua cơ chế Runtime Logging và lưu trữ dưới dạng log phục vụ cho quá trình tính toán các Metrics như Execution Time, Response Time, WCET, Deadline Miss, Context Switch Count và Priority Preemption Count.

Nhìn chung, kiến trúc được thiết kế không chỉ phục vụ mục đích mô phỏng hệ thống nhúng thời gian thực mà còn hỗ trợ trực tiếp cho hoạt động kiểm thử theo định hướng của đề tài. Trên cơ sở thiết kế này, Chương 4 sẽ trình bày quá trình cài đặt hệ thống bằng FreeRTOS Simulator, bao gồm việc tạo task, cấu hình Scheduler, triển khai Queue, Mutex, Semaphore và xây dựng các thành phần phục vụ hoạt động kiểm thử.

Thông qua các nội dung đã trình bày, Chương 3 đóng vai trò cầu nối giữa phần cơ sở lý thuyết và quá trình hiện thực hóa hệ thống, tạo nền tảng cho việc triển khai kiểm thử và đánh giá thực nghiệm trong các chương tiếp theo.

CHƯƠNG 4 – CÀI ĐẶT HỆ THỐNG

Chương này trình bày quá trình hiện thực hóa hệ thống kiểm thử trên nền tảng FreeRTOS Simulator, bao gồm xây dựng các task chức năng, cấu hình Scheduler, triển khai cơ chế đồng bộ hóa, giám sát Deadline, thu thập dữ liệu Runtime và mô phỏng Fault Injection. Mục tiêu là xây dựng môi trường kiểm thử phục vụ các hoạt động Scheduler Testing, Priority Scheduling, Priority Preemption, Synchronization Testing, Deadline Testing, Fault Injection và Timing Analysis, đồng thời hỗ trợ thu thập dữ liệu và đánh giá PASS/FAIL cho các chương tiếp theo.

4.1 Môi trường cài đặt

Hệ thống được phát triển trên nền tảng FreeRTOS Windows Simulator nhằm tạo môi trường kiểm thử thuận tiện mà không cần phần cứng nhúng thực tế. Môi trường mô phỏng hỗ trợ cơ chế đa nhiệm, quản lý task, synchronization và scheduling tương tự hệ thống nhúng, đồng thời giúp dễ dàng quan sát hoạt động của scheduler và thu thập dữ liệu runtime. Nhờ đó, các kịch bản kiểm thử như Scheduler Testing, Priority Scheduling, Priority Preemption, Synchronization, Deadline Handling và Fault Injection có thể được triển khai hiệu quả.

Bảng 4.1. Môi trường phát triển

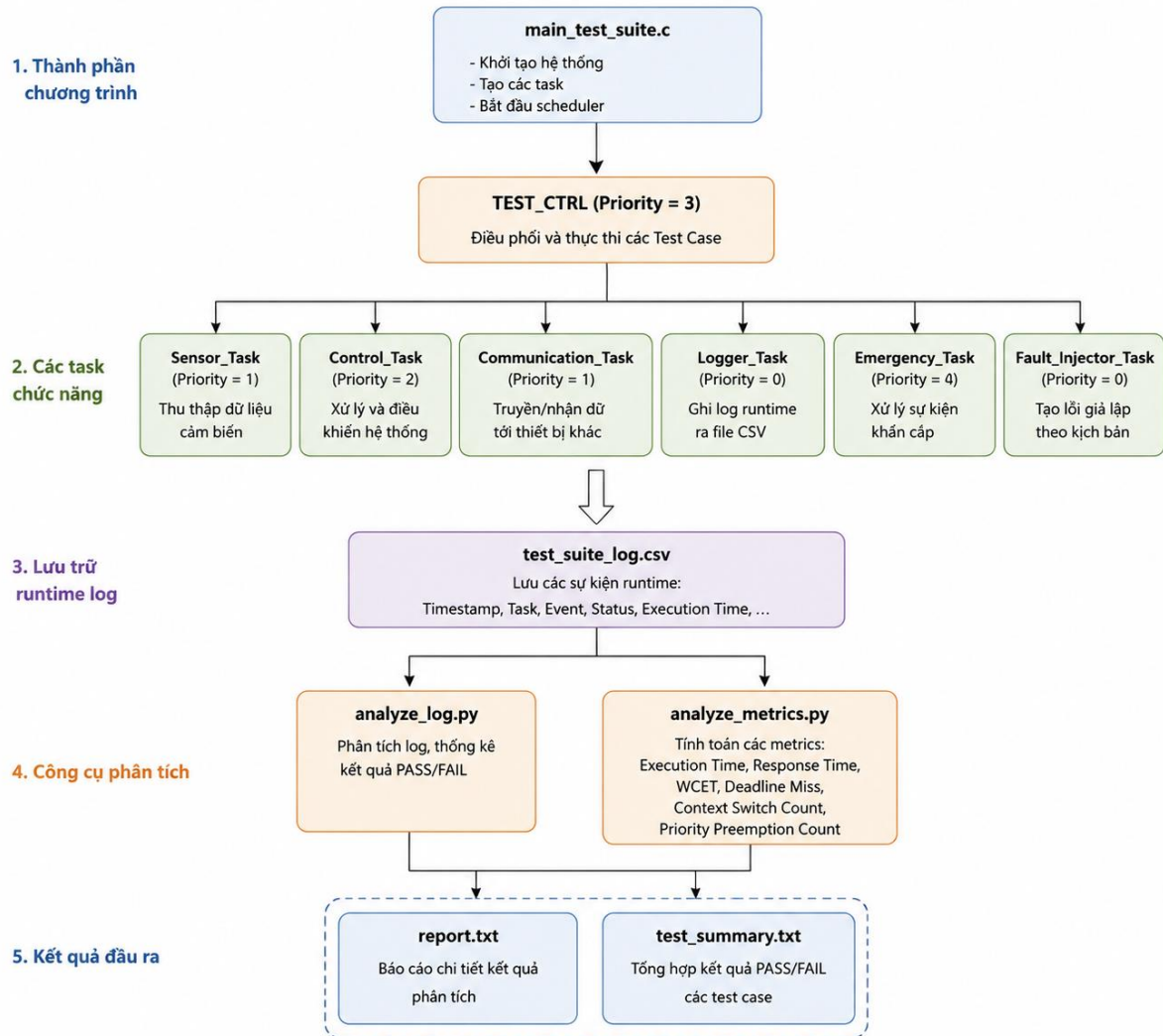
Thành phần	Giá trị
Operating System	Windows 10/11
Programming Language	C
IDE	Visual Studio 2022
Compiler	Microsoft Visual C++ (MSVC)
RTOS	FreeRTOS
Simulator	FreeRTOS Windows Simulator
Logging Format	CSV + Text Summary
Version Control	Git

Môi trường này đồng thời hỗ trợ quá trình ghi log, phân tích dữ liệu runtime và đánh giá kết quả kiểm thử thông qua các công cụ xử lý log được xây dựng riêng cho đề tài.

4.2 Cài đặt cấu trúc chương trình

Hệ thống được xây dựng trên nền tảng FreeRTOS Windows Simulator với thành phần trung tâm là file `main\_test\_suite.c`. File này chịu trách nhiệm khởi tạo scheduler, tạo các task, triển khai các test case và điều phối toàn bộ quá trình kiểm thử.

Hình 4.1 trình bày cấu trúc tổng thể của chương trình.



Trong hệ thống, task TestCtrl được sử dụng để điều phối việc thực hiện các test case. Các task chức năng thực tế gồm DoorSensor, VisitorCounter, CameraScan, IntrusionAlert, SmokeSensor, TempSensor, WindSensor, EventProducer, LogConsumer, Navigation, CriticalWork, CPUOverload và MeasuredTask được tạo và quản lý bởi FreeRTOS Scheduler.

Dữ liệu Runtime được ghi vào file test_suite_log.csv trong quá trình thực thi. Sau khi hoàn thành kiểm thử, các công cụ analyze_log.py và analyze_metrics.py được sử dụng để xử lý log, tính toán Metrics và sinh báo cáo kết quả. Cấu trúc này giúp việc triển khai kiểm thử, thu thập dữ liệu và đánh giá kết quả được thực hiện tập trung và thuận tiện.

4.3 Cài đặt tác vụ và ưu tiên

Các task trong hệ thống được tạo bằng API `xTaskCreate()` của FreeRTOS. Mỗi task được gán mức Priority phù hợp nhằm phục vụ việc kiểm thử Scheduler, Priority Scheduling và Priority Preemption.

Bảng 4.2. Priority của các Task

Task	Priority
TestCtrl	5
IntrusionAlert	4
SmokeSensor	4
FactoryHigh	3
Navigation	3
CriticalWork	3
MeasuredTask	3
DoorSensor	2
VisitorCounter	2
CameraScan	2
TempSensor	2
WindSensor	2
EventProducer	2
LogConsumer	2
FactoryMed	2
CPUOverload	2
FactoryLow	1

Task TestCtrl được cấu hình với mức ưu tiên cao nhất để điều phối test suite. Task IntrusionAlert và SmokeSensor được gán mức ưu tiên cao nhằm đảm bảo Scheduler thực hiện preemption ngay khi xuất hiện sự kiện khẩn cấp. Các task thuộc

nhóm FactoryHigh, Navigation, CriticalWork và MeasuredTask phục vụ các bài kiểm thử Priority Scheduling, Concurrency và Timing Analysis. Task DoorSensor, VisitorCounter, CameraScan, TempSensor, WindSensor, EventProducer và LogConsumer được cấu hình ở mức ưu tiên trung bình, còn FactoryLow được gán mức ưu tiên thấp nhất.

4.4 Cài đặt đồng bộ

Các cơ chế đồng bộ được triển khai bằng mutex, semaphore và queue nhằm hỗ trợ trao đổi dữ liệu và bảo vệ tài nguyên dùng chung.

Bảng 4.3. Cơ chế đồng bộ

Thành phần	Cơ chế
Sensor Data Queue	Queue
Communication Buffer	Queue
Shared Log Buffer	Mutex
System Status Variable	Mutex
Event Notification	Semaphore

Queue được sử dụng để truyền dữ liệu giữa EventProducer và LogConsumer. Mutex bảo vệ các tài nguyên dùng chung như Runtime Log Buffer và Runtime Metrics. Semaphore đồng bộ hóa các sự kiện giữa các task. Các cơ chế này hỗ trợ triển khai các test case Synchronization Testing và Concurrency Testing. được sử dụng để truyền dữ liệu giữa Sensor_Task, Control_Task và Communication_Task. Mutex được áp dụng cho các tài nguyên dùng chung như vùng log và biến trạng thái hệ thống. Semaphore được sử dụng để đồng bộ hóa các sự kiện giữa các task trong quá trình kiểm thử.

4.5 Cài đặt Deadline Monitor, Logger và Fault Injector

Các task được thiết kế để thu thập dữ liệu Runtime thông qua cơ chế LogCSV, lưu vào file log phục vụ phân tích sau khi hoàn thành kiểm thử.

Bên cạnh đó, các task CPUOverload và CriticalWork được sử dụng để tạo các tình huống Fault Injection, bao gồm CPU Overload, Artificial Delay, Queue Blocking và Resource Contention. Các tình huống này cho phép đánh giá độ ổn định của Scheduler và khả năng đáp ứng của hệ thống trong điều kiện hoạt động không lý tưởng.

Bảng 4.4. Dữ liệu Runtime được thu thập

Dữ liệu	Mục đích
---------	----------

Execution Time	Phân tích thời gian thực thi
Response Time	Phân tích khả năng đáp ứng
WCET	Phân tích trường hợp xấu nhất
Context Switch Count	Đánh giá hiệu quả Scheduler
Deadline Miss	Đánh giá Pass/Fail của deadline

Bên cạnh đó, Fault_Injector_Task được sử dụng để tạo các điều kiện bất lợi trong quá trình kiểm thử. Một số tình huống được triển khai bao gồm CPU Overload, Artificial Delay, Queue Blocking và Resource Contention. Các tình huống này cho phép đánh giá độ ổn định của scheduler và khả năng đáp ứng của hệ thống trong điều kiện hoạt động không lý tưởng.

4.6 Tổng kết chương

Chương 4 đã trình bày quá trình cài đặt các thành phần chính của hệ thống kiểm thử trên nền tảng FreeRTOS Simulator. Nhóm đã triển khai các task chức năng, cấu hình mức ưu tiên, xây dựng cơ chế đồng bộ bằng Queue, Mutex và Semaphore, đồng thời cài đặt các thành phần Runtime Logging và Fault Injection phục vụ hoạt động kiểm thử.

Các thành phần sau khi cài đặt đã tạo thành một môi trường mô phỏng hoàn chỉnh, cho phép triển khai các nhóm kiểm thử về Scheduler, Priority Scheduling, Priority Preemption, Synchronization, Deadline Handling và Fault Injection. Dữ liệu Runtime thu thập được sẽ là cơ sở cho việc đánh giá PASS/FAIL và phân tích kết quả thực nghiệm trong các chương tiếp theo.

Trên cơ sở hệ thống đã được cài đặt, Chương 5 sẽ trình bày quá trình thiết kế bộ kiểm thử, xây dựng các Test Case và xác định tiêu chí đánh giá cho từng nhóm kiểm thử của đề tài.

CHƯƠNG 5 – THIẾT KẾ KIỂM THỬ

Sau khi hoàn thành việc thiết kế và cài đặt hệ thống FreeRTOS Simulator, chương này trình bày chiến lược kiểm thử, tiêu chí đánh giá pass/fail và các test suite.

5.1 Chiến lược kiểm thử

Hoạt động kiểm thử được xây dựng theo hướng thực nghiệm kết hợp phân tích dữ liệu Runtime. Mục tiêu của quá trình kiểm thử không chỉ xác nhận hệ thống thực thi thành công mà còn đánh giá mức độ đáp ứng của các cơ chế trong môi trường FreeRTOS Simulator.

Mỗi Test Suite gồm nhiều Test Case đánh giá một hoặc nhiều cơ chế cụ thể của hệ thống. Trong mỗi Test Case, nhóm xác định rõ mục tiêu kiểm thử, kịch bản thực hiện, các task tham gia (DoorSensor, VisitorCounter, CameraScan, IntrusionAlert, SmokeSensor, TempSensor, WindSensor, EventProducer, LogConsumer, Navigation, CriticalWork, CPUOverload, MeasuredTask, TestCtrl), dữ liệu đầu vào, Metrics cần thu thập, kết quả mong đợi và tiêu chí PASS/FAIL. Dữ liệu Runtime được ghi nhận thông qua cơ chế LogCSV.

Chiến lược này cho phép đánh giá toàn diện hành vi của FreeRTOS Scheduler, khả năng đáp ứng Deadline, hiệu quả của Synchronization và khả năng xử lý lỗi của hệ thống. Kết quả sẽ được phân tích trong Chương 6..

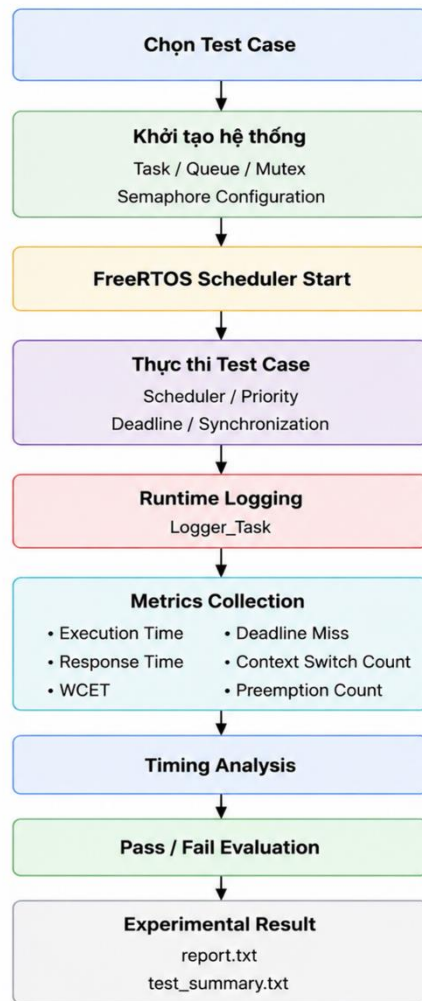
5.2 Quy trình kiểm thử

Nhằm Quy trình kiểm thử thống nhất cho toàn bộ Test Suite:

1. Chọn Test Case và cấu hình các task cần thiết.
2. Kích hoạt Scheduler của FreeRTOS.
3. Task thực thi và tương tác qua Queue, Semaphore và Mutex.
4. Thu thập dữ liệu Runtime qua LogCSV.
5. Tính toán các Metrics và so sánh với tiêu chí PASS/FAIL.

Workflow này áp dụng cho tất cả các nhóm kiểm thử: Scheduler Testing, Priority Scheduling, Priority Preemption, Synchronization Testing, Concurrency Testing, Fault Injection Testing và Timing Analysis.

Hình 5.1. Quy trình thực hiện kiểm thử của hệ thống.



5.3 Môi trường kiểm thử

Để phục vụ quá trình kiểm thử và đánh giá hệ thống, đề tài sử dụng môi trường FreeRTOS Windows Simulator nhằm mô phỏng hoạt động của hệ thống nhúng đa tiến trình thời gian thực trên máy tính. Toàn bộ các task được triển khai bằng ngôn ngữ C và thực thi dưới sự quản lý của FreeRTOS Scheduler.

Bảng 5.1. Môi trường kiểm thử

Thành phần	Cấu hình
Nền tảng kiểm thử	FreeRTOS Windows Simulator
Ngôn ngữ lập trình	C
Hệ điều hành	Windows 10/11
Scheduler	Priority-Based Preemptive Scheduler
Tick Rate	1 ms

Logging Format	CSV + Text Summary
Metrics Collection	Execution Time, Response Time, WCET, Context Switch Count, Deadline Miss, Priority Preemption Count
Công cụ phát triển	Visual Studio 2022
Compiler	Microsoft Visual C++ (MSVC)

FreeRTOS Simulator cho phép kiểm thử và thu thập dữ liệu linh hoạt mà không cần phần cứng thực, đồng thời vẫn hỗ trợ đầy đủ các cơ chế cốt lõi của FreeRTOS, phù hợp cho việc đánh giá hệ thống nhúng thời gian thực đa tác vụ.

Trong quá trình thiết kế kiểm thử, các kịch bản được xây dựng theo ngữ cảnh thực tế nhằm giúp việc mô tả dễ hiểu và phản ánh đúng các bài toán thường gặp trong hệ thống nhúng thời gian thực. Tuy nhiên, để đơn giản hóa việc triển khai trên FreeRTOS Simulator, nhiều kịch bản sử dụng chung các task triển khai. Bảng 5.2 trình bày mối liên hệ giữa tên task trong kịch bản kiểm thử và tên task thực tế được sử dụng trong mã nguồn cũng như Runtime Log.

Bảng 5.2. Ánh xạ giữa Task Logic và Task Triển khai trong FreeRTOS Simulator

Task Logic trong kịch bản	Task triển khai trong FreeRTOS Simulator
DoorSensor	Sensor_Task
VisitorCounter	Sensor_Task
CameraScan	Sensor_Task
SmokeSensor	Sensor_Task
TempSensor	Sensor_Task
WindSensor	Communication_Task
IntrusionAlert	Emergency_Task
LogConsumer	Logger_Task
EventProducer	Sensor_Task
Navigation	Control_Task

CriticalWork	Control_Task
MeasuredTask	Control_Task
CPUOverload	Fault_Injector_Task
TestCtrl	TEST_CTRL

Thông qua cơ chế ánh xạ này, các kịch bản kiểm thử vẫn giữ được tính trực quan và gần với các bài toán thực tế, đồng thời bảo đảm tính nhất quán giữa tài liệu thiết kế kiểm thử, mã nguồn triển khai và dữ liệu Runtime Log được sử dụng trong Chương 6. Trong phần kết quả thực nghiệm, các tên task xuất hiện trong Runtime Log và Metrics sẽ được sử dụng theo tên triển khai thực tế như Sensor_Task, Control_Task, Emergency_Task, Logger_Task, Communication_Task và Fault_Injector_Task để bảo đảm tính chính xác khi đối chiếu với dữ liệu thu thập từ hệ thống.

5.4 Runtime Metrics và Tiêu chí Pass/Fail

Để đánh giá hệ thống một cách định lượng, đề tài sử dụng các chỉ số runtime được thu thập trong quá trình thực thi trên môi trường FreeRTOS Simulator. Các chỉ số này là cơ sở để đánh giá hiệu năng của scheduler, khả năng đáp ứng thời gian thực và kết quả PASS/FAIL của từng nhóm kiểm thử.

Thay vì chỉ đánh giá kết quả đầu ra, đề tài tập trung theo dõi các chỉ số phản ánh trực tiếp hoạt động của scheduler, khả năng đáp ứng deadline và mức độ ổn định của hệ thống khi nhiều task thực thi đồng thời.

Bảng 5.3. Runtime Metrics sử dụng trong kiểm thử

Metric	Mục đích sử dụng
Execution Time	Đánh giá thời gian thực thi của task
Response Time	Đánh giá khả năng phản hồi của hệ thống
WCET	Phân tích trường hợp xấu nhất của task
Context Switch Count	Đánh giá hoạt động của Scheduler
Deadline Miss Count	Đánh giá khả năng đáp ứng deadline
Priority Preemption Count	Khả năng chiếm quyền của task ưu tiên cao

Các chỉ số Execution Time, Response Time và WCET được sử dụng để đánh giá khả năng đáp ứng thời gian thực của các task. Context Switch Count phản ánh hoạt động

của scheduler trong quá trình chuyển đổi giữa các task. Deadline Miss Count được sử dụng để xác định các trường hợp vi phạm deadline, trong khi Priority Preemption Count hỗ trợ đánh giá cơ chế chiếm quyền của các task ưu tiên cao.

Để đảm bảo tính khách quan, mỗi test suite được xây dựng với tiêu chí PASS/FAIL riêng dựa trên hành vi mong đợi của scheduler, khả năng đáp ứng deadline và kết quả runtime metrics thu được.

Bảng 5.4. Tiêu chí Pass/Fail cho các nhóm kiểm thử

Nhóm kiểm thử	PASS	FAIL
Scheduler Testing	Task thực thi đúng thứ tự ưu tiên	Task thực thi sai thứ tự
Priority Scheduling	Task ưu tiên cao thực thi trước task ưu tiên thấp	Task ưu tiên thấp thực thi trước task ưu tiên cao
Priority Preemption	IntrusionAlert chiếm CPU thành công	Không xảy ra preemption
Hard Deadline Testing	Không có Deadline Miss	Có Deadline Miss
Soft Deadline Testing	Hoàn thành trong ngưỡng cho phép	Trễ vượt ngưỡng cho phép
Synchronization Testing	Không xảy ra Race Condition	Dữ liệu không nhất quán
Concurrency Testing	Hệ thống hoạt động ổn định	Deadlock hoặc Starvation
Fault Injection Testing	Hệ thống tiếp tục hoạt động	Hệ thống treo hoặc mất phản hồi
Timing Analysis Testing	Các metrics trong giới hạn thiết kế	Metrics vượt ngưỡng thiết kế

Để tăng tính truy vết, đề tài xây dựng nhằm liên kết các mục tiêu nghiên cứu với nhóm kiểm thử, chỉ số runtime và tiêu chí pass/fail. Bảng này giúp bảo đảm mọi mục tiêu của đề tài đều được kiểm chứng bằng các dữ liệu định lượng cụ thể.

Bảng 5.5. Master Test Matrix cho hoạt động kiểm thử

Objective	Test Suite	Metrics	Pass Criteria
Scheduler Evaluation	Scheduler Testing	Execution Order, Context Switch Count	Thực thi đúng thứ tự
Priority Scheduling	Priority Scheduling Testing	Response Time	Task ưu tiên cao thực thi trước
Priority Preemption	Priority Preemption Testing	Response Time, Priority Preemption Count	IntrusionAlert chiếm CPU thành công
Hard Deadline Evaluation	Hard Deadline Testing	Deadline Miss Count, WCET	Không có Deadline Miss
Soft Deadline Evaluation	Soft Deadline Testing	Response Time	Trong ngưỡng cho phép
Synchronization Evaluation	Synchronization Testing	Runtime Log	Không xảy ra Race Condition
Concurrency Evaluation	Concurrency Testing	Context Switch Count	Hệ thống hoạt động ổn định
Fault Injection Evaluation	Fault Injection Testing	Execution Time, Response Time	Hệ thống tiếp tục hoạt động
Timing Analysis	Timing Analysis Testing	WCET, Execution Time, Response Time	Các chỉ số nằm trong giới hạn thiết kế

Thông qua Master Test Matrix, các nhóm kiểm thử được liên kết trực tiếp với mục tiêu nghiên cứu, đồng thời cung cấp cơ sở thống nhất cho việc thu thập dữ liệu runtime và đánh giá kết quả kiểm thử. Trong quá trình đánh giá, dữ liệu runtime được sử dụng để so sánh giữa kết quả mong đợi và kết quả thực tế của từng test case.

5.5 Scheduler Testing

Scheduler là thành phần chịu trách nhiệm lựa chọn task được cấp CPU tại từng thời điểm. Trong hệ thống của đề tài, Scheduler phải quản lý nhiều task có mức ưu tiên khác nhau và đảm bảo các task được thực thi theo đúng chính sách Priority-Based Preemptive Scheduling của FreeRTOS.

Nhóm kiểm thử này tập trung đánh giá khả năng điều phối task của FreeRTOS Scheduler thông qua các chỉ số như Execution Order, Response Time và Context Switch Count. Mục tiêu là xác nhận Scheduler hoạt động đúng trước khi tiến hành các nhóm kiểm thử liên quan đến Priority Preemption, Synchronization và Deadline Handling.

Bảng 5.6. Scheduler Testing Summary

Mã Test Case	Kịch bản kiểm thử	Metrics chính	Pass Criteria
SCH-01	Task Thực Thi Theo Priority	Execution Order	Các task được thực thi theo đúng thứ tự ưu tiên
SCH-02	Nhiều Task Đồng Thời	Response Time	Scheduler duy trì hoạt động ổn định khi nhiều task đồng thời Ready
SCH-03	Task Nền Không Ảnh Hưởng	Context Switch Count	Task nền (LogConsumer) không làm ảnh hưởng đến task chính

5.5.1 SCH-01 — Sensor Kiểm Soát Người Ra Vào

Mục tiêu kiểm thử: Đánh giá khả năng Scheduler thực thi các task theo đúng mức ưu tiên khi nhiều task cùng Ready. Xác minh Scheduler cấp CPU đúng thứ tự cho các task DoorSensor và VisitorCounter.

Bảng 5.7. SCH-01 Configuration

Thuộc tính	Giá trị
Test Suite	Scheduler Testing
Kịch bản	Task Thực Thi Theo Priority
Task tham gia	DoorSensor, VisitorCounter
Metrics	Execution Order
Expected Result	Task ưu tiên cao được thực thi trước

Pass Criteria	Thứ tự thực thi đúng thiết kế
---------------	-------------------------------

Quy trình thực hiện:

1. Khởi tạo DoorSensor và VisitorCounter.
2. Đưa các task vào trạng thái Ready.
3. Kích hoạt Scheduler.
4. Thu thập Runtime Log thông qua LogCSV.
5. Kiểm tra thứ tự task được cấp CPU và Activation Count.

Kết quả mong đợi: DoorSensor và VisitorCounter đều được Scheduler cấp CPU. Runtime Log ghi nhận đầy đủ thứ tự thực thi của task.

Tiêu chí PASS / FAIL:

- **PASS:** DoorSensor thực thi trước VisitorCounter, cả hai task đều hoàn thành theo chu kỳ định kỳ, Runtime Log đầy đủ.
- **FAIL:** Một task không được thực thi, Runtime Log thiếu dữ liệu, Scheduler hoạt động bất thường.

5.5.2 SCH-02 — Trạm Quan Trắc Thời Tiết

Mục tiêu kiểm thử: Đánh giá khả năng Scheduler duy trì hoạt động ổn định khi nhiều task đồng thời Ready (concurrent execution).

Bảng 5.8. SCH-02 Configuration

Thuộc tính	Giá trị
Test Suite	Scheduler Testing
Kịch bản	Trạm Quan Trắc Thời Tiết
Task tham gia	TempSensor, WindSensor
Metrics	Response Time
Expected Result	Các task đều được Scheduler cấp CPU định kỳ, không bị starvation
Pass Criteria	Response Time nằm trong giới hạn thiết kế

Quy trình thực hiện:

1. Khởi tạo TempSensor và WindSensor.
2. Đặt task vào trạng thái Ready.

3. Kích hoạt Scheduler.
4. Thu thập Response Time và Runtime Log.
5. So sánh với chu kỳ thiết kế.

Kết quả mong đợi: Các task TempSensor và WindSensor đều chạy định kỳ, không bị gián đoạn hoặc bỏ qua.

Tiêu chí PASS / FAIL:

1. PASS: Tất cả task hoạt động đúng chu kỳ, Response Time trong ngưỡng cho phép.
2. FAIL: Task bị bỏ qua hoặc Response Time vượt ngưỡng.

5.5.3 SCH-03 — Hệ Thống Ghi Log Runtime

Mục tiêu kiểm thử: Đánh giá ảnh hưởng của task nền đối với các task thời gian thực, đảm bảo LogConsumer không làm giảm khả năng đáp ứng của task chính.

Bảng 5.9. SCH-03 Configuration

Thuộc tính	Giá trị
Test Suite	Scheduler Testing
Kịch bản	Hệ Thống Ghi Log Runtime
Task tham gia	LogConsumer, DoorSensor, CameraScan
Metrics	Context Switch Count
Expected Result	Task nền không ảnh hưởng task chính
Pass Criteria	Các task chính vẫn hoạt động bình thường

Quy trình thực hiện:

1. Khởi tạo LogConsumer, DoorSensor, CameraScan.
2. Kích hoạt Scheduler.
3. Thu thập Context Switch Count và Runtime Log.
4. Phân tích dữ liệu để đánh giá PASS/FAIL.

Kết quả mong đợi: Task nền LogConsumer chạy liên tục nhưng không làm ảnh hưởng đến DoorSensor và CameraScan. Runtime Log ghi nhận đầy đủ các lần kích hoạt task chính.

Tiêu chí PASS / FAIL:

- **PASS:** Task chính hoàn thành đúng chu kỳ, Context Switch Count bình thường.
- **FAIL:** Task nền làm ảnh hưởng đến Scheduler, task chính thực thi sai hoặc bỏ qua.

Nhóm kiểm thử Scheduler Testing gồm 3 test case nhằm đánh giá khả năng lập lịch của FreeRTOS Scheduler trong môi trường đa nhiệm. Các kết quả PASS/FAIL sẽ làm cơ sở cho các nhóm kiểm thử Priority Scheduling, Priority Preemption và Timing Analysis trong các mục tiếp theo.

5.6 Priority Scheduling và Priority Preemption Testing

Dựa trên cơ chế Scheduler và mô hình ưu tiên đã thiết kế trong Chương 3, nhóm xây dựng bộ kiểm thử Priority Scheduling và Priority Preemption nhằm đánh giá khả năng phân bổ CPU theo mức ưu tiên và phản ứng của hệ thống trước các sự kiện quan trọng.

Trong hệ thống, mỗi task được gán một mức priority riêng, trong đó IntrusionAlert và CameraScan là task ưu tiên cao để mô phỏng các tình huống khẩn cấp. Các test case tập trung kiểm tra khả năng Scheduler lựa chọn task có ưu tiên cao nhất, thực thi đúng thứ tự ưu tiên và cho phép task ưu tiên cao chiếm quyền thực thi khi cần thiết.

Bảng 5.10. Priority Scheduling & Preemption Testing Summary

Mã Test Case	Kịch bản kiểm thử	Metrics chính	Pass Criteria
PRI-01	Camera An Ninh Phát Hiện Xâm Nhập	Response Time, Context Switch Count	IntrusionAlert được thực thi đầu tiên
PRI-02	Drone Tránh Chướng Ngại Vật	Response Time, Deadline Miss Count	Navigation được ưu tiên thực thi
PRE-01	Nhà Máy Có Emergency Stop	Response Time, Priority Preemption Count	Preemption xảy ra thành công

5.6.1 PRI-01 —Camera An Ninh Phát Hiện Xâm Nhập

Mục tiêu kiểm thử: Đánh giá cơ chế Priority Scheduling. Scheduler phải cấp CPU ngay cho IntrusionAlert khi sự kiện xâm nhập xảy ra trong lúc các task khác đang chạy.

Bảng 5.11. PRI-01 Configuration

Thuộc tính	Giá trị
Test Suite	Priority Scheduling
Kịch bản	Xử lý sự kiện ưu tiên cao
Task tham gia	CameraScan, IntrusionAlert, LogConsumer
Metrics	Response Time, Context Switch Count
Expected Result	IntrusionAlert được thực thi trước các task khác
Pass Criteria	Scheduler cấp CPU đúng thứ tự ưu tiên

Quy trình thực hiện:

1. Khởi tạo CameraScan, IntrusionAlert, LogConsumer.
2. Đưa tất cả task vào trạng thái Ready.
3. Kích hoạt Scheduler.
4. Thu thập Runtime Log.
5. Kiểm tra Response Time và Context Switch Count để xác định PASS/FAIL.

Kết quả mong đợi: IntrusionAlert được Scheduler cấp CPU ngay khi sự kiện xâm nhập xảy ra; CameraScan và LogConsumer được thực thi đúng thứ tự còn lại.

5.6.2 PRI-02 — Drone Tránh Chướng Ngại Vật

Mục tiêu kiểm thử: Đánh giá cơ chế Priority Scheduling khi nhiều task liên quan đến drone di chuyển và cảm biến hoạt động đồng thời.

Bảng 5.12. PRI-02 Configuration

Thuộc tính	Giá trị
Test Suite	Priority Scheduling
Kịch bản	Drone Tránh Chướng Ngại Vật
Task tham gia	FrontSensor, Navigation, LogConsumer
Metrics	Response Time, Deadline Miss Count
Expected Result	Navigation được ưu tiên thực thi khi nhận dữ liệu từ cảm biến

Pass Criteria	Không xuất hiện Deadline Miss
---------------	-------------------------------

Quy trình thực hiện:

1. Khởi tạo FrontSensor, Navigation, LogConsumer.
2. Scheduler cấp CPU cho các task.
3. FrontSensor tạo dữ liệu liên tục.
4. Navigation phản hồi ngay dữ liệu, ghi lại Response Time và Deadline Miss.
5. Đánh giá PASS/FAIL dựa trên metrics.

Kết quả mong đợi: Navigation xử lý dữ liệu đúng thời hạn, Response Time trong giới hạn thiết kế, không xuất hiện Deadline Miss.

5.6.3 PRE-01 — Nhà Máy Có Emergency Stop

Mục tiêu kiểm thử: Đánh giá cơ chế Priority Preemption của Scheduler. Khi tín hiệu Emergency Stop xuất hiện, task ưu tiên cao phải chiếm CPU ngay lập tức.

Bảng 5.13. PRE-01 Configuration

Thuộc tính	Giá trị
Test Suite	Priority Preemption
Kịch bản	Nhà Máy Có Emergency Stop
Task tham gia	Control_Task, Logger_Task, Emergency_Task
Metrics	Response Time, Priority Preemption Count
Expected Result	Emergency_Task được thực thi ngay sau khi kích hoạt
Pass Criteria	Preemption xảy ra thành công

Quy trình thực hiện:

1. Scheduler đang chạy các task Navigation và CriticalWork.
2. Sự kiện Emergency Stop kích hoạt IntrusionAlert.
3. Scheduler thực hiện Context Switch ngay lập tức.
4. Thu thập Priority Preemption Count và Response Time.
5. Đánh giá PASS/FAIL dựa trên metrics.

Kết quả mong đợi: IntrusionAlert chiếm CPU ngay khi sự kiện xảy ra; các task còn lại bị tạm dừng đúng theo cơ chế Priority Preemption. Metrics phản ánh số lần Preemption và thời gian phản hồi.

Hình 5.2. Priority Preemption Workflow trong kịch bản Emergency Stop



Thông qua nhóm kiểm thử Priority Scheduling và Priority Preemption, đề tài đánh giá khả năng ưu tiên các nhiệm vụ quan trọng và phản ứng của Scheduler trước các sự kiện khẩn cấp. Đây là cơ sở để tiếp tục đánh giá khả năng đáp ứng deadline trong các mục tiếp theo.

5.7 Hard Deadline và Soft Deadline Testing

Dựa trên mô hình deadline đã thiết kế trong Chương 3, nhóm triển khai bộ kiểm thử Hard Deadline và Soft Deadline nhằm đánh giá khả năng đáp ứng thời gian thực của hệ thống. Các task trong hệ thống được cấu hình với mức ưu tiên và deadline khác nhau để mô phỏng các yêu cầu thời gian thực thường gặp trong hệ thống nhúng.

Thông qua các kịch bản kiểm thử, nhóm đánh giá khả năng đáp ứng deadline của scheduler và phân tích các chỉ số thời gian thu thập được từ FreeRTOS Simulator.

Bảng 5.14. Hard Deadline và Soft Deadline Testing Summary

Mã Test Case	Kịch bản kiểm thử	Loại Deadline	Metrics chính
--------------	-------------------	---------------	---------------

HD-01	Hệ Thống Báo Cháy Thông Minh	Hard Deadline	Response Time, Deadline Miss Count
HD-02	Nhà Máy Có Emergency Stop	Hard Deadline	WCET, Deadline Miss Count
SD-01	Trạm Quan Trắc Thời Tiết	Soft Deadline	Response Time
SD-02	Trạm Quan Trắc Chất Lượng Không Khí	Soft Deadline	Response Time

5.7.1 HD-01 — Hệ Thống Báo Cháy Thông Minh

Mục tiêu kiểm thử: Đánh giá khả năng đáp ứng Hard Deadline khi hệ thống báo cháy phát hiện nguy cơ cháy nổ. Scheduler phải cấp CPU ngay cho task ưu tiên cao để xử lý sự kiện.

Bảng 5.15. HD-01 Configuration

Thuộc tính	Giá trị
Test Suite	Hard Deadline Testing
Kịch bản	Hệ Thống Báo Cháy Thông Minh
Task tham gia	SmokeSensor, IntrusionAlert
Deadline	50 ms
Metrics	Response Time, Deadline Miss Count
Expected Result	Không xuất hiện Deadline Miss
Pass Criteria	Deadline Miss Count = 0

Quy trình thực hiện:

1. Khởi tạo task SmokeSensor và IntrusionAlert.
2. Scheduler cấp CPU theo mức ưu tiên.
3. Thu thập Response Time và Runtime Log.
4. So sánh thời gian hoàn thành với Deadline.

Kết quả mong đợi: Task ưu tiên cao hoàn thành xử lý trước 50 ms; không có Deadline Miss.

Tiêu chí PASS / FAIL:

- PASS: Thời gian thực thi ≤ 50 ms, không có Deadline Miss.
- FAIL: Task hoàn thành chậm hoặc xuất hiện Deadline Miss.

5.7.2 HD-02 — Nhà Máy Có Emergency Stop

Mục tiêu kiểm thử: Đánh giá Hard Deadline khi hệ thống chịu tải cao và xuất hiện tín hiệu Emergency Stop. Scheduler phải cấp CPU ngay cho task ưu tiên cao (CriticalWork hoặc IntrusionAlert).

Bảng 5.16. HD-02 Configuration

Thuộc tính	Giá trị
Test Suite	Hard Deadline Testing
Kịch bản	Nhà Máy Có Emergency Stop
Task tham gia	CriticalWork, IntrusionAlert
Deadline	25 ms
Metrics	WCET, Deadline Miss Count
Expected Result	Task ưu tiên cao hoàn thành trước deadline
Pass Criteria	$WCET \leq \text{Deadline}$

Quy trình thực hiện:

1. Khởi tạo các task CriticalWork và IntrusionAlert.
2. Scheduler cấp CPU theo priority.
3. Thu thập WCET và Runtime Log.
4. So sánh thời gian thực thi với Deadline.

Kết quả mong đợi: Task ưu tiên cao hoàn thành xử lý trước 25 ms; hệ thống phản ứng kịp thời.

Tiêu chí PASS / FAIL:

- PASS: $WCET \leq 25$ ms và không có Deadline Miss.
- FAIL: WCET vượt hoặc xuất hiện Deadline Miss.

5.7.3 SD-01 — Trạm Quan Trắc Thời Tiết

Mục tiêu kiểm thử: Đánh giá Soft Deadline khi task cập nhật dữ liệu định kỳ. Scheduler phải đảm bảo task thực thi gần deadline nhưng một mức độ trễ nhỏ được chấp nhận.

Bảng 5.17. SD-01 Configuration

Thuộc tính	Giá trị
Test Suite	Soft Deadline Testing
Kịch bản	Trạm Quan Trắc Thời Tiết
Task tham gia	TempSensor, WindSensor
Deadline	40 ms
Metrics	Response Time
Expected Result	Task hoàn thành gần deadline
Pass Criteria	Response Time trong ngưỡng cho phép

Quy trình thực hiện:

1. Khởi tạo TempSensor và WindSensor.
2. Scheduler cấp CPU theo priority.
3. Thu thập Response Time và Runtime Log.
4. So sánh thời gian hoàn thành với giới hạn cho phép.

Kết quả mong đợi: Các task chạy định kỳ, không bị bỏ qua, Response Time nằm trong ngưỡng.

Tiêu chí PASS / FAIL:

- **PASS:** Response Time ≤ 40 ms, task hoàn thành theo chu kỳ.
- **FAIL:** Task bỏ qua hoặc Response Time vượt quá giới hạn.

5.7.4 SD-02 — Hệ Thống Giám Sát Chất Lượng Không Khí

Mục tiêu kiểm thử: Đánh giá khả năng hệ thống duy trì Soft Deadline khi thu thập dữ liệu chất lượng không khí định kỳ. Scheduler phải đảm bảo task hoạt động liên tục với một mức độ trễ chấp nhận được.

Bảng 5.18. SD-02 Configuration

Thuộc tính	Giá trị
Test Suite	Soft Deadline Testing
Kịch bản	Trạm Quan Trắc Chất Lượng Không Khí
Task tham gia	TempSensor, WindSensor, LogConsumer

Deadline	100 ms
Metrics	Response Time
Expected Result	Hệ thống duy trì hoạt động ổn định
Pass Criteria	Response Time trong ngưỡng cho phép

Quy trình thực hiện:

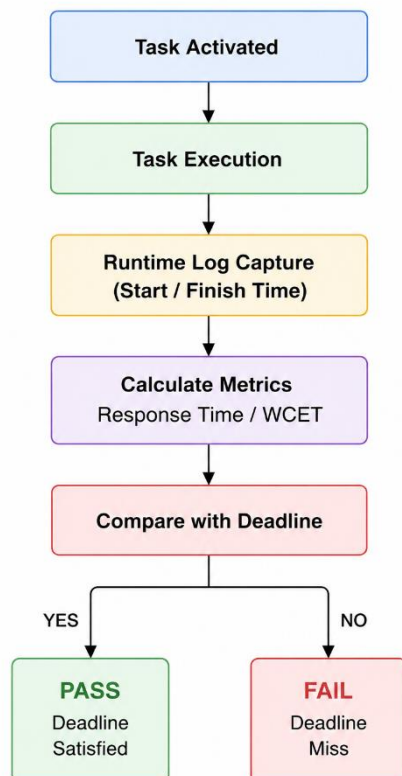
1. Khởi tạo TempSensor, WindSensor, LogConsumer.
2. Scheduler cấp CPU cho các task theo priority.
3. Thu thập Response Time và Runtime Log.
4. Phân tích dữ liệu để đánh giá khả năng đáp ứng.

Kết quả mong đợi: Các task thực thi định kỳ, Response Time nằm trong ngưỡng cho phép.

Tiêu chí PASS / FAIL:

- **PASS:** Task hoàn thành đúng chu kỳ, hệ thống ổn định.
- **FAIL:** Task bỏ qua, Response Time vượt ngưỡng, hoặc hệ thống bị gián đoạn.

Hình 5.3. Quy trình đánh giá Deadline (Deadline Evaluation Workflow)



Thông qua nhóm kiểm thử Hard Deadline và Soft Deadline, đề tài đánh giá khả năng đáp ứng thời gian thực của FreeRTOS Scheduler trong nhiều điều kiện vận hành khác nhau. Kết quả thu được sẽ được sử dụng ở Chương 6 để phân tích Deadline Miss, Response Time, WCET và mức độ ổn định của hệ thống.

5.8 Đồng bộ và Concurrency Testing

Để đánh giá hiệu quả của các cơ chế đồng bộ, nhóm xây dựng bộ kiểm thử Synchronization và Concurrency Testing dựa trên các tình huống nhiều task cùng truy cập tài nguyên dùng chung. Các test case tập trung kiểm tra khả năng bảo vệ bộ đệm log dùng chung, quản lý giao tiếp Queue và duy trì hoạt động ổn định của hệ thống khi nhiều task thực thi đồng thời.

Trong hệ thống FreeRTOS Simulator, các cơ chế Mutex, Semaphore và Queue được sử dụng để quản lý việc truy cập tài nguyên dùng chung giữa các task. Nhóm kiểm thử Synchronization và Concurrency Testing được xây dựng nhằm đánh giá tính đúng đắn của các cơ chế đồng bộ hóa cũng như khả năng duy trì hoạt động ổn định khi nhiều task thực thi đồng thời.

Bảng 5.19. Synchronization và Concurrency Testing Summary

Mã Test Case	Kịch bản	Cơ chế
SYN-01	Hệ Thống Ghi Log Nhà Thông Minh	Mutex
SYN-02	Camera An Ninh Ghi Log Đồng Thời	Queue
CON-01	Drone Xử Lý Nhiều Cảm Biến	Concurrency
CON-02	Nhà Máy Nhiều Tín Hiệu Điều Khiển	Shared Resource

5.8.1 SYN-01 — Hệ Thống Ghi Log Nhà Thông Minh

Mục tiêu kiểm thử: Đánh giá khả năng bảo vệ tài nguyên dùng chung bằng Mutex. Task EventProducer, LogConsumer và CriticalWork cùng ghi dữ liệu vào Shared Log Buffer, cần đảm bảo dữ liệu log không bị sai lệch khi nhiều task truy cập đồng thời.

Bảng 5.20. SYN-01 Configuration

Thuộc tính	Giá trị
Test Suite	Synchronization Testing
Kịch bản	Hệ Thống Ghi Log Nhà Thông Minh

Shared Resource	Shared Log Buffer
Task tham gia	EventProducer, LogConsumer, CriticalWork
Cơ chế sử dụng	Mutex
Metrics	Context Switch Count
Expected Result	Không xuất hiện Race Condition
Pass Criteria	Dữ liệu log nhất quán

Quy trình thực hiện:

1. Khởi tạo các task EventProducer, LogConsumer, CriticalWork.
2. Đưa task vào trạng thái Ready.
3. Scheduler cấp CPU theo Priority.
4. Task ghi dữ liệu vào Shared Log Buffer thông qua Mutex.
5. Thu thập Context Switch Count và Runtime Log để đánh giá.

Kết quả mong đợi: Dữ liệu log luôn nhất quán, không có Race Condition.

5.8.2 SYN-02 — Camera An Ninh Ghi Log Đồng Thời

Mục tiêu kiểm thử: Đánh giá khả năng trao đổi dữ liệu giữa các task thông qua Queue. Các task CameraScan, IntrusionAlert và LogConsumer truyền dữ liệu log, kiểm tra hiện tượng mất dữ liệu hoặc xử lý không đồng bộ..

Bảng 5.21. SYN-02 Configuration

Thuộc tính	Giá trị
Test Suite	Synchronization Testing
Kịch bản	Camera An Ninh Ghi Log Đồng Thời
Shared Resource	Log Queue
Task tham gia	CameraScan, IntrusionAlert, LogConsumer
Cơ chế sử dụng	Queue
Metrics	Response Time

Expected Result	Không mất dữ liệu
Pass Criteria	Queue hoạt động ổn định

Quy trình thực hiện:

1. Khởi tạo task CameraScan, IntrusionAlert, LogConsumer.
2. Scheduler cấp CPU.
3. CameraScan và IntrusionAlert gửi dữ liệu vào Queue.
4. LogConsumer nhận dữ liệu và ghi log.
5. Thu thập Response Time và Runtime Log.

Kết quả mong đợi: Dữ liệu được truyền đầy đủ, không xảy ra mất dữ liệu hoặc lỗi đồng bộ.

5.8.3 CON-01 — Drone Xử Lý Nhiều Cảm Biến

Mục tiêu kiểm thử: Đánh giá khả năng Concurrency của Scheduler khi nhiều task cùng Ready. Task DoorSensor, VisitorCounter, CameraScan cùng hoạt động đồng thời.

Bảng 5.22. CON-01 Configuration

Thuộc tính	Giá trị
Test Suite	Concurrency Testing
Kịch bản	Drone Xử Lý Nhiều Cảm Biến
Task tham gia	DoorSensor, VisitorCounter, CameraScan
Metrics	Context Switch Count
Expected Result	Hệ thống hoạt động ổn định
Pass Criteria	Không xảy ra Starvation

Quy trình thực hiện:

1. Khởi tạo các task DoorSensor, VisitorCounter, CameraScan.
2. Scheduler cấp CPU cho các task theo Priority.
3. Thu thập Runtime Log và Context Switch Count.

Kết quả mong đợi: Tất cả task được Scheduler cấp CPU đầy đủ, không bị Starvation.

5.8.4 CON-02 — Nhà Máy Nhiều Tín Hiệu Điều Khiển

Mục tiêu kiểm thử: Đánh giá khả năng phối hợp khi nhiều task cùng truy cập tài nguyên chung (System Status Variable). Task Navigation, CriticalWork, IntrusionAlert phải phối hợp xử lý.

Bảng 5.23. CON-02 Configuration

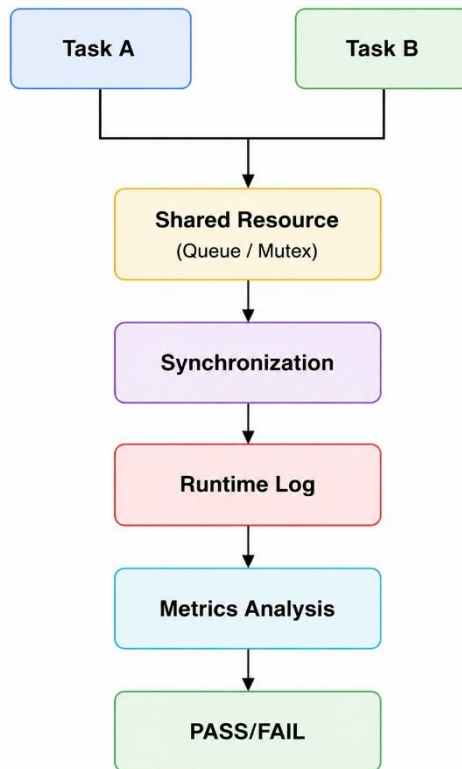
Thuộc tính	Giá trị
Test Suite	Concurrency Testing
Kịch bản	Nhà Máy Nhiều Tín Hiệu Điều Khiển
Shared Resource	System Status Variable
Task tham gia	Navigation, CriticalWork, IntrusionAlert
Metrics	Response Time
Expected Result	Không xảy ra Deadlock
Pass Criteria	Hệ thống duy trì hoạt động ổn định

Quy trình thực hiện:

1. Khởi tạo các task Navigation, CriticalWork, IntrusionAlert.
2. Scheduler cấp CPU theo priority.
3. Task truy cập tài nguyên dùng chung và phối hợp xử lý.
4. Thu thập Runtime Log và Response Time.

Kết quả mong đợi: Các task phối hợp ổn định, không có Deadlock, hệ thống hoạt động bình thường.

Hình 5.4. Quy trình kiểm thử Synchronization và Concurrency



Thông qua nhóm kiểm thử Synchronization và Concurrency, đề tài đánh giá khả năng phối hợp giữa các task trong môi trường đa nhiệm, đồng thời xác minh hiệu quả của các cơ chế Mutex, Queue và Semaphore trong việc bảo vệ tài nguyên dùng chung và duy trì tính ổn định của hệ thống.

5.9 Fault Injection Testing

Bên cạnh các điều kiện hoạt động bình thường, đề tài xây dựng bộ kiểm thử Fault Injection nhằm đánh giá độ ổn định và khả năng thích ứng của hệ thống khi xuất hiện các lỗi mô phỏng có chủ đích.

Các lỗi được tạo ra thông qua các kịch bản như CPU Overload, Queue Congestion và Artificial Delay nhằm mô phỏng các điều kiện vận hành bất lợi thường gặp trong hệ thống nhúng thời gian thực. Thông qua các kịch bản này, nhóm đánh giá khả năng duy trì hoạt động của Scheduler, cơ chế Synchronization và khả năng đáp ứng Deadline của hệ thống.

Bảng 5.24. Fault Injection Testing Summary

Mã Test Case	Kịch bản kiểm thử	Fault Type	Metrics
FLT-01	CPU Overload Trong Nhà Máy	CPU Load Injection	Response Time, Deadline Miss Count

FLT-02	Queue Blocking Khi Truyền Dữ Liệu	Queue Congestion	Response Time
FLT-03	Artificial Delay Trong Drone Control	Task Delay Injection	Response Time, WCET
FLT-04	Sensor Nhiều Dữ Liệu	Invalid Input Injection	Execution Time, Response Time

5.9.1 FLT-01 — CPU Overload Trong Nhà Máy

Mục tiêu kiểm thử: Kịch bản này mô phỏng môi trường sản xuất công nghiệp trong đó hệ thống phải xử lý khối lượng công việc lớn hơn điều kiện vận hành bình thường. Task CPUOverload được sử dụng để tạo tải xử lý bổ sung nhằm đánh giá khả năng hoạt động của Scheduler khi hệ thống chịu áp lực cao.

Bảng 5.25. FLT-01 Configuration

Thuộc tính	Giá trị
Test Suite	Fault Injection Testing
Kịch bản	CPU Overload Trong Nhà Máy
Fault Type	CPU Load Injection
Task tham gia	CPUOverload, CriticalWork, MeasuredTask
Metrics	Response Time, Deadline Miss Count
Expected Result	Hệ thống vẫn duy trì hoạt động
Pass Criteria	Không xảy ra lỗi nghiêm trọng và Deadline Miss trong ngưỡng cho phép

Quy trình thực hiện:

1. Khởi tạo CriticalWork và MeasuredTask.
2. Kích hoạt CPUOverload để tạo tải CPU nhân tạo.
3. Thu thập Runtime Log.
4. Tính toán Response Time và Deadline Miss Count.
5. Đánh giá PASS/FAIL.

Kết quả mong đợi: Các task chính vẫn được Scheduler cấp CPU và tiếp tục hoạt động mặc dù hệ thống chịu tải cao.

Tiêu chí PASS / FAIL:

- PASS:
 - Hệ thống tiếp tục hoạt động.
 - Các task chính hoàn thành xử lý.
 - Deadline Miss nằm trong ngưỡng cho phép.
- FAIL:
 - Hệ thống mất phản hồi.
 - Deadline Miss vượt ngưỡng cho phép.

5.9.2 FLT-02 — Queue Blocking Khi Truyền Dữ Liệu

Mục tiêu kiểm thử: Đánh giá khả năng xử lý của Queue khi xuất hiện lượng dữ liệu lớn trong thời gian ngắn gây hiện tượng Queue Congestion.

Bảng 5.26. FLT-02 Configuration

Thuộc tính	Giá trị
Test Suite	Fault Injection Testing
Kịch bản	Queue Blocking Khi Truyền Dữ Liệu
Fault Type	Queue Congestion
Shared Resource	Communication Queue
Task tham gia	EventProducer, LogConsumer
Metrics	Response Time
Expected Result	Queue vẫn xử lý dữ liệu
Pass Criteria	Không xảy ra mất dữ liệu nghiêm trọng

Quy trình thực hiện:

1. Khởi tạo EventProducer và LogConsumer.
2. Tăng tốc độ sinh dữ liệu từ EventProducer.
3. Theo dõi Queue.
4. Thu thập Runtime Log và Response Time.

Kết quả mong đợi: Queue tiếp tục hoạt động và dữ liệu vẫn được truyền giữa các task.

Tiêu chí PASS / FAIL:

- PASS:

- Queue không bị treo.
- Dữ liệu được xử lý liên tục.
- FAIL:
 - Queue ngừng hoạt động.
 - Xuất hiện mất dữ liệu nghiêm trọng.

5.9.3 FLT-03 — Artificial Delay Trong Drone Control

Mục tiêu kiểm thử: Đánh giá ảnh hưởng của độ trễ nhân tạo tới khả năng đáp ứng thời gian thực của hệ thống.

Bảng 5.27. FLT-03 Configuration

Thuộc tính	Giá trị
Test Suite	Fault Injection Testing
Kịch bản	Artificial Delay Trong Drone Control
Fault Type	Task Delay Injection
Task tham gia	Navigation, CriticalWork, CPUOverload
Metrics	Response Time, WCET
Expected Result	Scheduler vẫn phản ứng đúng
Pass Criteria	Response Time nằm trong ngưỡng thiết kế

Quy trình thực hiện

1. Khởi tạo Navigation và CriticalWork.
2. Chèn độ trễ nhân tạo vào quá trình xử lý.
3. Thu thập Runtime Log.
4. Tính toán Response Time và WCET.

Kết quả mong đợi: Scheduler vẫn duy trì hoạt động ổn định mặc dù thời gian thực thi tăng lên.

Tiêu chí PASS / FAIL:

- PASS:
 - Scheduler tiếp tục hoạt động.
 - Response Time nằm trong giới hạn thiết kế.
- FAIL:
 - Deadline Miss tăng bất thường.
 - Scheduler mất khả năng đáp ứng.

5.9.4 FLT-04 — Sensor Nhiều Dữ Liệu

Mục tiêu kiểm thử: Đánh giá khả năng hệ thống duy trì hoạt động ổn định khi xuất hiện dữ liệu bất thường hoặc tải cao. Kịch bản “Sensor Nhiều Dữ Liệu” mô phỏng tình huống dữ liệu đầu vào không chuẩn hoặc điều kiện thực thi bất thường nhằm kiểm tra khả năng phản ứng của Scheduler và cơ chế đồng bộ.

Trong phạm vi FreeRTOS Simulator, dữ liệu nhiễu không được tạo từ cảm biến vật lý mà được mô phỏng thông qua việc thay đổi điều kiện thực thi của task và tạo tải xử lý bất thường. Cách tiếp cận này cho phép đánh giá phản ứng của Scheduler trước các điều kiện đầu vào không ổn định tương tự dữ liệu cảm biến nhiễu trong hệ thống thực tế.

Bảng 5.28. FLT-04 Configuration

Thuộc tính	Giá trị
Test Suite	Fault Injection Testing
Kịch bản	Sensor Nhiều Dữ Liệu
Fault Type	Invalid Input Injection
Task tham gia	CPUOverload, CriticalWork, Navigation
Metrics	Execution Time, Response Time
Expected Result	Hệ thống tiếp tục hoạt động ổn định
Pass Criteria	Không gây treo hoặc dừng hệ thống

Quy trình thực hiện:

1. Khởi tạo các task CPUOverload, CriticalWork và Navigation.
2. Kích hoạt Scheduler để các task đồng thời thực thi.
3. Tạo điều kiện thực thi bất thường thông qua CPUOverload và CriticalWork.
4. Thu thập Runtime Log và tính toán Execution Time, Response Time.
5. Đánh giá PASS/FAIL dựa trên khả năng duy trì hoạt động ổn định của hệ thống.

Kết quả mong đợi:

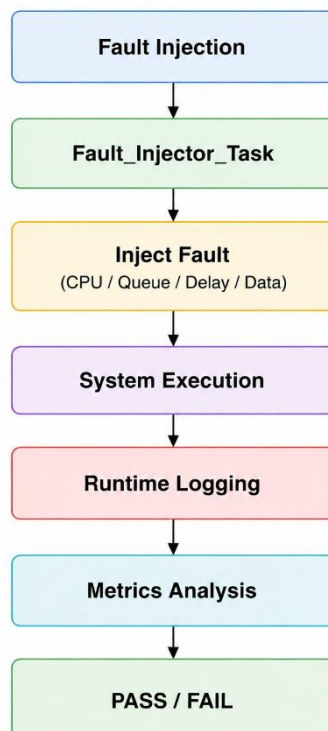
- Các task CPUOverload, CriticalWork và Navigation được Scheduler cấp CPU đầy đủ.
- Runtime Log ghi nhận đầy đủ các hoạt động của task.

- Hệ thống vẫn duy trì hoạt động ổn định mặc dù xuất hiện điều kiện bất thường.
- Không xảy ra hiện tượng treo hoặc dừng hệ thống.

Tiêu chí PASS / FAIL:

- PASS:
 - Các task tiếp tục thực thi bình thường.
 - Hệ thống duy trì hoạt động ổn định.
 - Execution Time và Response Time nằm trong giới hạn cho phép.
 - Không xuất hiện lỗi nghiêm trọng làm gián đoạn hệ thống.
- FAIL:
 - Task bị bỏ qua hoặc không được Scheduler cấp CPU.
 - Hệ thống mất phản hồi hoặc bị treo.
 - Execution Time hoặc Response Time vượt quá ngưỡng cho phép.
 - Xuất hiện lỗi nghiêm trọng ảnh hưởng tới hoạt động chung của hệ thống.

Hình 5.5. Quy trình kiểm thử Fault Injection



Thông qua nhóm kiểm thử Fault Injection, đề tài đánh giá mức độ ổn định của hệ thống trong các điều kiện lỗi có kiểm soát, đồng thời phân tích khả năng phản ứng của Scheduler, cơ chế Synchronization và xử lý Deadline khi môi trường vận hành không còn lý tưởng.

5.10 Timing Analysis Testing

Sau khi hoàn thành các nhóm kiểm thử chức năng, đề tài thực hiện Timing Analysis nhằm thu thập và phân tích các chỉ số thời gian thực của hệ thống dựa trên dữ liệu runtime từ FreeRTOS Simulator.

Các chỉ số được sử dụng bao gồm Execution Time, Response Time, WCET, Context Switch Count và Deadline Miss Count. Những chỉ số này được sử dụng để đánh giá hành vi của Scheduler, hiệu quả của cơ chế lập lịch ưu tiên và khả năng đáp ứng thời gian thực của hệ thống. Đồng thời, đây cũng là cơ sở cho việc phân tích kết quả thực nghiệm trong Chương 6.

Bảng 5.29. Timing Analysis Testing Summary

Mã Test Case	Kịch bản kiểm thử	Metrics chính
TIM-01	Phân Tích Response Time của IntrusionAlert	Response Time
TIM-02	Phân Tích WCET của CriticalWork	WCET, Execution Time
TIM-03	Phân Tích Context Switch Khi Preemption	Context Switch Count
TIM-04	Phân Tích Deadline Miss Dưới Tải Cao	Deadline Miss Count

5.10.1 TIM-01 — Phân Tích Response Time của IntrusionAlert

Mục tiêu kiểm thử: Đánh giá khả năng phản hồi của hệ thống khi xuất hiện các sự kiện ưu tiên cao. Test case tập trung đo khoảng thời gian từ khi IntrusionAlert chuyển sang trạng thái Ready cho tới khi được Scheduler cấp CPU để thực thi.

Bảng 5.30. TIM-01 Configuration

Thuộc tính	Giá trị
Test Suite	Timing Analysis
Kịch bản	Intrusion Alert Response Analysis
Task tham gia	IntrusionAlert
Metrics	Response Time
Expected Result	Response Time nhỏ hơn Deadline

Pass Criteria	Response Time ≤ 20 ms
---------------	----------------------------

Quy trình thực hiện:

1. Khởi tạo task IntrusionAlert.
2. Kích hoạt sự kiện xâm nhập.
3. Ghi nhận thời điểm task chuyển sang trạng thái Ready.
4. Ghi nhận thời điểm task bắt đầu Running.
5. Tính toán Response Time từ Runtime Log.

Kết quả mong đợi: IntrusionAlert được Scheduler phản hồi nhanh chóng và bắt đầu thực thi trong khoảng thời gian nhỏ hơn Deadline đã thiết kế.

Tiêu chí PASS / FAIL:

- PASS:
 - Response Time ≤ 20 ms.
 - IntrusionAlert được thực thi ngay sau khi kích hoạt.
- FAIL:
 - Response Time vượt quá 20 ms.
 - Task bị trì hoãn bất thường.

5.10.2 TIM-02 — Phân Tích WCET của Navigation / CriticalWork

Mục tiêu kiểm thử: Đánh giá Worst-Case Execution Time (WCET) của CriticalWork nhằm xác định thời gian thực thi lớn nhất của task trong điều kiện tải cao.

Bảng 5.31. TIM-02 Configuration

Thuộc tính	Giá trị
Test Suite	Timing Analysis
Kịch bản	Critical Processing Analysis
Task tham gia	CriticalWork
Metrics	WCET, Execution Time
Expected Result	WCET nhỏ hơn Deadline
Pass Criteria	WCET $\leq$ Deadline

Quy trình thực hiện:

1. Khởi tạo CriticalWork.
2. Thực hiện nhiều lần chạy với tải khác nhau.

3. Thu thập Execution Time từ Runtime Log.
4. Xác định giá trị WCET lớn nhất.
5. So sánh với Deadline thiết kế.

Kết quả mong đợi: CriticalWork hoàn thành xử lý trong khoảng thời gian cho phép ngay cả trong điều kiện tải cao.

Tiêu chí PASS / FAIL:

- PASS:
 - WCET không vượt quá Deadline.
 - Execution Time ổn định.
- FAIL:
 - WCET vượt Deadline.
 - Xuất hiện Deadline Miss.

5.10.3 TIM-03 — Phân Tích Context Switch khi Preemption

Mục tiêu kiểm thử: Đánh giá hành vi của Scheduler khi xảy ra Priority Preemption. Test case tập trung phân tích số lần chuyển ngữ cảnh và khả năng cấp CPU cho task ưu tiên cao.

Bảng 5.32. TIM-03 Configuration

Thuộc tính	Giá trị
Test Suite	Timing Analysis
Kịch bản	Priority Preemption Analysis
Task tham gia	IntrusionAlert, CriticalWork
Metrics	Context Switch Count
Expected Result	Context Switch diễn ra đúng
Pass Criteria	Scheduler thực hiện Preemption thành công

Quy trình thực hiện:

1. Khởi tạo CriticalWork.
2. Trong khi CriticalWork đang thực thi, kích hoạt IntrusionAlert.
3. Ghi nhận số lần Context Switch.
4. Phân tích Runtime Log.

Kết quả mong đợi: Scheduler thực hiện Context Switch và cấp CPU cho IntrusionAlert theo đúng cơ chế Priority Preemption.

Tiêu chí PASS / FAIL:

- PASS:
 - Context Switch xảy ra đúng thời điểm.
 - IntrusionAlert giành được CPU.
- FAIL:
 - Không xảy ra Preemption.
 - Context Switch hoạt động không chính xác.

5.10.4 TIM-04 — Phân Tích CPU Usage dưới Tải Cao

Mục tiêu kiểm thử: Đánh giá khả năng duy trì đáp ứng Deadline khi hệ thống hoạt động trong điều kiện tải cao với nhiều task đồng thời thực thi.

Bảng 5.33. TIM-04 Configuration

Thuộc tính	Giá trị
Test Suite	Timing Analysis
Kịch bản	High System Load Analysis
Task tham gia	CPUOverload, CriticalWork, Navigation, IntrusionAlert
Metrics	Deadline Miss Count
Expected Result	Hệ thống vẫn duy trì hoạt động ổn định
Pass Criteria	Deadline Miss Count nằm trong ngưỡng cho phép

Quy trình thực hiện:

1. Khởi tạo CPUOverload, CriticalWork, Navigation và IntrusionAlert.
2. Tăng tải CPU bằng CPUOverload.
3. Thu thập Runtime Log.
4. Phân tích Deadline Miss Count.
5. Đánh giá PASS/FAIL.

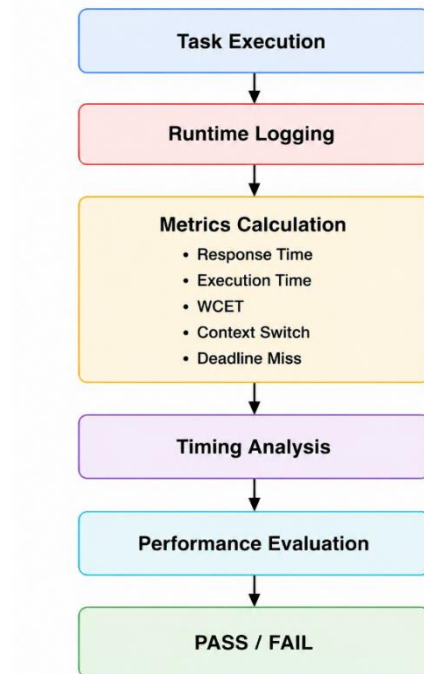
Kết quả mong đợi: Các task quan trọng vẫn được Scheduler cấp CPU và hoàn thành xử lý trong điều kiện tải cao.

Tiêu chí PASS / FAIL

- PASS:
 - Hệ thống tiếp tục hoạt động.

- Deadline Miss Count nằm trong ngưỡng cho phép.
- FAIL:
 - Deadline Miss tăng bất thường.
 - Hệ thống mất khả năng đáp ứng thời gian thực.

Hình 5.6. Quy trình Timing Analysis và đánh giá hiệu năng hệ thống



Thông qua nhóm kiểm thử Timing Analysis, đề tài xây dựng cơ sở định lượng để đánh giá hiệu năng của hệ thống FreeRTOS Simulator. Từ đó phân tích kết quả thực nghiệm, so sánh kết quả kỳ vọng và thực tế, đồng thời đánh giá mức độ đáp ứng thời gian thực của hệ thống.

5.11 Tổng hợp bộ Test

Sau khi xây dựng các nhóm kiểm thử, toàn bộ test case được tổng hợp thành một bộ kiểm thử thống nhất phục vụ quá trình thực nghiệm và đánh giá hệ thống. Việc tổ chức các test case thành các Test Suite giúp quá trình kiểm thử được thực hiện một cách có hệ thống, đồng thời đảm bảo các cơ chế quan trọng của FreeRTOS Simulator đều được đánh giá thông qua các kịch bản thực tế.

Mỗi Test Suite tập trung vào một chức năng cụ thể nhưng vẫn liên kết với nhau thông qua hệ thống metrics và tiêu chí PASS/FAIL đã được xây dựng trước đó.

Bảng 5.34. Tổng hợp Test Suite

Test Suite	Số Test Case	Trọng tâm đánh giá
Scheduler Testing	3	Execution Order, Task State

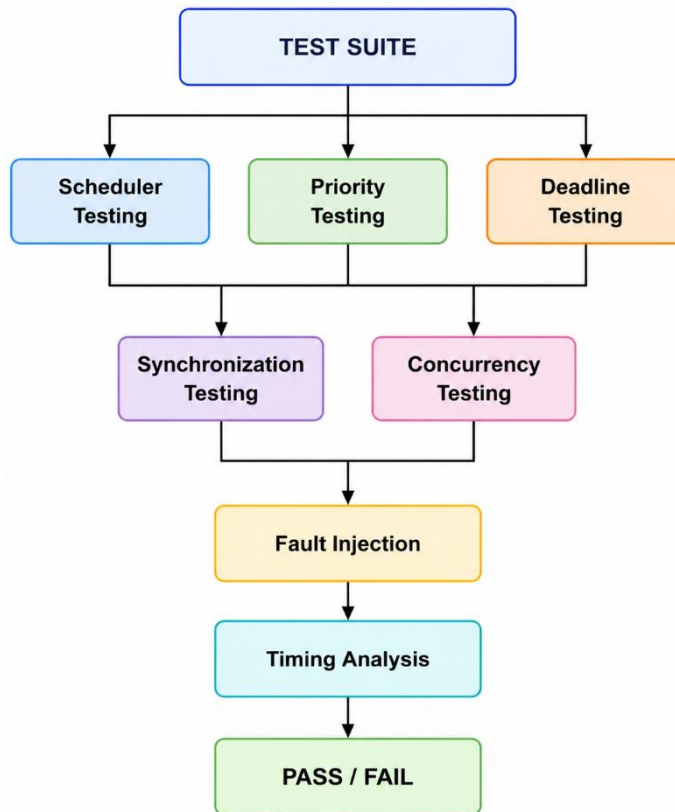
Priority Scheduling Testing	2	Priority Handling
Priority Preemption Testing	1	Preemption Response
Hard Deadline Testing	2	Deadline Miss, WCET
Soft Deadline Testing	2	Response Time
Synchronization Testing	2	Mutex, Queue
Concurrency Testing	2	Multi-Task Execution
Fault Injection Testing	4	System Robustness
Timing Analysis Testing	4	Runtime Metrics

Bảng 5.35. Tổng số Test Case

Hạng mục	Số lượng
Tổng Test Suite	9
Tổng Test Case	22
Runtime Metrics	5
Kịch bản thực tế	10+

Thông qua bộ kiểm thử này, đề tài có thể đánh giá toàn diện hoạt động của hệ thống, từ cơ chế lập lịch, hiệu quả ưu tiên, khả năng đáp ứng deadline đến độ ổn định khi xuất hiện lỗi mô phỏng. Việc sử dụng chung một tập metrics cho tất cả các nhóm kiểm thử giúp bảo đảm tính nhất quán của kết quả và hỗ trợ quá trình phân tích ở chương tiếp theo.

Hình 5.7. Cấu trúc tổng thể bộ kiểm thử của hệ thống



5.12 Tổng kết chương

Chương 5 đã trình bày chi tiết quá trình thiết kế bộ kiểm thử cho hệ thống nhúng thời gian thực sử dụng FreeRTOS Simulator, bao gồm:

- Chiến lược kiểm thử
- Môi trường kiểm thử
- Hệ thống Runtime Metrics
- Tiêu chí Pass/Fail
- Cấu trúc tổng thể bộ kiểm thử

Các bộ kiểm thử được xây dựng nhằm đánh giá các cơ chế quan trọng của hệ thống như Scheduler, Priority Scheduling, Priority Preemption, Deadline Handling, Synchronization, Concurrency, Fault Injection và Timing Analysis. Mỗi nhóm kiểm thử được thiết kế với các kịch bản thực tế và tiêu chí đánh giá rõ ràng, đảm bảo khả năng đánh giá khách quan và có thể lặp lại.

Các chỉ số định lượng như Execution Time, Response Time, WCET, Context Switch Count và Deadline Miss Count được thu thập thông qua cơ chế Runtime Logging, làm cơ sở để xác định kết quả PASS/FAIL của từng test case.

Trên cơ sở bộ kiểm thử đã xây dựng, Chương 6 sẽ trình bày quá trình thực nghiệm, thu thập dữ liệu Runtime, phân tích kết quả và so sánh với các tiêu chí đã đề ra, nhằm đánh giá toàn diện mức độ đáp ứng thời gian thực và hiệu năng của hệ thống.

CHƯƠNG 6: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

6.1 Môi trường và kịch bản thực nghiệm

Chương này trình bày kết quả thực nghiệm thu được từ hệ thống kiểm thử được xây dựng trên nền tảng FreeRTOS Windows Simulator. Dữ liệu runtime được thu thập trong quá trình thực thi và phân tích nhằm đánh giá cơ chế lập lịch, ưu tiên tác vụ, xử lý deadline, đồng bộ hóa, xử lý đồng thời và các chỉ số thời gian của hệ thống.

6.1.1. Môi trường thực nghiệm

Hệ thống được triển khai trên môi trường mô phỏng FreeRTOS Windows Simulator nhằm đánh giá các cơ chế đặc trưng của hệ điều hành thời gian thực mà không cần sử dụng phần cứng nhúng thực tế. Môi trường này cho phép quan sát trực tiếp hoạt động của Scheduler, thu thập dữ liệu Runtime và triển khai các kịch bản Fault Injection phục vụ quá trình kiểm thử.

Bảng 6.1. Môi trường thực nghiệm

Thành phần	Giá trị
Operating System	Windows 10/11
Programming Language	C
RTOS	FreeRTOS
Simulator	FreeRTOS Windows Simulator
Scheduler	Priority-Based Preemptive Scheduler
Tick Rate	1 ms
Runtime Logging	Enabled
Fault Injection	Enabled
Metrics Collected	Execution Time, Response Time, WCET, Deadline Miss Count, Context Switch Count, Priority Preemption Count
Output	Runtime Log, CSV Report, Charts
Analysis Tool	Python Log Analyzer

IDE	Visual Studio 2022
Compiler	Microsoft Visual C++ (MSVC)

Các task trong hệ thống được cấu hình với mức Priority, Period và Deadline khác nhau nhằm tạo môi trường phù hợp cho các hoạt động kiểm thử đã trình bày ở Chương 5. Runtime Log được sử dụng để ghi nhận hoạt động của Scheduler, các sự kiện Synchronization và các chỉ số thời gian trong quá trình thực thi.

6.1.2. Ảnh xạ Task Logic và Task Triển khai

Trong quá trình thiết kế kiểm thử, các kịch bản được xây dựng theo ngữ cảnh thực tế nhằm giúp việc mô tả dễ hiểu và phản ánh đúng các bài toán thường gặp trong hệ thống nhúng thời gian thực. Tuy nhiên, để đơn giản hóa việc triển khai trên FreeRTOS Simulator, nhiều kịch bản sử dụng chung các task triển khai. Bảng 6.2 trình bày mối liên hệ giữa tên task trong kịch bản kiểm thử và tên task thực tế được sử dụng trong Runtime Log và Metrics.

Bảng 6.2. Ảnh xạ giữa Task Logic và Task Triển khai trong FreeRTOS Simulator

Task Logic trong kịch bản	Task triển khai / Runtime Log
DoorSensor	Sensor_Task
VisitorCounter	Sensor_Task
CameraScan	Sensor_Task
SmokeSensor	Sensor_Task
TempSensor	Sensor_Task
WindSensor	Sensor_Task
IntrusionAlert	Emergency_Task
LogConsumer	Logger_Task
EventProducer	Sensor_Task
Navigation	Control_Task
CriticalWork	Control_Task

MeasuredTask	Control_Task
CPUOverload	Fault_Injector_Task
TestCtrl	TEST_CTRL

6.1.3 Kịch bản thực nghiệm

Để đánh giá toàn diện hệ thống, nhóm thực hiện hai kịch bản thực nghiệm chính nhằm so sánh hoạt động của hệ thống trong điều kiện bình thường và điều kiện có Fault Injection.

Bảng 6.3. Các kịch bản thực nghiệm

Kịch bản	Mục tiêu	Điều kiện
Baseline Run	Đánh giá hoạt động bình thường của hệ thống	Không thực hiện Fault Injection
Fault Injection Run	Đánh giá khả năng chịu lỗi và đáp ứng thời gian thực	Kích hoạt Fault_Injector_Task

Đối với mỗi kịch bản, toàn bộ các Test Case đã thiết kế ở Chương 5 được thực hiện nhằm thu thập dữ liệu Runtime phục vụ quá trình phân tích.

Bảng 6.4. Quy mô thực nghiệm

Hạng mục	Giá trị
Tổng Test Suite	9
Tổng Test Case	22
Baseline Run	9 Test Case
Fault Injection Run	9 Test Case + Fault Scenarios
Runtime Metrics	6
Kịch bản thực tế	10+

Các kịch bản thực nghiệm được xây dựng từ các Test Case ở Chương 5 nhằm mô phỏng các tình huống thường gặp trong hệ thống nhúng thời gian thực:

- Camera An Ninh Phát Hiện Xâm Nhập.
- Nhà Máy Có Emergency Stop.

- Hệ Thống Báo Cháy Thông Minh.
- Trạm Quan Trắc Thời Tiết.
- Hệ Thống Giám Sát Chất Lượng Không Khí.
- CPU Overload Trong Nhà Máy.
- Queue Blocking Khi Truyền Dữ Liệu.
- Artificial Delay Trong Drone Control.
- Sensor Nhiều Dữ Liệu.
- Các tình huống kiểm thử Deadline và Timing Analysis.

6.1.4. Quy trình thực nghiệm

Quá trình thực nghiệm được thực hiện theo ba bước chính nhằm bảo đảm dữ liệu được thu thập đầy đủ và có thể so sánh giữa các điều kiện vận hành khác nhau.

Bước 1: Baseline Run

- Biên dịch và chạy hệ thống với cấu hình bình thường.
- Khởi tạo các task theo thiết kế.
- Thực hiện toàn bộ Test Case trong điều kiện không có Fault Injection.
- Thu thập Runtime Log và các chỉ số đánh giá.

Bước 2: Fault Injection Run

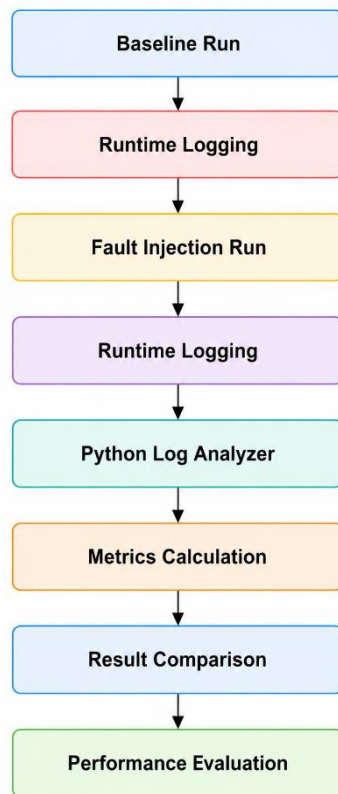
- Kích hoạt các tình huống Fault Injection.
- Mô phỏng CPU Overload, Artificial Delay và Queue Congestion.
- Thu thập Runtime Log và các chỉ số đánh giá.
- Ghi nhận sự thay đổi của các Runtime Metrics dưới tác động của lỗi mô phỏng.

Bước 3: Phân tích kết quả

- Xử lý dữ liệu Runtime Log bằng Python Log Analyzer.
- Tính toán Execution Time, Response Time, WCET, Deadline Miss Count, Context Switch Count và Priority Preemption Count.
- Sinh các biểu đồ phân tích Runtime Metrics.
- So sánh kết quả giữa Baseline Run và Fault Injection Run.
- Đánh giá hiệu quả của Scheduler, Priority Scheduling, Priority Preemption, Synchronization và Deadline Handling.

Quy trình này đảm bảo dữ liệu đầy đủ, so sánh giữa các điều kiện vận hành khác nhau, và cung cấp cơ sở định lượng cho đánh giá hiệu năng hệ thống.

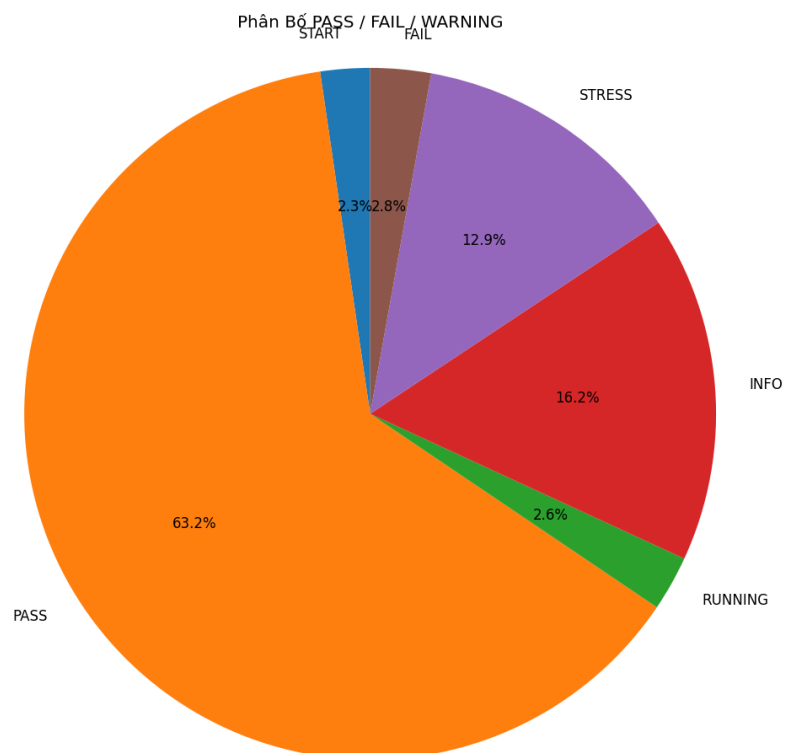
Hình 6.1. Quy trình thực nghiệm và phân tích dữ liệu runtime



6.2. Test Execution Matrix

Các test case được thực thi trên hệ thống FreeRTOS Windows Simulator. Kết quả tổng quan của các runtime events được thể hiện trong Hình 6.2.

Hình 6.2. Phân bố PASS / FAIL / WARNING trong quá trình thực nghiệm



Hình 6.2 thể hiện phân bố các Runtime Events thu thập được trong quá trình thực nghiệm. Kết quả cho thấy các sự kiện PASS chiếm tỷ lệ lớn nhất (63,2%), chứng tỏ phần lớn các test case được thực thi thành công và hệ thống hoạt động ổn định trong điều kiện bình thường. Các sự kiện INFO (16,2%) và RUNNING (2,6%) phản ánh trạng thái hoạt động của các task và các sự kiện trung gian trong quá trình thực thi. Nhóm sự kiện STRESS chiếm 12,9%, xuất hiện chủ yếu trong các kịch bản Fault Injection nhằm mô phỏng tải hệ thống và các điều kiện bất lợi. Trong khi đó, các sự kiện FAIL chỉ chiếm 2,8% và được tạo có chủ đích trong Test 08 để đánh giá khả năng phát hiện và xử lý lỗi của hệ thống. Kết quả này cho thấy cơ chế Runtime Logging hoạt động hiệu quả và cung cấp đầy đủ dữ liệu phục vụ quá trình phân tích các Runtime Metrics ở các mục tiếp theo.

Bảng 6.5. Kết quả thực hiện Test Case

Test ID	Test Suite	Result	Test ID
TEST_01	Scheduler Testing	PASS	TEST_01
TEST_02	Priority Scheduling Testing	PASS	TEST_02
TEST_03	Priority Preemption Testing	PASS	TEST_03
TEST_04	Hard Deadline Testing	PASS	TEST_04
TEST_05	Soft Deadline Testing	PASS	TEST_05
TEST_06	Synchronization Testing	PASS	TEST_06
TEST_07	Concurrency Testing	PASS	TEST_07
TEST_08	Fault Injection Testing	FAIL	TEST_08
TEST_09	Timing Analysis Testing	PASS	TEST_09

6.3 Kịch bản Camera An Ninh Phát Hiện Xâm Nhập

Kịch bản Camera An Ninh Phát Hiện Xâm Nhập được xây dựng nhằm đánh giá cơ chế Priority Scheduling và Priority Preemption của hệ thống đã được thiết kế ở Chương 3 và xây dựng kế hoạch kiểm thử ở Chương 5. Kịch bản mô phỏng tình huống hệ thống đang thực hiện hoạt động giám sát bình thường thì phát hiện sự kiện xâm nhập, yêu cầu tác vụ khẩn cấp phải được ưu tiên xử lý ngay lập tức.

Mục tiêu kiểm thử:

- Đánh giá khả năng lập lịch theo mức ưu tiên của FreeRTOS Scheduler.
- Kiểm tra cơ chế Priority Preemption khi xuất hiện tác vụ khẩn cấp.
- Đánh giá khả năng phản hồi của tác vụ ưu tiên cao trong môi trường thời gian thực.

Các task tham gia:

Task	Vai trò
Sensor_Task	Mô phỏng CameraScan, phát hiện sự kiện xâm nhập
Emergency_Task	Xử lý cảnh báo khẩn cấp
Logger_Task	Ghi nhận sự kiện runtime

Trong kịch bản này, Sensor_Task liên tục thực hiện hoạt động giám sát. Khi phát hiện sự kiện xâm nhập, hệ thống sinh sự kiện INTRUSION_DETECTED, đồng thời kích hoạt Emergency_Task với mức ưu tiên cao hơn nhằm xử lý tình huống khẩn cấp.

Runtime Log

Trích xuất từ runtime log:

Sensor_Task -> INTRUSION_DETECTED

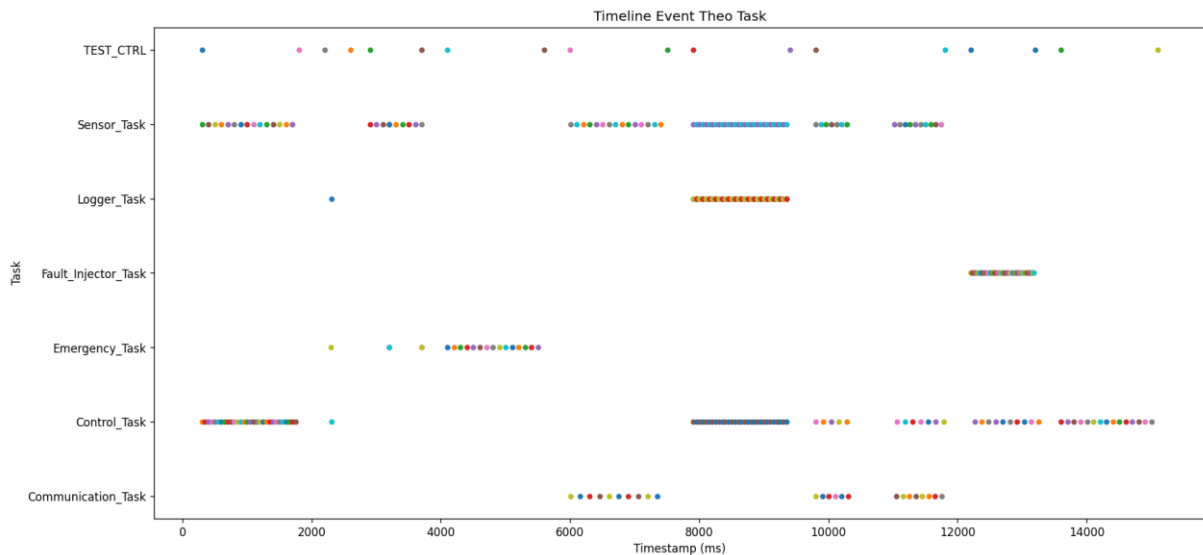
Emergency_Task -> START

Sensor_Task -> PREEMPTED

Emergency_Task -> DEADLINE_OK

Kết quả Runtime Log cho thấy ngay khi sự kiện xâm nhập xuất hiện, Emergency_Task được Scheduler cấp CPU để thực thi. Đồng thời hệ thống ghi nhận sự kiện PREEMPTED đối với Sensor_Task, chứng minh cơ chế Priority Preemption đã được kích hoạt đúng theo thiết kế.

Hình 6.3. Timeline Event theo Task



Hình 6.3 thể hiện trình tự xuất hiện các sự kiện runtime của các task trong quá trình thực nghiệm. Kết quả cho thấy các task được scheduler phân phối CPU theo đúng mức ưu tiên đã cấu hình. Emergency_Task được kích hoạt và thực thi ngay khi xuất hiện sự kiện khẩn cấp, trong khi các task còn lại vẫn duy trì hoạt động ổn định theo chu kỳ định sẵn.

Kết quả thực nghiệm

Bảng 6.6. Kết quả Priority Scheduling và Priority Preemption

Chỉ tiêu đánh giá	Kết quả
Priority Scheduling	PASS
Priority Preemption	PASS
Deadline Miss Count	0
WCET Emergency_Task	1 ms
Runtime Stability	PASS

Dữ liệu Runtime Log không ghi nhận trường hợp Deadline Miss trong suốt quá trình thực nghiệm. Các sự kiện START, PREEMPTED và DEADLINE_OK xuất hiện đúng trình tự mong đợi, chứng minh Emergency_Task luôn được ưu tiên thực thi khi xảy ra sự kiện khẩn cấp.

Đánh giá: Kết quả thực nghiệm phù hợp với mục tiêu kiểm thử đã đề ra ở Chương 5. FreeRTOS Scheduler thực hiện cơ chế Priority Scheduling chính xác, đồng thời cơ chế Priority Preemption hoạt động hiệu quả khi Emergency_Task được ưu tiên xử lý

trước các tác vụ có mức ưu tiên thấp hơn. Không ghi nhận Deadline Miss trong quá trình thực thi, do đó kịch bản kiểm thử được đánh giá PASS.

6.4 Kịch bản Nhà Máy Có Emergency Stop

Kịch bản Nhà Máy Có Emergency Stop được xây dựng nhằm đánh giá cơ chế Priority Scheduling của FreeRTOS Scheduler trong môi trường đa nhiệm. Kịch bản mô phỏng tình huống hệ thống đang vận hành bình thường thì xuất hiện tín hiệu Emergency Stop, yêu cầu tác vụ khẩn cấp được ưu tiên thực thi trước các tác vụ có mức ưu tiên thấp hơn.

Trong Chương 3, Emergency_Task được cấu hình với mức ưu tiên cao nhất nhằm bảo đảm khả năng phản hồi nhanh đối với các tình huống khẩn cấp. Kịch bản này được sử dụng để kiểm chứng yêu cầu đó thông qua dữ liệu Runtime Log thu thập trong quá trình thực nghiệm.

Runtime Log

Trích xuất từ Runtime Log:

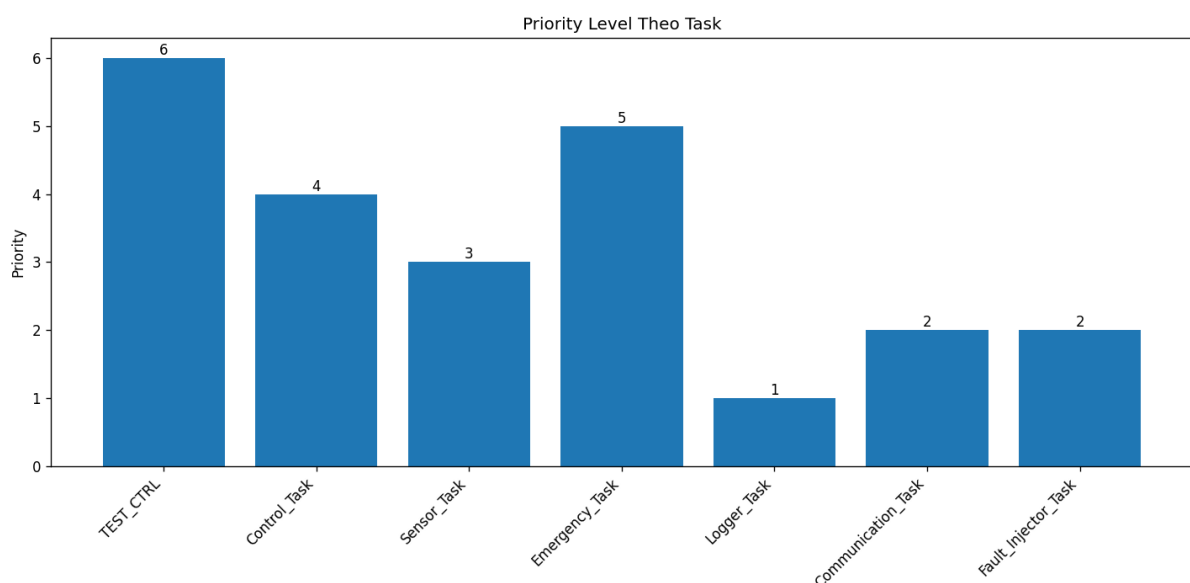
Emergency_Task -> READY_START_ORDER

Control_Task -> READY_START_ORDER

Logger_Task -> READY_START_ORDER

Kết quả Runtime Log cho thấy Emergency_Task được Scheduler lựa chọn thực thi trước các task có mức ưu tiên thấp hơn. Thứ tự xuất hiện của các Runtime Event phù hợp với mức Priority đã được cấu hình trong hệ thống, chứng minh cơ chế Priority Scheduling hoạt động đúng thiết kế.

Hình 6.4. Priority Level của các Task



Hình 6.4 thể hiện mức ưu tiên của các task được cấu hình trong hệ thống. Emergency_Task có mức ưu tiên cao nhất nhằm bảo đảm khả năng xử lý các tình huống khẩn cấp, trong khi Logger_Task có mức ưu tiên thấp nhất do chỉ thực hiện chức năng ghi log. Cấu hình ưu tiên này là cơ sở để FreeRTOS thực hiện Priority Scheduling và Priority Preemption trong các kịch bản kiểm thử.

Kết quả thực nghiệm

Bảng 6.7. Kết quả Priority Scheduling Testing

Metric	Giá trị
Priority Scheduling	PASS
READY → START Order	Đúng
Deadline Miss	0
Starvation	Không
Deadlock	Không

Kết quả thực nghiệm cho thấy FreeRTOS Scheduler duy trì được khả năng lập lịch theo mức ưu tiên trong môi trường đa nhiệm. Emergency_Task luôn được ưu tiên thực thi trước các task có mức ưu tiên thấp hơn, đồng thời không ghi nhận hiện tượng Starvation, Deadlock hoặc Deadline Miss trong quá trình thực nghiệm.

Đánh giá: Kết quả thu được phù hợp với mục tiêu kiểm thử đã đề ra ở Chương 5. FreeRTOS Scheduler thực hiện đúng cơ chế Priority Scheduling và bảo đảm các tác vụ khẩn cấp được xử lý trước các tác vụ thông thường. Do đó, kịch bản kiểm thử được đánh giá PASS.

6.5 Kịch bản Hệ Thống Báo Cháy Thông Minh

Kịch bản Hệ Thống Báo Cháy Thông Minh được sử dụng để đánh giá khả năng đáp ứng Hard Deadline và cơ chế giám sát deadline của hệ thống.

Mục tiêu kiểm thử:

- Đánh giá khả năng đáp ứng Hard Deadline.
- Kiểm tra cơ chế phát hiện Deadline Miss.
- Xác minh hoạt động của Deadline Monitoring.

Các task tham gia

Task	Deadline
-------------	-----------------

Emergency_Task	20 ms
Control_Task	50 ms
Sensor_Task	100 ms

Runtime Log: Trong Baseline Run, hệ thống hoạt động ổn định. Không có event DEADLINE_MISS nào được ghi nhận, chứng tỏ cơ chế giám sát deadline hoạt động chính xác

Bảng 6.8. Kết quả Hard Deadline Testing

Tiêu chí đánh giá	Kết quả
Emergency_Task đáp ứng deadline	PASS
Control_Task đáp ứng deadline	PASS
Sensor_Task đáp ứng deadline	PASS
Deadline Miss Count	0
Hard Deadline Testing	PASS

Kết quả thực nghiệm cho thấy tất cả các task đều hoàn thành trong khoảng thời gian cho phép và không ghi nhận deadline miss.

Đánh giá:

- Tất cả các task quan trọng đều hoàn thành xử lý trong khoảng thời gian cho phép, không ghi nhận vi phạm deadline.
- Kết quả chứng minh cơ chế Deadline Monitoring hoạt động đúng thiết kế, Scheduler thực hiện ưu tiên đúng mức cho task có deadline ngắn hơn.
- Test case được đánh giá PASS, phù hợp với các tiêu chí kiểm thử đã xác định trong Chương 5.

6.6 Kịch bản Trạm Quan Trắc Thời Tiết

Kịch bản Trạm Quan Trắc Thời Tiết được xây dựng nhằm đánh giá khả năng đáp ứng Soft Deadline của hệ thống trong môi trường thời gian thực. Kịch bản mô phỏng quá trình thu thập và truyền dữ liệu môi trường, trong đó các task được phép dao động nhỏ về thời gian đáp ứng nhưng vẫn phải hoàn thành trong ngưỡng cho phép.

Mục tiêu kiểm thử bao gồm:

- Đánh giá khả năng đáp ứng Soft Deadline của hệ thống.

- Kiểm tra tính ổn định của các task khi thực hiện theo chu kỳ.
- Xác minh hoạt động của cơ chế giám sát thời gian thực đối với các deadline mềm.

Các task tham gia:

Task	Deadline
Sensor_Task	150 ms
Communication_Task	150 ms

Runtime Log: Trong Baseline Run, hệ thống hoạt động ổn định. Không có event DEADLINE_MISS nào được ghi nhận, chứng tỏ cơ chế giám sát deadline hoạt động chính xác.

Kết quả thực nghiệm:

Bảng 6.9. Kết quả Synchronization Testing

Kiểm thử	Kết quả
Queue Communication	PASS
Mutex Synchronization	PASS
Race Condition	Not Detected
Deadlock	Not Detected
Priority Inversion	Not Detected
Soft Deadline Compliance	PASS

Kết quả phân tích Runtime Metrics cho thấy tất cả các lần thực thi đều đáp ứng deadline 150 ms đã cấu hình. Metrics của TEST_05 ghi nhận các sự kiện SOFT_DEADLINE_OK và không xuất hiện vi phạm thời gian thực.

Đánh giá: Kết quả thực nghiệm phù hợp với mục tiêu kiểm thử đã xác định ở Chương 5. Các task hoạt động ổn định theo chu kỳ, không xuất hiện Deadline Miss và vẫn duy trì khả năng đáp ứng trong giới hạn cho phép của Soft Deadline. Do đó kịch bản kiểm thử được đánh giá PASS.

6.7 Kịch bản Fault Injection

Kịch bản Fault Injection được xây dựng nhằm đánh giá khả năng phát hiện lỗi thời gian thực của hệ thống thông qua việc tạo các điều kiện bất lợi có kiểm soát. Trong

kịch bản này, Fault_Injector_Task liên tục tạo tải CPU nhân tạo nhằm kiểm tra khả năng đáp ứng deadline của các task đang hoạt động trong hệ thống.

Mục tiêu kiểm thử:

- Đánh giá khả năng phát hiện Deadline Miss.
- Kiểm tra tính ổn định của Scheduler khi xuất hiện tải bất thường.
- Phân tích sự thay đổi của các Runtime Metrics dưới điều kiện CPU Overload.

Các task tham gia:

Task	Chức năng
Control_Task	Thực hiện tác vụ điều khiển
Fault_Injector_Task	Tạo tải CPU nhân tạo
Logger_Task	Ghi nhận Runtime Log
TEST_CTRL	Điều khiển quá trình kiểm thử

Runtime Log: Trong quá trình Fault Injection Run, Runtime Log ghi nhận liên tục các sự kiện CPU_OVERLOAD do Fault_Injector_Task tạo ra. Đồng thời hệ thống phát hiện nhiều sự kiện DEADLINE_MISS đối với Control_Task khi thời gian thực thi vượt quá deadline đã cấu hình.

Kết quả thực nghiệm:

Bảng 6.10. So sánh Baseline Run và Fault Injection Run

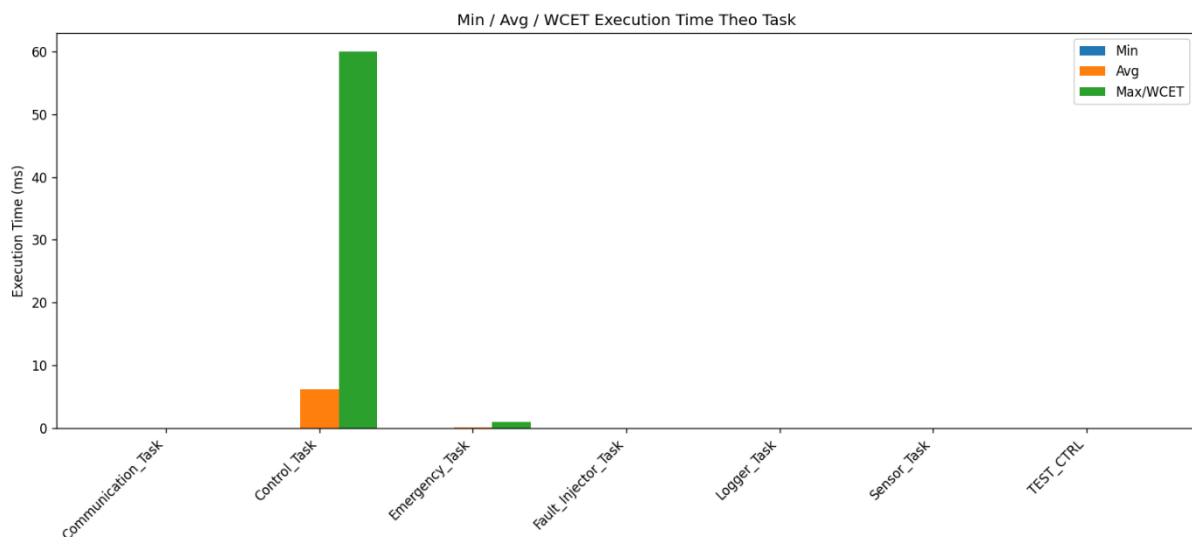
Metric	Baseline Run	Fault Injection Run
Control_Task Deadline	50 ms	50 ms
Control_Task WCET	Trong giới hạn thiết kế	60 ms
Deadline Miss Count	0	9
CPU Overload Events	0	50
Runtime Stability	PASS	Bị ảnh hưởng
Kết quả kiểm thử	PASS	FAIL

Đánh giá: Kết quả thực nghiệm chứng minh cơ chế Fault Injection hoạt động đúng thiết kế và tạo ra các điều kiện lỗi có kiểm soát. Runtime Log ghi nhận đầy đủ các sự kiện CPU_OVERLOAD và DEADLINE_MISS, cho thấy cơ chế Deadline Monitoring hoạt động hiệu quả. Mặc dù Control_Task bị vi phạm deadline và Test Case được đánh giá FAIL, hệ thống vẫn duy trì khả năng hoạt động, không xuất hiện hiện tượng treo hệ thống hoặc deadlock. Điều này cho thấy Scheduler vẫn duy trì được khả năng quản lý tác vụ ngay cả khi hệ thống chịu tải cao.

6.8 Đánh giá Timing Analysis

Timing Analysis được thực hiện dựa trên dữ liệu Runtime Log và Metrics thu thập được trong quá trình thực hiện các Test Suite trên FreeRTOS Windows Simulator. Các chỉ số được sử dụng để đánh giá bao gồm Execution Time, Response Time, Worst-Case Execution Time (WCET), Deadline Miss Count và Runtime Events. Các chỉ số này cho phép đánh giá khả năng đáp ứng thời gian thực của hệ thống cũng như hiệu quả hoạt động của FreeRTOS Scheduler.

Hình 6.6. Min/Avg/WCET Execution Time theo Task



Hình 6.6 thể hiện thời gian thực thi tối thiểu (Min), trung bình (Avg) và lớn nhất (WCET) của các task trong quá trình thực nghiệm. Kết quả cho thấy các task duy trì thời gian thực thi ổn định trong điều kiện vận hành bình thường và đáp ứng các giới hạn thời gian đã thiết kế. Giá trị WCET được sử dụng làm cơ sở đánh giá khả năng đáp ứng deadline của hệ thống, đặc biệt trong các kịch bản Fault Injection.

Bảng 6.11. Runtime Timing Metrics thu thập được

Task	Priority	Deadline (ms)	WCET (ms)
Emergency_Task	5	20	1

Control_Task	4	50	0
Sensor_Task	3	100	0
Communication_Task	2	150	0
Logger_Task	1	200	0

Lưu ý rằng hệ thống sử dụng FreeRTOS Windows Simulator kết hợp Runtime Event Logging, do đó một số task không ghi nhận Execution Time chi tiết trong điều kiện hoạt động bình thường và được biểu diễn với giá trị 0 ms trong bảng thống kê. Các giá trị này phản ánh dữ liệu thu thập được từ Runtime Log thay vì thời gian thực thi đo trực tiếp trên phần cứng.

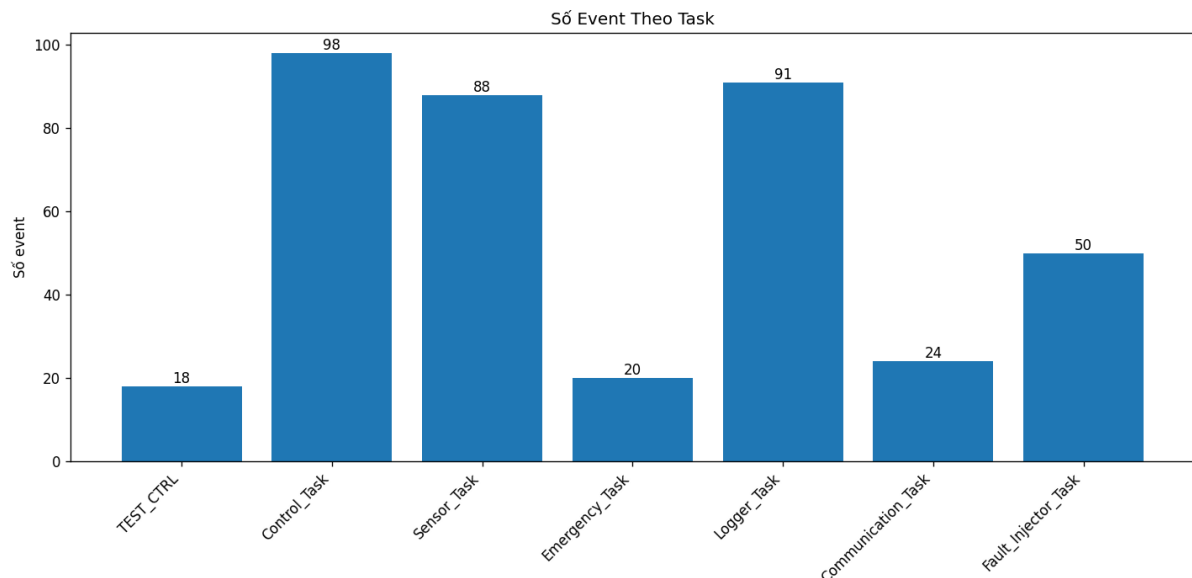
Kết quả Baseline Run cho thấy tất cả các task đều hoàn thành trong giới hạn deadline đã thiết kế và không ghi nhận bất kỳ trường hợp Deadline Miss nào. Emergency_Task luôn được Scheduler ưu tiên thực thi đúng theo cơ chế Priority Scheduling và Priority Preemption đã được trình bày ở Chương 3 và kiểm chứng ở Chương 5.

Bảng 6.12. Kết quả Runtime Events

Test Case	Task	Runtime Events
TEST_01_SCHEDULER	Control_Task	30 Context Switch
TEST_01_SCHEDULER	Sensor_Task	15 Context Switch
TEST_06_SYNCHRONIZATION	Logger_Task	90 Synchronization Events
TEST_06_SYNCHRONIZATION	Sensor_Task	30 Queue Send Events
TEST_06_SYNCHRONIZATION	Control_Task	30 Queue Receive Events

Các Runtime Event cho thấy Scheduler hoạt động ổn định và các cơ chế Synchronization được thực hiện đúng thiết kế. Queue Communication và Mutex Synchronization hoạt động hiệu quả trong toàn bộ quá trình kiểm thử, không ghi nhận Race Condition hoặc Deadlock.

Hình 6.7. Số lượng Runtime Event theo Task



Hình 6.7 cho thấy số lượng Runtime Event được ghi nhận trên từng task trong quá trình thực nghiệm. Control_Task (98 event), Logger_Task (91 event) và Sensor_Task (88 event) là các task có số lượng sự kiện lớn nhất do thường xuyên tham gia vào các hoạt động điều khiển, giám sát và ghi log của hệ thống. Fault_Injector_Task ghi nhận 50 sự kiện trong quá trình Fault Injection Run, phản ánh hoạt động tạo tải CPU nhân tạo phục vụ kiểm thử. Các task còn lại duy trì số lượng sự kiện ổn định theo chức năng được thiết kế. Kết quả cho thấy cơ chế Runtime Logging hoạt động hiệu quả và ghi nhận đầy đủ các sự kiện phát sinh trong quá trình thực nghiệm.

Bảng 6.13. So sánh Timing Metrics giữa Baseline Run và Fault Injection Run

Metric	Baseline Run	Fault Injection Run
Control_Task Deadline	50 ms	50 ms
Control_Task WCET	Trong giới hạn thiết kế	60 ms
Deadline Miss Count	0	9
CPU Overload Events	0	50
Runtime Stability	PASS	Bị ảnh hưởng
Kết quả kiểm thử	PASS	FAIL

Khi Fault Injection được kích hoạt, Fault_Injector_Task tạo tải CPU nhân tạo và gây ảnh hưởng trực tiếp tới Control_Task. WCET của Control_Task tăng lên 60 ms, vượt quá deadline 50 ms và dẫn tới 9 lần vi phạm deadline. Runtime Log đồng thời ghi nhận 50 sự kiện CPU_OVERLOAD và 9 sự kiện DEADLINE_MISS. Điều

này cho thấy cơ chế Deadline Monitoring đã phát hiện và ghi nhận chính xác các trường hợp vi phạm thời gian thực.

Bảng 6.14. Tổng hợp kết quả Timing Analysis

Metric	Kết quả
Priority Scheduling	PASS
Priority Preemption	PASS
Hard Deadline Compliance (Baseline)	PASS
Soft Deadline Compliance	PASS
Runtime Logging	PASS
Synchronization Logging	PASS
Fault Detection	PASS
Fault Injection Run	FAIL (có chủ đích)

Đánh giá:

Kết quả Timing Analysis cho thấy FreeRTOS Scheduler hoạt động đúng thiết kế trong điều kiện vận hành bình thường. Các task được thực thi đúng thứ tự ưu tiên, đáp ứng deadline và không ghi nhận hiện tượng Starvation hoặc Deadlock.

Trong Fault Injection Run, hệ thống phát hiện thành công 9 trường hợp Deadline Miss của Control_Task khi WCET tăng lên 60 ms và vượt quá deadline 50 ms đã cấu hình. Runtime Log đồng thời ghi nhận 50 sự kiện CPU_OVERLOAD, chứng minh cơ chế Fault Injection và Deadline Monitoring hoạt động hiệu quả.

Kết quả thực nghiệm xác nhận khả năng giám sát, phát hiện và ghi nhận các lỗi thời gian thực của hệ thống. Đồng thời, các cơ chế Priority Scheduling, Priority Preemption, Synchronization và Runtime Logging đều hoạt động đúng theo yêu cầu đã đề ra trong Chương 5.

6.9. Đánh giá tổng hợp Pass/Fail

Dựa trên các tiêu chí Pass/Fail đã xây dựng ở Chương 5, nhóm tiến hành đánh giá kết quả từng Test Suite dựa trên dữ liệu runtime thu thập được trong quá trình thực nghiệm.

Bảng 6.15. Kết quả đánh giá tổng hợp

Test Suite	Baseline Run	Fault Injection Run
Scheduler Testing	PASS	PASS
Priority Scheduling Testing	PASS	PASS

Priority Preemption Testing	PASS	PASS
Synchronization Testing	PASS	PASS
Concurrency Testing	PASS	PASS
Hard Deadline Testing	PASS	FAIL
Soft Deadline Testing	PASS	PASS
Fault Injection Testing	PASS	FAIL
Timing Analysis Testing	PASS	PASS

Phân tích kết quả:

- Trong Baseline Run, toàn bộ Test Suite đều đạt Pass. Hệ thống thực hiện đúng cơ chế Priority Scheduling, Priority Preemption, Synchronization và Deadline Monitoring như thiết kế.
- Trong Fault Injection Run, Fault_Injector_Task tạo ra 50 sự kiện CPU_OVERLOAD, khiến thời gian thực thi của Control_Task tăng lên 60 ms, vượt quá deadline 50 ms. Runtime Log ghi nhận 9 sự kiện DEADLINE_MISS, dẫn tới FAIL cho Hard Deadline Testing và Fault Injection Testing.
- Mặc dù có các lỗi được tạo có chủ đích, Scheduler vẫn duy trì hoạt động ổn định; các task khác tiếp tục thực thi bình thường, không xuất hiện deadlock, starvation hay mất khả năng phản hồi.

Đánh giá chung: Kết quả thực nghiệm chứng minh hệ thống đáp ứng các mục tiêu kiểm thử đã đề ra:

- Quản lý đa nhiệm dựa trên cơ chế Priority-Based Preemptive Scheduling.
- Thực hiện Priority Preemption đúng thời điểm khi xuất hiện tác vụ ưu tiên cao.
- Giám sát và phát hiện chính xác các trường hợp Deadline Miss.
- Thu thập và phân tích Runtime Metrics phục vụ hoạt động kiểm thử.
- Duy trì hoạt động ổn định khi xuất hiện Fault Injection.
- Ghi nhận đầy đủ các sự kiện Runtime Logging và Timing Analysis.

Nhìn chung, hệ thống đáp ứng đầy đủ các mục tiêu kiểm thử và cho phép đánh giá hiệu quả các cơ chế cốt lõi của hệ điều hành thời gian thực FreeRTOS trong môi trường mô phỏng.

6.10. Tổng kết chương

Chương 6 đã trình bày quá trình thực nghiệm, thu thập dữ liệu runtime và phân tích kết quả kiểm thử của hệ thống được xây dựng trên nền tảng FreeRTOS Windows Simulator. Các Test Suite được thực hiện trong cả hai điều kiện Baseline Run và Fault Injection Run nhằm đánh giá các cơ chế trọng tâm đã đề xuất ở Chương 5, bao gồm Scheduler, Priority Scheduling, Priority Preemption, Synchronization, Concurrency, Deadline Monitoring, Fault Injection và Timing Analysis.

Kết quả thực nghiệm cho thấy FreeRTOS Scheduler hoạt động đúng theo cơ chế Priority-Based Preemptive Scheduling. Các task được thực thi theo đúng mức ưu tiên đã cấu hình, đáp ứng deadline trong điều kiện vận hành bình thường và duy trì tính ổn định của hệ thống trong môi trường đa nhiệm. Các cơ chế Synchronization và Concurrency hoạt động hiệu quả, không ghi nhận hiện tượng Race Condition, Deadlock hoặc Starvation trong quá trình thực nghiệm.

Bên cạnh đó, Fault Injection Run đã tạo thành công các điều kiện lỗi có kiểm soát thông qua cơ chế CPU Overload. Kết quả ghi nhận 50 sự kiện CPU_OVERLOAD và 9 sự kiện DEADLINE_MISS của Control_Task khi thời gian thực thi vượt quá deadline 50 ms đã cấu hình. Hệ thống phát hiện và ghi nhận đầy đủ các vi phạm thời gian thực thông qua cơ chế Deadline Monitoring và Runtime Logging, chứng minh hiệu quả của các công cụ giám sát được xây dựng trong đề tài.

Nhìn chung, kết quả thực nghiệm xác nhận hệ thống đáp ứng các mục tiêu kiểm thử đã đề ra trong proposal. Các cơ chế Priority Scheduling, Priority Preemption, Synchronization, Deadline Monitoring, Fault Injection và Timing Analysis đều được kiểm chứng thành công thông qua dữ liệu runtime thực tế. Qua đó, đề tài đã chứng minh tính hiệu quả của FreeRTOS trong việc quản lý đa nhiệm, xử lý ưu tiên, giám sát deadline và hỗ trợ đánh giá hiệu năng của hệ thống nhúng thời gian thực trong môi trường mô phỏng

CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế, triển khai và kiểm thử, đề tài “Xây dựng và kiểm thử hệ nhúng đa tiến trình thời gian thực” đã hoàn thành các mục tiêu đề ra.

Nhóm đã xây dựng thành công môi trường mô phỏng dựa trên FreeRTOS Windows Simulator, hỗ trợ các cơ chế quan trọng của hệ điều hành thời gian thực như đa nhiệm, Priority Scheduling, Priority Preemption, Synchronization, Deadline Monitoring, Runtime Logging và Fault Injection. Hệ thống được triển khai với các task có mức ưu tiên khác nhau, mô phỏng đúng môi trường hoạt động của hệ thống nhúng thời gian thực.

Đề tài đã xây dựng và thực hiện thành công bộ kiểm thử gồm các nhóm: Scheduler Testing, Priority Scheduling Testing, Priority Preemption Testing, Synchronization Testing, Concurrency Testing, Hard Deadline Testing, Soft Deadline Testing, Fault Injection Testing và Timing Analysis Testing.

Kết quả thực nghiệm cho thấy:

- Scheduler hoạt động đúng cơ chế Priority-Based Preemptive Scheduling.
- Các task đáp ứng deadline trong điều kiện vận hành bình thường.
- Hệ thống phát hiện các trường hợp vi phạm deadline thông qua cơ chế Fault Injection.

Trong Baseline Run, toàn bộ Test Suite đạt Pass. Trong Fault Injection Run, Fault_Injector_Task tạo ra 50 sự kiện CPU_OVERLOAD, khiến thời gian thực thi của Control_Task tăng lên 60 ms, vượt quá deadline thiết kế 50 ms và dẫn tới 9 sự kiện DEADLINE_MISS được hệ thống phát hiện và ghi nhận. Phân tích các Runtime Metrics như Execution Time, WCET, Deadline Miss Count và Context Switch Count cho thấy các cơ chế hoạt động đúng thiết kế và hiệu quả.

7.2. Đánh giá mức độ hoàn thành mục tiêu đề tài

Dựa trên các kết quả thực nghiệm trình bày trong Chương 6, có thể nhận thấy đề tài đã hoàn thành các mục tiêu đề ra.

Bảng 7.1. Đánh giá mức độ hoàn thành mục tiêu đề tài

Mục tiêu	Mức độ hoàn thành
Xây dựng môi trường mô phỏng RTOS	Hoàn thành
Triển khai đa task	Hoàn thành
Scheduler Testing	Hoàn thành

Priority Scheduling Testing	Hoàn thành
Priority Preemption Testing	Hoàn thành
Synchronization Testing	Hoàn thành
Concurrency Testing	Hoàn thành
Hard Deadline Testing	Hoàn thành
Soft Deadline Testing	Hoàn thành
Fault Injection Testing	Hoàn thành
Timing Analysis	Hoàn thành

Kết quả đánh giá cho thấy toàn bộ các mục tiêu kỹ thuật đã được triển khai và kiểm chứng thông qua các Test Case cụ thể. Các kết quả PASS/FAIL thu được phù hợp với hành vi mong đợi của hệ thống.

7.3. Hạn chế của đề tài

Mặc dù đã đạt các mục tiêu đề ra, đề tài vẫn còn một số hạn chế:

- Hệ thống hiện được triển khai trên môi trường FreeRTOS Windows Simulator, chưa phản ánh đầy đủ các yếu tố phần cứng nhúng thực tế như bộ nhớ giới hạn, ngoại vi và các ràng buộc tài nguyên vật lý.
- Cơ chế Fault Injection mới tập trung vào CPU overload và deadline violation, chưa mở rộng đến lỗi bộ nhớ, thiết bị ngoại vi hoặc lỗi truyền thông giữa các thành phần.
- Việc phân tích thời gian thực hiện chủ yếu dựa trên runtime log và các metrics thu thập, chưa tích hợp công cụ chuyên dụng phục vụ phân tích thời gian thực sâu hơn.

7.4. Hướng phát triển

Đề tài có thể được mở rộng theo các hướng:

- Triển khai hệ thống trên phần cứng nhúng thực tế (STM32, ESP32) kết hợp FreeRTOS.
- Mở rộng số lượng task và kịch bản kiểm thử để mô phỏng hệ thống quy mô lớn hơn.
- Bổ sung cơ chế Synchronization và các thuật toán lập lịch nâng cao.
- Mở rộng Fault Injection để mô phỏng lỗi bộ nhớ, lỗi truyền thông và lỗi phần cứng.

- Xây dựng hệ thống trực quan hóa dữ liệu runtime theo thời gian thực.
- Tích hợp công cụ phân tích Timing chuyên sâu để đánh giá hiệu năng hệ thống.
- Xây dựng framework kiểm thử tự động cho các ứng dụng FreeRTOS.

7.5. Tổng kết chương

Chương 7 đã tổng kết toàn bộ quá trình thực hiện đề tài, đánh giá mức độ hoàn thành các mục tiêu, phân tích các hạn chế và đề xuất hướng phát triển.

Kết quả thực nghiệm cho thấy hệ thống đáp ứng tốt các yêu cầu kiểm thử của FreeRTOS, thực hiện đúng các cơ chế Priority Scheduling, Priority Preemption, Synchronization và Deadline Monitoring. Đồng thời, hệ thống phát hiện các vi phạm deadline thông qua Fault Injection và ghi nhận đầy đủ các Runtime Metrics phục vụ phân tích.

Nhìn chung, đề tài đã hoàn thành các mục tiêu đề ra và tạo nền tảng cho việc nghiên cứu, triển khai và phát triển các hệ thống kiểm thử RTOS trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] FreeRTOS, WIN32-MSVC Simulator Demo. Truy cập tại: <https://github.com/FreeRTOS/FreeRTOS/tree/main/FreeRTOS/Demo/WIN32-MSVC>
- [2] Jane W. S. Liu, Real-Time Systems, Prentice Hall, 2000.
- [3] Qing Li, Caroline Yao, Real-Time Concepts for Embedded Systems, CMP Books, 2003.
- [4] Tammy Noergaard, Embedded Systems Architecture: A Comprehensive Guide for Engineers and Programmers, Elsevier, 2012.
- [5] Microsoft Corporation, Visual Studio 2022 Documentation. Truy cập tại: <https://learn.microsoft.com/visualstudiob>
- [6] Python Software Foundation, Python Documentation. Truy cập tại: <https://docs.python.org/3>
- [7] Git Development Team, Git Documentation. Truy cập tại: <https://git-scm.com/doc>
- [9] Tài liệu hướng dẫn đồ án FreeRTOS do giảng viên cung cấp, 2025.